



SAPIENZA
UNIVERSITÀ DI ROMA

Sapienza University of Rome

Dipartimento di Ingegneria informatica, automatica e gestionale
PhD in Artificial Intelligence

THESIS FOR THE DEGREE OF DOCTOR OF COMPUTER SCIENCE

Learning Symbolic Planning Models from Images

Thesis Advisor

Prof. Alfonso Emilio Gerevini

Correlatore

Federico Cerutti

Candidate

Aymeric Gaspard Barbin

1996923

Academic Year MMXIX-MMXXII (XXXV cycle)

Abstract

La costruzione di modelli simbolici per la pianificazione, in particolare nel linguaggio PDDL, è ancora una sfida nell’Intelligenza Artificiale, poiché richiede progettazione manuale.

Questa tesi esplora come apprendere automaticamente, e senza supervisione, modelli d’azione PDDL da input visivi grezzi, con attenzione all’efficacia della pianificazione e all’interpretabilità semantica. Il lavoro si basa su Latplan, un sistema che apprende rappresentazioni simboliche da immagini tramite codifica in uno spazio latente. Nella prima parte del lavoro di tesi, viene migliorato Latplan senza apportare modifiche architetturali, usando regolarizzazione fuzzy e ottimizzazione degli iperparametri. Questi metodi migliorano le prestazioni in domini semplici, ma non risolvono i limiti principali di Latplan: scarsa interpretabilità delle azioni e piani generati irrealistici (“allucinati”). Proponiamo quindi **R-latplan**, un’estensione architetturale del sistema con etichettatura deterministica delle azioni basata su differenze visive, migliorando le prestazioni anche in presenza di rumore. Tuttavia, la scalabilità resta un problema per l’elevato numero di azioni generate. Perciò, per ridurre lo spazio delle azioni, nel lavoro di tesi si sono investigate diverse tecniche: inizialmente vengono usati Alberi di Decisione per identificare pattern di effetti e precondizioni nelle azioni generate da **R-latplan**. Tuttavia questo approccio non consente di produrre modelli di azioni che supportano una pianificazione efficiente. Successivamente, si propone un approccio denominato RC-Latplan, in cui vengono clusterizzate le azioni di basso livello di **R-latplan** in gruppi semanticamente equivalenti, ciascuno dei quali è trasformato in una singola azione PDDL. Un’analisi sperimentale dimostra che RC-Latplan riduce considerevolmente lo spazio delle azioni e preserva la loro interpretabilità. Sebbene RC-Latplan generalmente risulti in modelli di azioni che sono meno performanti dei modelli di **R-latplan**, una delle varianti considerate in RC-Latplan genera una rappresentazione del dominio logicamente equivalente a quella di **R-latplan** (ma più compatta).

Infine la tesi esplora una ricerca complementare in cui ho contribuito nello studio delle Visual Reward Machines (VRMs), un framework per apprendere automi simbolici da osservazioni visuali, al fine di migliorare l’apprendimento in ambienti con reward sparso.

Abstract

The construction of symbolic planning models, particularly in the PDDL language, remains a longstanding challenge in Artificial Intelligence due to its reliance on manual domain engineering.

This thesis explores how such models can be learned automatically and without supervision, from raw visual input. We focus on learning PDDL-style action models for Classical Planning, with an emphasis on planning effectiveness and semantic interpretability. Our work builds on Latplan, a system that learns symbolic representations from images via deep latent-space encoding. In the first part of the thesis we improve Latplan without architectural changes by enforcing logical invariants through fuzzy logic regularization and tuning hyperparameters. While these methods yield improvements in simpler domains, they do not overcome Latplan’s two key limitations: the lack of action interpretability and poor planning performance due to hallucinated plans. To address this, we propose **R-latplan**, an architectural extension introducing deterministic action labeling based on visual differences. This enhances planning performance and aligns learned actions with high-level behaviors, even under noise. However, scalability remains a challenge due to the proliferation of generated actions, which increases planning time. To compress the action space, we investigate various techniques, first we use Decision Trees in order to identify patterns among the effects and preconditions of **R-latplan** and translate these patterns into higher level actions, but this approach does not enable efficient action models; then we propose **RC-latplan**, which clusters low-level actions into semantically coherent groups, each translated into a PDDL action. Experiments show that **RC-latplan** greatly reduces domain size while preserving interpretability. Although it slightly underperforms **R-latplan**, a variant, clustering actions with the same effects achieves logical equivalence with **R-latplan**.

Finally, the thesis explores a complementary research in which I worked, on Visual Reward Machines (VRMs), a framework for learning symbolic automata from visual observations to improve learning in sparse-reward environments.

Keywords: artificial intelligence, neuro-symbolic planning

Contents

List of Figures	iv
List of Tables	vi
Acronyms	viii
1 Introduction	1
2 Background and Related Work	5
2.1 Classical Planning	5
2.1.1 Classical Propositional Planning	8
2.1.2 The Knowledge Acquisition Bottleneck in Planning	9
2.2 Symbol Grounding	9
2.2.1 Using Pixel Changes between images to find out symbols	10
2.2.2 Using objects-recognition systems to find out symbols	10
2.2.3 Variational Auto-encoder, and Discrete Variational Auto- encoder	11
2.3 Action Model Acquisition	15
2.3.1 Combinatorial approaches	15
2.3.2 Probabilistic Approaches	17
2.4 Latplan: a domain-independent, image-based learner for classical planning .	17
2.4.1 Overview of the architecture	17
2.4.2 More details about the loss function	19
2.4.3 Training Data	20
2.4.4 Latplan previous versions of the AMA	22
2.5 How does Latplan fail?	22
2.5.1 Planning performances	22
2.5.2 Hallucinations	24
2.5.3 Lack of explainability of the action model	25
2.6 Why Latplan fails	25
3 Improving Latplan through invariant enforcing and hyperparameters tuning	26
3.1 Invariants enforcing	26
3.1.1 What are the invariants?	26
3.1.2 Why did we make use of invariants?	27
3.1.3 Searching for invariants of interest	27
3.1.4 Applying the constraints using Fuzzy logic	29
3.2 Modifying the hyperparameters	31

3.2.1	Hyperparameters search	31
3.2.2	Reducing the number of possible actions	32
3.2.3	Reducing the size of the learned state space.	32
4	R-latplan	34
4.1	Overview of the approach	34
4.2	Methodology	34
4.3	Research questions	37
4.4	Benchmarks	38
4.5	Experimental setup	39
4.6	Results	41
4.7	Related Works	43
5	RC-latplan: compressing the action model	45
5.1	Intuition	45
5.2	Method that did not work: Using Decision Trees	48
5.2.1	Intuition	48
5.2.2	Learning via Decision trees: a classification task	48
5.2.3	Overview of the Approach	51
5.3	RC-latplan with Hierarchical Clustering	54
5.3.1	The Hierarchical Clustering Method	54
5.3.2	Testing different hyperparameters for the clusterings	59
5.3.3	Translation of the clusters into PDDL actions	60
5.3.4	Results	66
5.4	RC-latplan with Clustering on the effects	69
5.4.1	Proving logical equivalence	70
5.4.2	Results	73
6	Visual Reward Machines	75
6.1	Introduction	76
6.2	Background	77
6.3	Related work	78
6.4	Method	79
6.4.1	Visual Reward Machine definition	79
6.4.2	VRM implementation with neural networks	80
6.5	Preliminary experiments	81
6.5.1	Visual Minecraft Environment	82
6.5.2	Offline symbol grounding	83
6.5.3	Reinforcement Learning and online grounding	84
6.5.4	Learning the machine from imperfectly grounded symbols	86
6.6	Conclusion	87
7	Conclusions and Future Works	88
	Bibliography	92

List of Figures

2.1	A Towers of Hanoi problem instance	6
2.2	Two consecutive states in the Towers of Hanoi domain. Disks Y (yellow), R (red), G (green), and B (blue) are ordered from smallest to largest. Pegs peg1-peg4 are shown from left to right.	8
2.3	Architecture of a Variational Autoencoder (VAE). The encoder maps the input to a distribution over latent variables, parameterized by μ and σ . A latent sample z is drawn using the reparameterization trick and passed to the decoder to reconstruct the input.	12
2.4	Sampling from a discrete distribution (left) using Gumbel-Max; and its continuous relaxation using the Gumbel-Softmax trick (right).	14
2.5	Effect of temperature parameter λ on Binary Concrete distribution. As $\lambda \rightarrow 0$, the output approaches a binary step function.	15
2.6	High-level illustration of Latplan [9].	18
2.7	Overview of the action model learning process from Latplan framework, image from the original Latplan paper by Asai and Fukunaga (2018). i : indexes a particular transition. x_0^i and x_1^i are observed input images (<i>i.e.</i> , before and after an action), z_0^i and z_1^i are their latent propositional states, z_2^i and z_3^i are next state predictions, after action application or regression.	21
2.8	Hallucinated output from Latplan decoder	24
3.1	Reducing the latent state size, leads to a better state reconstruction from the VAE. From left to right: input image to the VAE, latent state representation of the VAE, outputted image of VAE decoder, difference between input and output.	33
3.2	Summary of our different improvement on existing Latplan architecture. Green cells indicate positive results, red cells negative results and yellow cells Future Work.	33
4.1	An illustration of two states in the Visual DFA connected by a transition. . .	35
4.2	An illustration of the case where the transition identifier function labels erroneously two versions of the same transitions with different labels (ID14 and ID15 in the figure), due to visual noise.	36
4.3	The overall workflow of R-Latplan.	36
4.4	Comparison of dependency graphs in Latplan and R-Latplan. A grey node means an observable variable, a white node means a latent variable.	37
4.5	Success number of tested executions (maximum success is 3) on the longest possible paths from the derived Visual DFAs on Exp1: Crisp and Complete Visual DFAs and Exp2: Noisy and Complete Visual DFAs	40
4.6	Not visually correct plan generated by (vanilla) Latplan	41

4.7	Path found by FD from a PDDL generated by R-latplan in Data3 (with the 1-step transition/solution pruned out)	42
4.8	Average error (log-scale) and standard deviation in predicting the successor state after applying an action	42
5.1	A simple decision tree with Precondition 1 as the root, Precondition 2 as intermediate nodes, and the application (or not) of Effect 5 as the leaves. . .	49
5.2	Decision Tree predicting the effect "add z2" for high-level action <i>move(d1, peg2)</i> in Hanoi.	53
5.3	Dendrogram of clustering low level actions	58
5.4	Success number of tested executions (maximum success is 3) on the longest possible paths from the derived Visual DFAs on Exp1: Crisp and Complete Visual DFAs and Exp2: Noisy and Complete Visual DFAs	67
6.1	a) Visual Minecraft environment. b) Non-Markovian task specification for the Minecraft agent: the agent has to collect the pickaxe and the gem (in any order) and then go to the door, while always avoiding the lava. c) Implementation of a Visual Reward Machine with convolutional and recurrent neural modules.	80
6.2	Results obtained by exploiting the reward machine structure to learn the symbol grounding function offline for the Visual Minecraft environment. a) Visual Reward Machine accuracy over sequences. b) Symbol grounding accuracy over single images.	83
6.3	Results obtained by exploiting the reward machine structure to learn the symbol grounding function for the Visual Minecraft environment. a) Training rewards. b) Reward prediction accuracy during training. c) Symbol grounding accuracy during training.	85
6.4	Learning DFA from traces composed of imperfectly grounded symbols: results on the 7 Tomita languages.	86

List of Tables

2.1	Comparison of Methods for Learning Action Models	23
2.2	Performance comparison of AMA4+ using Blind, LAMA, LM-Cut, and M&S.	24
2.3	Performance comparison of AMA4+ using Blind, LAMA, LM-Cut, and M&S, on Towers Of Hanoi domain, in different configuration (d: number of disks, t: number of towers)	24
3.1	Number of solutions found per domain and problem size (7-steps and 14- steps), with and without enforcing invariants (each cell shows X/Y with, X: number solutions found, Y: number of problems tested). “all-at-once” means that all the invariants by FD were used in the loss function.	30
3.2	State Variance with and without enforcing invariants.	31
3.3	Comparison of χ^2 scores for Sokoban and Blocksworld.	31
4.1	Number of unique states and transitions.	38
4.2	Image resolutions in pixels	40
4.3	lengths of paths found with R-latplan for Exp3,4	42
4.4	Results of Exp5 showing R-latplan’s robustness against errors produced by the transition identifier function.	43
4.5	Gap analysis with the existing state-of-the-art.	44
5.1	Comparison of clustering low-level actions of the high-level action move(d1, d2), under different conditions. #cluster: number of clusters, #con_pre: number of contradictions among the preconditions, #con_eff: number of con- tradictions among the effects.	59
5.2	Number of actions in R-latplan vs RC-latplan	66
5.3	Planning metrics: R-latplan vs RC-latplan	67
5.4	Comparing DFA coverage of R-latplan and RC-latplan	68
5.5	Number of actions in R-latplan vs RC-latplan with clustering of the effects .	73
5.6	Planning metrics: R-latplan vs RC-latplan	73
5.7	Comparing DFA coverage of R-latplan and RC-latplan with same effects clustering	74

Acronyms

AE Auto-Encoder

AI Artificial Intelligence

AMA Action Model Acquisition

BC Binary Concrete

DFA Deterministic Finite Automaton

DT Decision Tree

ELBO Evidence Lower Bound

FD Fast Downward

GS Gumbel-Softmax

KL Kullback-Leibler divergence

PDDL Planning Domain Description Language

RL Reinforcement Learning

SAE State Autoencoder

SGP Symbol Grounding Problem

STRIPS Stanford Research Institute Problem Solver

ToH Towers of Hanoi

VAE Variational Auto-Encoder

Chapter 1

Introduction

In Artificial Intelligence (AI), an agent is an entity that perceives its environment through sensors and acts upon it using actuators to achieve specific goals [74]. A *planning model* for such an agent is a representation that, at a minimum, specifies two essential components: the state in which the agent currently resides and the action or actions that it can or should take in that state [31, 79]. Various types of planning models exist, differing both in the way they are constructed and how they are used by the agent. In this thesis, we focus on *action models* [33], which play a central role in automated planning. We also explore, as a complementary line of work, a model based on Reinforcement Learning (RL) [79].

In AI, *Planning* [33] refers to the process of generating sequences of actions that enable an agent to reach specific goals, given a symbolic representation of the environment and the agent’s available actions. An *action model* formally describes each action in terms of preconditions — conditions under which the action is applicable — and effects, *i.e.*, how the state changes as a result of the action. Traditionally, these models are handcrafted by human experts, a process that is both time-consuming and error-prone.

Recent research [38] [2] [7] [16] has shifted toward automatically learning action models directly from sensory data, often with minimal or no human supervision. This is a promising yet challenging direction, for various reasons: (i) training data may be limited or noisy; (ii) the learned models may not be suitable for planning — e.g., they may produce actions with contradictory preconditions or effects; and (iii) the learned actions may lack semantic coherence or interpretability. Additionally, different planning paradigms exist (*e.g.*, contingent planning [70], probabilistic planning [17], hierarchical planning [26]), each introducing different challenges when learning their corresponding action model.

In this thesis, we focus on *Classical Planning* [31] — a simplified planning setting where the environment is fully observable, static, deterministic, and controlled by a single agent. We represent planning problems using the Planning Domain Description Language (PDDL) [1], where each action is defined by a set of preconditions and effects. Our goal is to advance the state-of-the-art in *automated PDDL model learning* without human supervision, using only raw images of the environment as input.

Several recent works have addressed the problem of learning planning models from visual input [38] [2] [91] [59]. Our approach distinguishes itself in three important ways: (1) it learns a *subsymbolic representation* based on the latent space of a neural network; (2) it does not require explicit human expert knowledge; and (3) it produces *efficient and reliable actions* — actions that are usable in visual planning benchmarks, correspond to meaningful high-level agent actions, and avoid the hallucination artifacts common in generative models.

Our work builds upon an existing model, *Latplan* [7], which learns PDDL-style actions from images without supervision. However, Latplan exhibits several limitations: it struggles with problems that require many planning steps; it often produces hallucinated objects (*e.g.*, duplicated or missing entities); and its learned actions usually lack semantic alignment with the agent’s high-level behaviors.

First, we attempt to improve Latplan without altering its architecture. We introduce symbolic constraints into the loss function during training by extracting *invariants*—logical constraints that capture relationships among PDDL variables — from the learned domain [3]. These invariants are derived using tools like TIM and Fast Downward [32, 41], and enforced through fuzzy logic regularization [37, 94]. While this approach improves planning performance in simple domains (*e.g.*, 8-puzzle with MNIST digits), it has a limited impact in more complex benchmarks.

We also conduct a grid search over key hyperparameters, such as the latent state size and loss term weights. Increasing the weight of the next-state prediction term leads to better planning outcomes and more invariant extraction in Sokoban and Blocksworld. Additionally, we explore setting the number of actions to match the number of grounded high-level actions in the domain and scaling the latent size according to the number of observable states. Although reducing the number of actions has no significant impact, reducing latent size improved robustness to noise, yielding better reconstructions and visually accurate plans.

Next, we propose an architectural modification of Latplan, resulting in a new model, R-latplan [13]. This version introduces a mechanism to associate each state transition

with an action label, using a transition identifier based on image differences. This simple change greatly improves **Latplan**’s capabilities: **R-latplan** finds optimal and visually correct plans across all benchmark domains, even under noise (*i.e.*, corrupted images or incorrect transition labels). Crucially, the learned PDDL actions consistently correspond to the agent’s high-level actions. A limitation of **R-latplan** is that the number of learned actions grows with the size of the state space, increasing planning time.

To mitigate this scalability issue, we explore compressing the action space by generating new PDDL actions from **R-latplan**’s action database. Our first attempt is a *top-down translation*: for each high-level action, we group corresponding low-level actions and train two decision trees to infer appropriate preconditions and effects. This approach has limited success since it can only solve very short planning tasks (up to 3 steps) and often generates visually incorrect plans.

Our final and most successful approach is a *bottom-up translation*: we apply clustering to the low-level actions and define new PDDL actions for each cluster — termed *clustered actions*. Clustered actions improved planning efficiency in terms of time, memory, and search effort. Importantly, each cluster corresponded to a specific high-level action, enabling a semantic interpretation of the learned actions. Nevertheless, after having conducted an experiment to test the coverage of the representation in terms of plans produced compared to the total number of possible plans in each domain, we notice that the coverage of **RC-latplan** is lower than **R-latplan**. This motivates the design of a new action model design where we cluster low-level actions of same effects. We prove that **RC-latplan** with this type of clustering is logically equivalent to **R-latplan** and we validate this demonstration with experimental results.

In a parallel line of research, conducted in the context of the PhD course on *Non-Markov Decision Processes and Reinforcement Learning*, we explore the use of Reinforcement Learning to improve agent behavior by leveraging symbolic structures extracted from visual environments. Specifically, we introduce the concept of *Visual Reward Machines (VRMs)*, where automata-based reward functions [46] — represented as Deterministic Finite Automaton (DFA) — are learned from visual sequences. These DFAs capture the high-level structure of tasks and are used to augment the RL agent’s input. By concatenating the DFA state representation with the agent’s visual state, we provide additional structured context. This enhances learning efficiency and generalization, particularly in sparse-reward or temporally extended tasks. While distinct from the work on PDDL action learning, this work

shares the overarching goal of extracting symbolic structure from raw visual input to guide decision-making.

This thesis is organized as follows. Chapter 2 reviews background knowledge and related work in AI planning and model learning. Chapter 3 presents our initial efforts to improve **Latplan** via invariant enforcement and hyperparameter optimization. Chapter 4 introduces **R-latplan**, our modified architecture that overcomes **Latplan**’s main limitations. Chapter 5 describes **RC-latplan**, which compresses and improves the learned action models through clustering. Chapter 6 presents the subsidiary work on Visual Reward Machines. Finally, Chapter 7 concludes the thesis and outlines directions for future research.

Chapter 2

Background and Related Work

To learn symbolic planning models from images, one must bridge the gap between high-dimensional sensory data and the discrete, structured representations required by Classical Planning systems. Classical Planning traditionally assumes symbolic models — defining states and actions — are manually provided by human experts, which limits the scalability of such systems. This challenge, known as the Knowledge Acquisition Bottleneck [22], motivates the development of methods that can automatically extract symbolic representations from raw data. In this chapter, we first introduce the fundamentals of Classical and Propositional Planning, followed by an overview of recent approaches addressing the bottleneck through Symbol Grounding and Action Model Acquisition. We conclude with a detailed review of **Latplan**, a representative neuro-symbolic framework that aims to learn STRIPS-compatible [27] planning models directly from images.

2.1 Classical Planning

In Artificial Intelligence, planning refers to the process of generating a sequence of actions, or *plan*, that transitions a system from an initial state to a goal state in order to solve a given task [44]. Classical Planning [31] is a well-established subfield of planning that makes several simplifying assumptions: the environment is deterministic, fully observable, static, and the effects of actions are known and instantaneous. Under these assumptions, planning reduces to a combinatorial search problem over discrete state transitions.

The Towers of Hanoi [74] puzzle offers a well-known example of a Classical Planning problem.

Example 1. The Towers of Hanoi (Towers of Hanoi (ToH)) problem is a mathematical

puzzle involving P pegs and D disks of distinct sizes. The following rules apply:

- Only one disk may be moved at a time.
- A disk may only be placed on a larger disk or on an empty peg.

The planning task consists of finding a valid sequence of moves that transfers all disks from one peg to another, obeying the rules, to reach a specific goal configuration. Figure 2.1 shows an example of such a task.

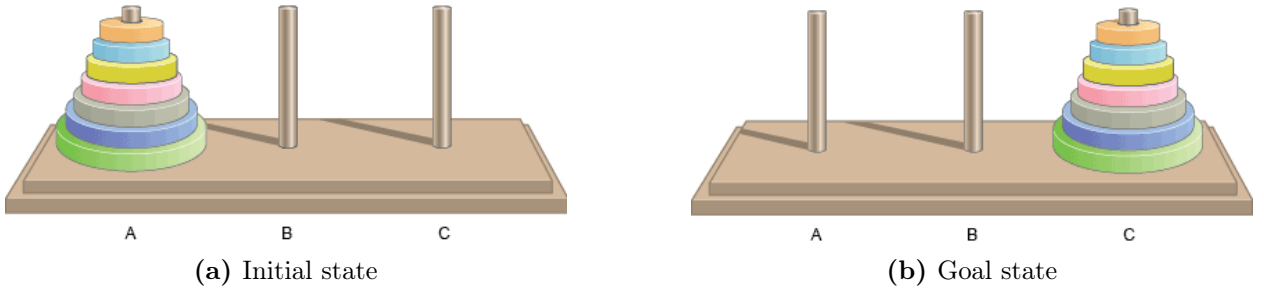


Figure 2.1: A Towers of Hanoi problem instance

Formally, a Classical Planning problem is defined as follows:

Definition 2.1.1. A **Classical Planning problem** [31] is defined by a tuple $\langle S, A, s_0, G, \pi \rangle$, where:

- S is a finite set of states, each describing a complete configuration of the environment using logical propositions over objects in the domain.
- A is a set of actions, each defined by preconditions and effects.
- $s_0 \in S$ is the initial state.
- $G \subseteq S$ is a set of goal states (usually described via a goal condition).
- $\pi : S \times A \rightarrow S$ is the deterministic transition function, such that $\pi(s, a)$ returns the successor state resulting from applying action a in state s .

For describing planning problems, we adopt the Planning Domain Description Language (PDDL) [1], a widely used modeling language for automated planning. In particular, we focus on the Stanford Research Institute Problem Solver (STRIPS) formalism, which defines a subset of PDDL well-suited for classical planning tasks.

Definition 2.1.2. STRIPS Formalism. Let P be a set of propositional variables (also called facts). Each action $a \in A$ is defined as a 4-tuple:

$$a = \langle \text{POS}(a), \text{NEG}(a), \text{ADD}(a), \text{DEL}(a) \rangle,$$

where:

- $\text{POS}(a) \subseteq P$ are the positive preconditions (facts that must be true for a to be applicable),
- $\text{NEG}(a) \subseteq P$ are the negative preconditions (facts that must be false for a to be applicable),
- $\text{ADD}(a) \subseteq P$ are the add effects (facts that become true after executing a),
- $\text{DEL}(a) \subseteq P$ are the delete effects (facts that become false after executing a),
- $\text{POS}(a) \cap \text{NEG}(a) = \emptyset$ and $\text{ADD}(a) \cap \text{DEL}(a) = \emptyset$.

Example 2 (Towers of Hanoi in STRIPS). To illustrate the STRIPS formalism, we use the Towers of Hanoi (ToH) problem with 4 disks and 4 pegs. Figure 2.2 shows two consecutive states in this domain, where each state is described using a set of grounded predicates. The predicates used are:

- **clear(x)** — true if no disk is on top of object x ,
- **on(x, y)** — true if object x is directly on top of object y .

For instance, **on(G, peg1)** indicates that the green disk is on peg 1.

The grounded action for the transition is: **move(G, B)**, representing the movement of the green disk onto the blue disk. This action can be encoded in STRIPS as follows:

- **Positive preconditions:**

- **clear(G)**
- **clear(B)**

- **Add effects:**

- **on(G, B)**

- **Delete effects:**

- `clear(B)`

- **Negative preconditions:**

- None ($\emptyset$)



Figure 2.2: Two consecutive states in the Towers of Hanoi domain. Disks Y (yellow), R (red), G (green), and B (blue) are ordered from smallest to largest. Pegs `peg1`-`peg4` are shown from left to right.

2.1.1 Classical Propositional Planning

Classical Propositional Planning [28] is a restricted form of Classical Planning in which all elements of the planning model — states, preconditions, and effects — are represented using propositional variables. These variables are atoms, interpreted as binary facts that are either true or false. This results in a propositional (*i.e.*, Boolean) state space where each state is represented by a fixed-length vector over $\mathbb{B} = \{0, 1\}$.

Each planning state can be represented as a binary vector $\mathbf{s} = (s_1, s_2, \dots, s_n)$ that instantiates a corresponding vector of propositional variables $\mathbf{z} = (z_1, z_2, \dots, z_n)$, where each $s_i \in \{0, 1\}$ indicates whether the atomic fact represented by z_i holds in that state. Actions are defined by propositional preconditions and effects, which operate over these variables using logical conjunctions and negations.

Definition 2.1.3 (Grounding). In automated planning **grounding** refers to the process of instantiating abstract predicates or action schemas with specific objects from the planning domain. In propositional STRIPS planning, for a domain defined over n propositional

variables $z_1, \dots, z_n$, a *grounded state* is a binary vector $s \in \{0, 1\}^n$ indicating the truth value of each variable. A *grounded action* is defined by a tuple of four binary vectors $(pre^+, pre^-, eff^+, eff^-)$, each in $\{0, 1\}^n$:

- pre^+ : the variables that must be true (positive preconditions),
- pre^- : the variables that must be false (negative preconditions),
- eff^+ : the variables to be set to true (add effects),
- eff^- : the variables to be set to false (delete effects).

Grounding is typically performed either manually or using domain compilers. However, in our work, we aim to bypass this manual engineering effort by learning grounded symbolic representations directly from raw data, such as images.

2.1.2 The Knowledge Acquisition Bottleneck in Planning

States and actions representations in Planning are traditionally encoded by human experts. This need of humans involvement results in the Knowledge Acquisition Bottleneck ([22]) and refers to the high amount of human effort required to design these representations.

What if we do not have access to human resources? Can we learn symbolic representations directly from raw data and therefore bypass the Knowledge Acquisition Bottleneck? The ideal machine learning system would learn the representation automatically without any human help. In order to better clarify what is the Knowledge Acquisition Bottleneck of designing planning representations (for the states and actions), we illustrate in the following sections, two major steps (introduced by Asai et. al in [9]) to achieve in this endeavor: Symbol Grounding and Action Model Acquisition.

2.2 Symbol Grounding

In this thesis we refer to Symbol Grounding — not to be confused with the process of grounding variables in Classical Planning, see definition 2.1.3 — as the process that generates symbols from raw data [78]. In Machine Learning, in particular when dealing with images or other types of sensory information, this process can be automatized through the use of various tools that we review in this section. In particular, in our work, Symbol Grounding

refers to mapping high-dimensional vectors representing images of a planning problem into planning symbols, such as, in PDDL: objects, predicates, propositions, actions, etc. In the following, we present different techniques, from papers related to our work, of discovering symbols from images.

2.2.1 Using Pixel Changes between images to find out symbols

In [38], the authors present an approach for automatically generating symbolic action definitions using unlabeled pairs of images representing transitions. They identify objects and their locations by detecting pixel changes between the two images of a transition. More specifically, two types of symbols are discovered: *locations* and *objects*. *Locations* correspond to the locations where objects are located in the image and are discovered by iteratively computing the pixel differences over all the transitions of a dataset and correspond to the smallest areas where pixel changes overlap throughout the dataset. *Objects* correspond to the pixel areas representing objects and are discovered similarly as for the location (pixel changes).

This method has two major drawbacks. First, it is very likely that it is not robust against noise since for a given *object* o_i , if one pixel value changes in one image, then an additional object o_j would be identified (furthermore the authors specifically indicate that their method was tested on non-noisy images), second, since the method to retrieve the objects and locations requires to parse the entire dataset and the size of the dataset grows exponentially with the complexity of the problem (or with the level of noise), the computing time itself is exponential with regards to the dataset complexity.

2.2.2 Using objects-recognition systems to find out symbols

Objects-recognition vision systems (also known as *image parsers*) map images into a symbolic description of the objects. In [59] the authors introduce the O2D (Object in 2D) language, where a given parser maps the image to the different objects of the scene. Generally, such parser uses an object-tuple representation of the form $\langle type(o), loc(o), bb(o), att(o) \rangle_o$ which encodes the type, location, bounding boxes and attributes of an object o , instead, the O2D language has the signature $\Sigma = (C, U, R)$, where C is the set of constant symbols for representing the objects and shapes, U is the set of unary predicates that describe the types and characteristics of the objects and R the set of binary predicates describing the relation between object, *e.g.*, $left(o1, o2)$.

In reality, the authors do not use a parser directly on the images but instead, use the output of the PDDL_{Gym} [76] *step* function, which already provides high-level, symbolic information describing the content of the image. Even though a well trained image parser can partially cope with the noise problem evoked in section 2.2.1, parsers are still subject to outputting errors in the image predicates, such errors can be very problematic for the Action Model Acquisition part (see section 2.3).

2.2.3 Variational Auto-encoder, and Discrete Variational Auto-encoder

The two previous methods deal with obtaining a human-like representation of the states. But neural networks can learn, without any supervision, a latent representation (not understandable by humans) that can also be used as a state representation. In the following we introduce such a neural network, called Variational Auto-Encoder (VAE) and its discrete form (Discrete VAE) that we used to learn a binary representation of images.

Variational Autoencoders

A Variational Auto-Encoder (VAE) [53] is a class of generative models that extends the standard Auto-Encoder (AE) by introducing a probabilistic interpretation of the latent space. Unlike a traditional autoencoder, which encodes an input into a single deterministic latent vector, the VAE learns to map inputs to a distribution over latent variables. This formulation enables the model to generate novel data samples by sampling from this learned latent distribution, thereby supporting generative capabilities.

More concretely, the encoder network in a VAE outputs the parameters of a probability distribution over the latent variables — typically a Gaussian distribution characterized by a mean vector μ and a standard deviation vector σ . Instead of directly passing these parameters through the decoder, latent variables are sampled as follows:

$$z = \mu + \sigma \cdot \epsilon, \quad \epsilon \sim \mathcal{N}(0, \mathbf{I}), \quad (2.1)$$

where ϵ is an auxiliary noise variable drawn from a standard normal distribution. This formulation, known as the *reparameterization trick*, ensures that the sampling operation is differentiable with respect to the parameters μ and σ , which is critical for enabling gradient-

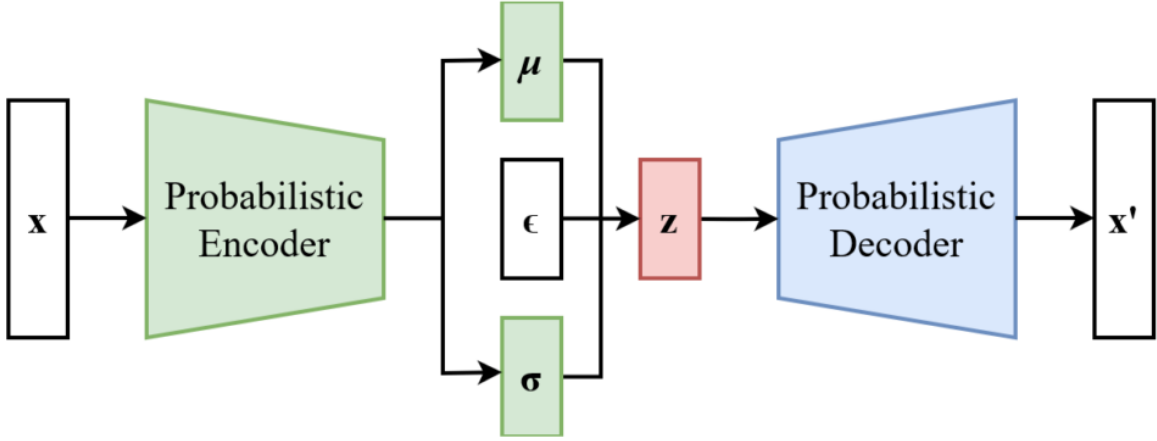


Figure 2.3: Architecture of a Variational Autoencoder (VAE). The encoder maps the input to a distribution over latent variables, parameterized by μ and σ . A latent sample z is drawn using the reparameterization trick and passed to the decoder to reconstruct the input.

based optimization during training.

The decoder then maps the sampled latent variable z back to the data space, aiming to reconstruct the original input. Training a VAE involves maximizing a variational lower bound (the Evidence Lower Bound, or ELBO) on the data log-likelihood, which balances reconstruction accuracy and the closeness of the learned latent distribution to a prior (usually a standard normal distribution).

Figure 2.3 illustrates the VAE architecture, emphasizing how the latent variable z is derived via the reparameterization trick.

The loss function of a Variational Auto-Encoder (VAE) consists of two primary components:

- **Reconstruction Loss:** Similar to the objective in a standard AE, this term measures how accurately the decoder reconstructs the input data from the latent representation. It is typically formulated as the negative log-likelihood of the input given the latent variable, and in the case of continuous input data, it often corresponds to a mean squared error or Gaussian likelihood.
- **Kullback-Leibler divergence Divergence Loss [87]:** This term quantifies the divergence between the approximate posterior distribution $q(z | x)$ learned by the encoder and a predefined prior distribution $p(z)$ over the latent space — typically a standard multivariate normal distribution $\mathcal{N}(0, \mathbf{I})$. By minimizing this divergence, the model

regularizes the latent space and encourages the encoded representations to conform to the prior, facilitating meaningful sampling and generalization.

In addition to enabling probabilistic modeling, a VAE possesses several desirable properties over a traditional autoencoder:

- **Meaningful Sampling:** because the latent space is structured to follow a known prior distribution, it becomes possible to sample latent vectors at random and decode them into coherent, semantically meaningful outputs that resemble the training data.
- **Smooth Interpolation:** the structure of the latent space allows for smooth transitions between different data points, such that, small changes in the latent space result in gradual and coherent output changes.

In this work, we primarily exploit the capacity of the VAE to learn the underlying distribution of latent variables. Specifically, our goal is to obtain a compact and discrete representation of states, in which each state is encoded using a set of binary latent variables. These binary variables are assumed to follow a discrete probability distribution, namely, the Bernoulli distribution.

Definition 2.2.1 (Bernoulli Distribution). A random variable X is said to follow a Bernoulli distribution with parameter $p \in [0, 1]$, denoted $X \sim \mathcal{B}(p)$, if it takes value 1 with probability p and value 0 with probability $1 - p$. Formally:

$$\mathbb{P}(X = k) = \begin{cases} p & \text{if } k = 1, \\ 1 - p & \text{if } k = 0, \\ 0 & \text{otherwise.} \end{cases}$$

This motivates the need for a specific type of a VAE that can handle discrete latent variables, namely, the Discrete Variational Autoencoders.

Discrete Variational Autoencoders

While standard Variational Auto-Encoders (VAEs) use continuous latent variables, Discrete-VAEs aim to learn latent representations that are categorical or binary. These models are particularly well suited for learning symbolic or discrete structures from continuous input data, such as binary features or logical predicates.

A popular way to implement a Discrete-VAE is through the Gumbel-Softmax (GS) trick [47], which allows us to sample from a categorical distribution in a way that supports backpropagation. The key idea is to reparameterize the categorical distribution using the so-called Gumbel-Max trick: given class probabilities α_i , we add Gumbel noise to $\log \alpha_i$ and take the arg max over the result to sample a category.

However, this discrete sampling operation is not differentiable. To work around this, the *Gumbel-Softmax trick* replaces the arg max with a differentiable softmax function controlled by a temperature parameter λ . As $\lambda \rightarrow 0$, the softmax output approaches a one-hot vector, as illustrated on figure 2.5.

In the special case of binary variables (*i.e.*, when the latent variable should follow a Bernoulli distribution), we use a variant called the Binary Concrete (BC) distribution [64]. This is a continuous relaxation of the Bernoulli distribution that enables gradient-based training.

In the BC-VAE, the relaxed binary sample z is computed as:

$$z = \frac{\exp((\log \pi + g_1)/\lambda)}{\exp((\log \pi + g_1)/\lambda) + \exp((\log(1 - \pi) + g_0)/\lambda)}$$

where π is the probability of the variable being 1, and g_0, g_1 are Gumbel noise samples for the two outcomes. If we define $g = g_1 - g_0$, which follows a Logistic(0,1) distribution [49], the formula simplifies to a sigmoid:

$$z = \sigma \left(\frac{1}{\lambda} (\log \pi - \log(1 - \pi) + g) \right)$$

This approach allows the model to learn binary latent features that can be interpreted symbolically. Figure 2.4 illustrates the difference between a sampling from a discrete Bernoulli and the relaxed version.



Figure 2.4: Sampling from a discrete distribution (left) using Gumbel-Max; and its continuous relaxation using the Gumbel-Softmax trick (right).

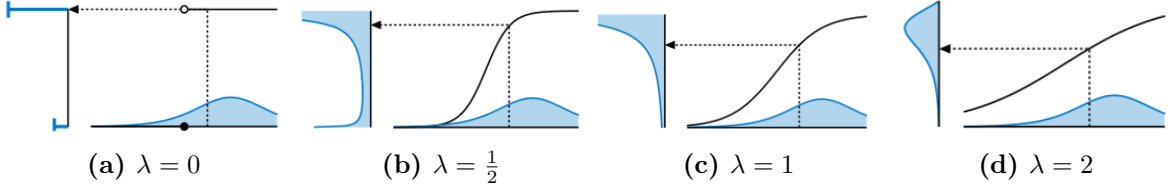


Figure 2.5: Effect of temperature parameter λ on Binary Concrete distribution. As $\lambda \rightarrow 0$, the output approaches a binary step function.

We observe that the latent representations learned by the Discrete-VAE converge towards binary values (0 or 1), making them suitable for encoding symbolic structures. This ability to learn discrete features from data is central to our approach. In the next section, we describe how these learned symbolic states can be used in the second step of the Knowledge Acquisition Bottleneck: the Action Model Acquisition.

2.3 Action Model Acquisition

Action Model Acquisition (AMA) is the process of obtaining an action model from data through learning (or any other automated mean) [96]. In this section we present various approaches of AMA.

2.3.1 Combinatorial approaches

Top-down approach: SAT-based Domain Learning

Liberman et al. [59] propose a top-down method for learning STRIPS-like PDDL action models from raw image sequences. Their approach combines symbolic image parsing with a constraint-based formulation solved using a SAT solver [14].

The process begins by applying a pre-trained image parser to extract symbolic facts (predicates) from each image frame, producing structured state representations. Given a set of these symbolic transitions, the goal is to recover an action model that is logically consistent with all observed changes.

To constrain the search, the authors define a *domain space* consisting of all possible action models built from a fixed set of lifted predicates (obtained from the parser), and of a predefined number of operator schemas, each with a limited number of parameters.

Each candidate domain is a STRIPS domain that specifies the preconditions and effects of its operators. For example, if a parser identifies predicates such as `on`, `clear`, and `holding`,

then the domain space includes all possible action schemas that can be constructed using those predicates and a small number of typed parameters.

The key innovation is to decompose the global SAT encoding into smaller independent problems, one for each grounded action instance. This factored formulation reduces the complexity of the SAT problem and allows the learning process to scale better. Each subproblem is encoded as a SAT formula that captures whether a particular transition between symbolic states could be explained by any instantiation of the operator templates from the domain space.

While this method guarantees consistency with the symbolic input data, it is sensitive to parsing errors, as any missing or incorrect predicates may prevent the SAT solver from finding a correct model. Moreover, the size of the domain space — and hence the SAT problem — grows rapidly with the number of predicates and action schemas, which impacts computational efficiency.

Bottom-up approach: Learning via Logical Reasoning

In [34] the authors propose a grammar induction-based method to learn the preconditions and effects of actions from noisy traces composed of sequences of states and actions.

The approach consists of two main stages. In the first stage, they apply the RNPI algorithm [68], which uses grammar induction to construct a Deterministic Finite Automaton (DFA) that models the valid and invalid sequences of actions and states in the domain. This DFA captures the structure of legal transitions, even in the presence of noise.

In the second stage, they extract STRIPS-like action models by analyzing the learned DFA. Preconditions for an action a are computed by intersecting all symbolic states that immediately precede occurrences of a in the trace. Positive effects are identified as propositions that appear consistently in the next state but not in the preceding one; negative effects are found by reversing this condition. Since the traces are noisy and incomplete, this stage is followed by a refinement process: for each pair of consecutive actions a and a' , the algorithm checks for inconsistencies between the inferred preconditions and effects, and adjusts the model accordingly.

This method depends on having access to traces where each state is represented in logical form — typically produced by a symbolic parser. As a result, its performance is limited by the parser accuracy. Like the SAT-based method discussed previously, it also inherits a degree of

domain dependence, since different environments may require domain-specific parsing logic.

2.3.2 Probabilistic Approaches

In [55], the author introduces NOLAM (Noisy Offline Learning of Action Models), a probabilistic framework designed to learn planning domain models from noisy sequences of actions and states. NOLAM represents the preconditions and effects of planning operators as random variables, enabling the model to capture uncertainty directly in the learning process. Given observed traces — where state and action information may be incomplete or noisy — it performs probabilistic inference to estimate the posterior probabilities that particular predicates are preconditions or effects of an action. This allows the system to reason under uncertainty and produce action models that are statistically consistent with the data, rather than relying on strict logical consistency. By incorporating noise models and prior distributions, NOLAM can generalize from imperfect observations and mitigate errors introduced during state parsing or observation. This makes it particularly well-suited for real-world data settings, where symbolic state representations are often noisy or unreliable. Yet, NOLAM, as the two previous approaches, requires a parser, and is therefore partly domain-dependent.

Latplan [9], which we introduce now, is the base model used in our work, and it can also be considered as a probabilistic approach, since it learns using neural networks.

2.4 Latplan: a domain-independent, image-based learner for classical planning

2.4.1 Overview of the architecture

Latplan [9] is a neurosymbolic architecture that can learn a PDDL representation of actions and states from pairs of images representing transitions in a visual domain. It consists mainly of two parts: the State Autoencoder (SAE) and the Action Model Acquisition module (AMA4+ in the latest version).

The SAE, is a discrete-VAE that learns a multinomial binary latent representation of the image, which can then be transformed into a propositional formula. The AMA module learns to associate an action’s label and preconditions to each transition, and then to predict the next state (and corresponding image) from the current state and the action label. It is

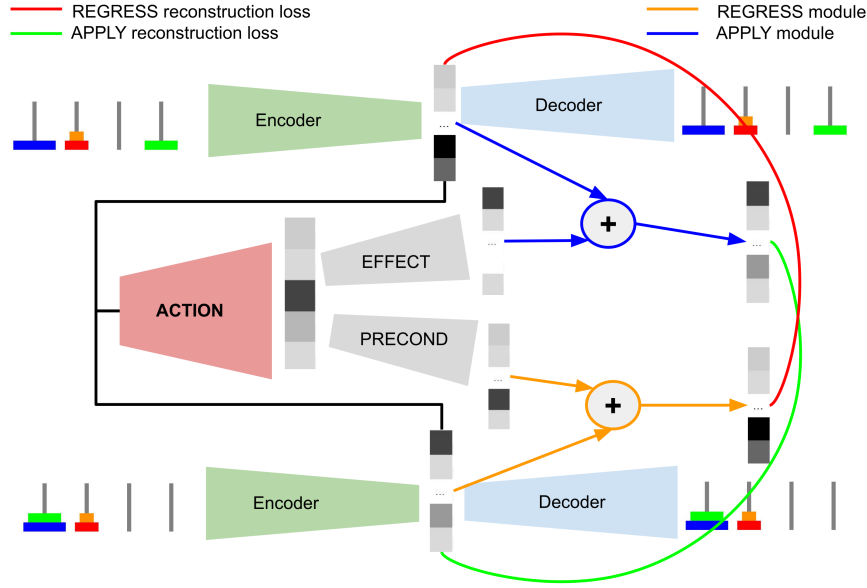


Figure 2.6: High-level illustration of Latplan [9].

composed of:

- an ACTION network, which consists of a dense layer of size 6000 followed by a Gumbel-Softmax (GS) activation layer. It takes as input two latent states representing a transition and outputs a continuous GS vector that converges, during training, toward a one-hot vector representing the label of the action;
- an EFFECT network, that takes as input an action and has N Binary Concrete outputs, where N is the size of the latent state representation;
- an APPLY module, that applies an action’s effects to a state and predicts the next state (we discuss it in more detail below).

Fig. 2.6 shows an overview of Latplan’s architecture.

Importantly, Latplan also learns the preconditions of an action with the same mechanism as for the effects. For this, it uses the PRECOND network (for predicting the preconditions) and REGRESS module (for predicting the first state of a transition).

The Latplan APPLY module in AMA4+ coincides with the Back To Logit (BTL) Mechanism, a technique for handling symbolic state or action representations while enabling backpropagation. It converts discrete symbols into logits via an embedding layer; then the softmax function outputs probability distributions and feeds them into a neural network. Loss computation, backpropagation, and gradient descent update the model, improving it iteratively.

Latplan’s loss function is very articulated. Latplan models the likelihood of observing a transition between two images x^0 and x^1 : $p(x^0, x^1)$, since this computation is known to be intractable [23], an Evidence Lower BOund (ELBO) is used as a loss function. The ELBO approximates the Likelihood from below and is a sum of the different losses and KL divergences [50] composing the network. For simplicity, we illustrate the figures only with the reconstruction loss of the APPLY and REGRESS network. The next section shows the complete form of the ELBO.

Latplan’s resulting action schemas are built from the grounded values of the latent states vectors generated by the SAE and the EFFECT/PRECOND networks. Here is an example of such action, in PDDL formatting:

```
(:action a15 :parameters ()
:precondition
  (and (not (z0)) (not (z1)) (z2) (not (z5)))
:effect
  (and (not (z0)) (not (z1)) (z2) (not (z5)) (not (z6))))
```

2.4.2 More details about the loss function

Latplan’s architecture does not only learn image representations via its SAE but it also learns an action model (via the AMA module), this involves a complex ELBO where an additional latent variable representing the label of the action a is introduced as well as a new variational distribution: $q(a \mid x^0, x^1)$, the following is the final stage of the ELBO derivation, each color indicates the correspondence between a loss term and the part of the network it optimizes (see figure 2.7), for the complete derivation, refer to Latplan’s paper [9]:

$$\begin{aligned}
 \log p(x^0, x^1) \geq & -\beta_1 D_{\text{KL}}(q(z^0 | x^0) \parallel p(z^0)) \\
 & + \mathbb{E}_{q(z^0|x^0)} \left[-\beta_2 D_{\text{KL}}(q(a | x^0, x^1) \parallel p(a | z^0)) \right] \\
 & + \mathbb{E}_{q(z^0|x^0)} \log p(x^0 | z^0) \\
 & + \frac{1}{2} \mathbb{E}_{q(z^1|x^1)} \log p(x^1 | z^1) \\
 & + \frac{1}{2} \mathbb{E}_{q(z^0|x^0)q(a|x^0, x^1)} \left[-\beta_3 D_{\text{KL}}(q(z^1 | x^1) \parallel p(z^1 | z^0, a)) \right] \\
 & + \frac{1}{2} \mathbb{E}_{q(z^0|x^0)q(a|x^0, x^1)p(z^1|z^0, a)} \log p(x^1 | z^1).
 \end{aligned}$$

Following a common practice of VAE implementations, the authors of Latplan compute the expectations by merely taking one single sample of the variables, rather than computing a mean from multiple samples. For instance, $\mathbb{E}_{q(z^1|x^1)} \log p(x^1 | z^1)$ becomes $\log p(x^1 | z^1)$.

The final implemented version of the loss is:

$$\begin{aligned}
 & \log p(x^0 | z^0) + \frac{1}{2} \log p(x^1 | z^1) + \frac{1}{2} \log p(x^1 | z^1) \\
 & - \beta_1 D_{\text{KL}}(q(z^0 | x^0) \parallel p(z^0)) \\
 & - \beta_2 D_{\text{KL}}(q(a | x^0, x^1) \parallel p(a | z^0)) \\
 & - \frac{1}{2} \beta_3 D_{\text{KL}}(q(z^1 | x^1) \parallel p(z^1 | z^0, a))
 \end{aligned}$$

2.4.3 Training Data

Latplan was trained and tested on 6 different visual domains, for each of which pairs of images representing transitions in the domain are given as training data. For the testing phase, two images, representing the initial and goal state of a planning problem are given.

Here is a quick description of each domain data:

- The **MNIST 8-Puzzle** ([8]) is a monochrome, 42x42-pixel adaptation of the classic 8-Puzzle. Each tile contains a 14x14 image of a handwritten digit (0–9) from the MNIST database ([57]), with the “0” tile acting as the blank space. Moves involve swapping

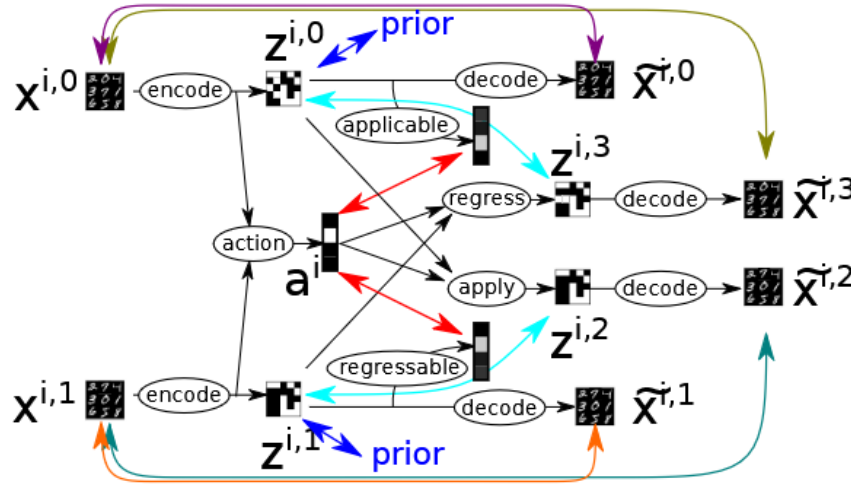


Figure 2.7: Overview of the action model learning process from Latplan framework, image from the original Latplan paper by Asai and Fukunaga (2018). i : indexes a particular transition. x_0^i and x_1^i are observed input images (*i.e.*, before and after an action), z_0^i and z_1^i are their latent propositional states, z_2^i and z_3^i are next state predictions, after action application or regression.

the “0” tile with an adjacent one. The puzzle has 362,880 possible states ($9!$), but only half are reachable. The longest solution path requires 31 moves.

- The **Scrambled Photograph** (Mandrill) 15-puzzle uses a real photograph cut into tiles and scrambled, similar to store-bought puzzles. It is based on a “Mandrill” image from the USC-SIPI benchmark set ([90]). Unlike the MNIST 8-puzzle, the tiles lack clear black boundaries, and Latplan does not assume square or movable regions. The state space can be expanded by using a 4x4 grid instead of 3x3, creating larger 15-puzzle instances. The image, originally 56x56 pixels, was converted to greyscale and processed with histogram normalization and contrast enhancement.
- **LightsOut** ([4]), originally released by Tiger Electronics in 1995 and also known as the “Magic Square” game on Merlin, is an electronic puzzle game. It features a grid of lights in an on/off configuration (+ : On). Pressing a light toggles its state and that of its neighbors, with the goal of turning all lights off. Unlike traditional puzzles, a single move can affect up to five locations and remove some lights, showing Latplan’s ability to handle non-local effects and dynamic objects. An $n \times n$ LightsOut grid has 2^{n^2} states and $n^2 \cdot 2^{n^2}$ transitions. This study used a 5×5 grid (45x45 pixel images), while earlier work used a 4×4 grid.
- **Twisted LightsOut** distorts images in LightsOut using a swirl effect from the `scikit-image` package, demonstrating that Latplan is not restricted to handling rectangular “objects”

or regions. The images retain the same dimensions as the original LightsOut. The swirl effect is applied to the center with a strength of 3, linear interpolation, and a radius equal to 0.75 times the image dimension.

- **Photo-realistic Blocksworld** ([6]) is a dataset consisting of 100×150 RGB images rendered using the Blender 3D engine. The images feature complex visual elements, such as reflections from neighboring blocks. To simplify the dataset, we modified the rendering parameters by limiting the object shapes to cylinders and removing jitters (small noise) in object placements. The final images were resized to 30×45 pixels.
- **Sokoban** ([21]; [51]) is a PSPACE-hard puzzle domain. Its 112×112 pixel visualizations are sourced from the PDDLgym library (Silver & Chitnis, 2020) and resized to 28×28 pixels. Unlike other domains, Sokoban is irreversible and contains dead-ends. The `p006-microban-sequential` instance was used as the visualization source.

2.4.4 Latplan previous versions of the AMA

Before reaching AMA4+ the authors started from a simplistic architecture (AMA1) where each action is derived directly from the bit differences between two transitions states returned by the SAE, this method does not generalize over (noisy) images. Then, the authors developed incrementally more complex models. The difference between each version are summarized in table 2.1. Importantly, they assumed from the ground up that the actions are not known but are *to be learned*, we explain further that this is the main reason of Latplan inefficiency.

2.5 How does Latplan fail?

2.5.1 Planning performances

We considered only the AMA4+ version of Latplan since it is the most efficient. For each domain, the authors generated forty initial/goal image states corresponding to twenty 7-steps and twenty 14-steps problems (they used a backward Dijkstra ([25]) search starting from the goal to find out length-specific problems). Then each image was encoded using the Latplan SAE encoder and converted into a PDDL encoding. Each problem was tested

Method	Description	Key Features and Limitations	Compatibility with Classical Planning	Generalization Capability
AMA1	Applies SAE to image transitions, directly converts transitions to STRIPS actions.	Demonstrates feasibility but impractical due to lack of generalization. Requires all possible transitions in training data. Large PDDL models slow down preprocessing and planning.	Compatible (PDDL-based solvers)	Limited; does not generalize beyond training data.
AMA2	Learns action symbols and successor functions in an unsupervised manner.	Requires no high-level action symbols but does not produce PDDL-compatible action descriptions. Needs specialized search algorithms.	Incompatible (requires custom solvers)	Generalizes better due to unsupervised learning but lacks STRIPS assumptions.
AMA3	Neural architecture producing descriptive action models with a prior.	Models STRIPS-compatible action effects but requires ad-hoc process for precondition learning. Theoretical limitations in optimization objectives.	Compatible (after precondition learning)	Moderate; neural component supports STRIPS/PDDL but needs precondition augmentation.
AMA4+	Enhanced version of AMA3 with complete state regression semantics.	Learns and emits STRIPS-compatible preconditions directly without ad-hoc processes. More accurate and efficient.	Fully compatible (PDDL-based solvers)	High; models full state space with STRIPS/PDDL compatibility.

Table 2.1: Comparison of Methods for Learning Action Models

with Fast Downward on Blind heuristic and three different heuristics (Landmark-Cut [42], Merge-and-Shrink (M&S) [43], LAMA [72]).

Finally, they sorted the solutions in three categories: **found**, **valid** and **optimal**. “Found” corresponds to the case where FD found a solution. “Valid” corresponds to the case where the found solution is *visually correct* (see more about this in the next section) and “optimal” is when a solution is valid and optimal in the number of steps (shortest path). The results are shown in table 2.2.

We see that all optimal solutions were found only on two domains (Lights Out and Twisted) but only with two heuristics (Blind and M&S). We also notice that a good amount of found solutions are not visually correct (not valid).

Finally, the authors present the results for an additional domain (failure case): the Towers Of Hanoi (ToH) [15]. They tested on several ToH datasets varying in number of disks and towers, namely where $(d,t) = (4,4), (3,9), (4,9), (5,9)$ (d : number of disks, t : number of towers). For $(3,9)$ they generated 5-steps problems (because this configuration lacks 7-steps plans) and for the others they generated 7-steps problems (they lack 14-steps plans). See the results in table 2.3.

	Blind			LAMA			LMCut			M&S		
	found	valid	optimal	found	valid	optimal	found	valid	optimal	found	valid	optimal
Blocks	32	32	-	19	19	-	34	34	-	34	33	-
LOut	40	40	40	40	40	1	20	20	20	40	40	40
Twisted	40	40	40	40	40	1	20	20	20	40	40	40
Mandrill	25	23	23	20	15	11	38	30	30	40	32	32
MNIST	40	39	6	35	35	16	40	39	6	40	39	6
Random MNIST	36	34	11	32	32	5	35	32	11	36	33	11
Sokoban	40	39	38	40	31	21	40	38	37	40	39	38
Total	254	247	158	226	212	55	227	213	124	270	256	167

Table 2.2: Performance comparison of AMA4+ using Blind, LAMA, LM-Cut, and M&S.

	Blind			LAMA			LMCut			M&S		
	found	valid	optimal	found	valid	optimal	found	valid	optimal	found	valid	optimal
(d, t) = (4, 4)	0	0	0	0	0	0	0	0	0	0	0	0
(d, t) = (3, 9)	20	12	8	17	1	0	20	12	7	20	12	8
(d, t) = (4, 9)	0	0	0	0	0	0	0	0	0	0	0	0
(d, t) = (5, 9)	0	0	0	0	0	0	0	0	0	0	0	0

Table 2.3: Performance comparison of AMA4+ using Blind, LAMA, LM-Cut, and M&S, on Towers Of Hanoi domain, in different configuration (d: number of disks, t: number of towers)**Figure 2.8:** Hallucinated output from Latplan decoder

Even though the state space of ToH is arguably smaller than for the other domains (e.g. for (4,4) the number of states is 256) as well as the size of image (each image is of dimension $(H, W, C) = (h \times d, w \times t, 3)$ where $h = 1$ pixel and $w = 4$ pixels), **Latplan** is unable to produce efficient PDDL domains on most configurations, except on (3, 9), where almost half are invalid solutions. The authors argue that this failure could be based on several reasons: the small size of the dataset not fit for DL approaches, or an imbalance in the visual features since, for example, the states where the smallest disk is on top are very rare but are needed for solving the problems.

2.5.2 Hallucinations

While **Latplan** does not always find solutions, even when it does, they are often visually invalid in that the generated images are what is commonly referred to as *hallucinations*. A hallucination occurs when a generative model produces an output that does not match the expectation, for instance, when it includes unintended artifacts. Figure 2.8 is an example of a hallucination produced by **Latplan**.

2.5.3 Lack of explainability of the action model

In *Latplan*, the action’s label is not known but it is learned during training. It corresponds to the ACTION network output and is interconnected to both states of a transition (given as input of ACTION) and to the APPLY and REGRESS network for next (resp. previous) state prediction. Nothing relates a learned action’s label to a high-level action performed by an agent (*e.g.*, in ToH *move(d1, d2)*), therefore, since the label is learned, it is possible to find hallucinations also in the actions labels, for instance, a label could correspond to two transitions that would normally not correspond to the same high-level action.

2.6 Why *Latplan* fails

The authors of *Latplan* argue that their architecture is sound with regards to the likelihood maximization of $p(x^0, x^1)$ (*i.e.*, the probability of the model to generate two images that correspond to a real transition in the data). Yet, this does not impend *Latplan* to produce erroneous actions, such that, when applicable, the resulting transition could not be implemented by any real action of the domain (either because the next state contains spurious elements or because, as a whole, the transition corresponds to an illegal move). Indeed, action’s label are seen as latent variables to be learned, instead we believe that they should be seen as *observables*, notably because the agent actions are generally known (if we have access to the images seen by an agent, in most of the scenarios, we also have access to the agent actions, for instance, in PDDL_{Gym} domains).

Chapter 3

Improving Latplan through invariant enforcing and hyperparameters tuning

In this chapter we present two main paths of research we have conducted in order to improve **Latplan**, namely invariants enforcing and hyperparameters search. Some of the text draw directly from our published work on Invariants Enforcing [12] presented at the Italian Workshop on Planning and Scheduling (IPS-2022, 10th edition) in 2022.

3.1 Invariants enforcing

3.1.1 What are the invariants?

In Classical Planning, an invariant is defined as a logical formula over variables of the domain which is true in any reachable state [3]. Invariants can be seen as hidden but logical properties of the domain, and can also be termed as *state constraints*.

An important part of the research in Classical Planning focuses on discovering invariants in planning domains. Today, automatic tools already exist that can find a variety of invariants[32] [41], such as static invariants on predicates (ones that are unaffected by any operator), simple implicative invariants (*e.g.*, if $z_1 \rightarrow z_2$), mutually exclusive invariant or *mutexes* (*e.g.*, $\neg z_1 \vee \neg z_2$) etc.

3.1.2 Why did we make use of invariants?

Since **Latplan** learns symbolic representations from subsymbolic data (*e.g.*, images), it is subject to the Symbol Grounding Problem (SGP) [80] which refers to the case where two different inputs (*e.g.*, images) grounding the same symbol (*e.g.*, digit “1”) have different vector representations, revealing a lack of generalization power. In planning, this duplication of symbols poses various issues, for instance it can increase substantially the search time or prevent the search from succeeding.

Even though **Latplan** is able to deal with the SGP up to a certain amount (notably because the representations are learned from a VAE which by nature filters out the noise) which can be assessed by measuring the variance of the representations on perturbed input, the SGP is not entirely solved (the variance is not null). Therefore, we proposed another way to address the SGP which is to enforce symbolic constraints on the learning of the categorical vectors. If we consider a vector v as an ideally grounded symbol and its unstable version v' , then v' contains some noise, *i.e.*, variables with undesired values. If we know appropriate constraints to apply during training, the noisy representation corresponding to v' could converge to v .

The authors of **Latplan** propose a metric to measure the magnitude of the SGP on **Latplan** representations, namely the State Invariance. To compute it, they use the state variance for noisy input, *i.e.*, the variance of the latent vectors :

$$z^{i,0} = ENCODE(x^{i,0} + n)$$

where:

- $n \sim N(\mu = 0, \sigma = 0.3)$
- $x^{i,0}$: 1st image of the i^{th} transition pair of the dataset

The State Variance is computed by iterating over 10 random vectors, then averaged over F bits in the latent space and over the dataset indexed by i . Formally:

$$\mathbb{E}_{f \in 0 \dots F} \mathbb{E}_i \text{Var}_{j \in 0 \dots 10} [ENCODE(x^{i,0}, n^j)_f] \quad (3.1)$$

3.1.3 Searching for invariants of interest

Automatic tools, like the Fast Downward (FD) preprocessor [41] and DISCOPLAN [32], can find invariants in the PDDL representation outputted by **Latplan** using invariant synthesis.

But, since **Latplan** learns the PDDL by statistical inference, there is no guarantee that any of the invariants found in the PDDL maps to the ground truth invariants, *i.e.*, to invariants of the (unknown) ground truth PDDL domain. For example, if $z_1 \rightarrow z_2$ is returned by FD as an invariant, but z_1 is true only once in the whole dataset, it is possible that $z_1 \rightarrow z_2$ would be false if we had more data with z_1 being true. Thus, in our work we consider the invariants computed by an invariants generation tool (on **Latplan** PDDL) as a set of *probable invariants* that we intend to test.

In our work we used the Fast Downward preprocessor that generates a SAS file [40] where we can find two types of information of interest, variables and mutexes. Variables are not explicitly present in the original PDDL, but are generated automatically as part of Fast Downward’s internal encoding:

```
begin_variable
var203
-1
5
Atom z10()
Atom z108()
Atom z19()
Atom z88()
NegatedAtom z88()
<none of those>
end_variable
```

For instance, this block, shows that the variable 203 can assume 5 values that are mutually exclusive and that correspond to literals we find in the PDDL domain (*e.g.*, $z88$ or its negation).

A second block of information is the following:

```
begin_mutex_group
5
203 0
203 1
200 1
```

```

205 2
201 1
end_mutex_group

```

It shows mutual exclusivity between the values of different variables (as defined in the `begin_variable` block), for instance in this example there is a mutual exclusion between the values indexed by 0 and 1 (*i.e.*, literals `z10` and `z108`) of variable 203, the value indexed 1 of variable 200, the value indexed by 2 of variable 205 and the value indexed by 1 of variable 201.

In our experiments, we took into account all mutexes given by both sources of information, and computed a loss term for each of them (see next section).

3.1.4 Applying the constraints using Fuzzy logic

Extensive work on integrating logical constraints *during the training* of neural networks have been conducted in the last few years [11] [65]. Notably, one can express constraints among neural networks outputs taken as predicates thanks to *t-norm* operations and the fuzzy generalization of First Order Logic (FOL) [37] [94].

For example, if we want to express the truth value of $\neg z_1 \vee \neg z_2$, we can compute it by its average over the dataset, *i.e.*,

$$\frac{1}{|Z|} \sum_{z_1, z_2 \in Z} (1 - P(z_1)) + (1 - P(z_2)) \quad (3.2)$$

where $P(z_1)$ is the (continuous) value of truth of one grounding of “ z_1 is true”, and Z is the set of all the groundings. The inverse of this value can be directly appended as a penalty to the loss function, which eventually forces the network to re-adapt its weights to this new constraint.

In `Latplan`, internal categorical representations are continuous vectors from which elements converge to binary values during training. At each batch, we can compute the truth value — thanks to fuzzy logic — of any constraint, expressed as a logical formula, over these binary variables, for instance “ z_1 is true.” Then, by taking the inverse of this value and multiplying it by a normalizing factor, we obtain an additional loss that we can append to the total loss. This way, the network will adapt its weights through back-propagation to comply with the constraint.

Experimental results

From the set of invariants retrieved from a PDDL generated by Fast Downward we performed a re-training (with weights reinitialization) where we enforced the invariants using the fuzzy logic formulas in the loss.

Table 3.1 shows the Planning performances for MNIST, Sokoban and the Blocksworld domains. We see that the scenario where we enforce invariants brings more optimal plans on the longest problems (14-steps) only on MNIST puzzle dataset whereas for the two others, i.e. Sokoban and BlocksWorld, the invariant enforcing worsened the planning performances (no plans found). One possible explanation for this mitigated result could be that MNIST is arguably simpler to learn than the two others, hence the invariants learned through back-propagation have more likelihood to be “useful” and not just noise, but this is of course only a supposition.

Table 3.2 shows the performances on Sokoban and MNIST in terms of metrics, namely we computed the State invariance as defined in section 3.1.2. We see that invariant enforcing clearly help reducing the State Variance, hence, solving the SGP. It also confirms (as mentioned in Latplan’s paper) that solving the SGP is not the only criteria for assessing the model performances (*e.g.*, the next state prediction accuracy is also important).

	no enforcing		“all-at-once”	
	7-steps	14-steps	7-steps	14-steps
MNIST puzzle	16/20	6/20	16/20	15/20
Sokoban	11/20	3/20	0/20	0/20
BlocksWorld	9/20	0/20	0/20	0/20

Table 3.1: Number of solutions found per domain and problem size (7-steps and 14-steps), with and without enforcing invariants (each cell shows X/Y with, X: number solutions found, Y: number of problems tested). “all-at-once” means that all the invariants by FD were used in the loss function.

	Sokoban	Mnist
Without enforced invariants constraints	1.5×10^{-6}	7.7×10^{-2}
With enforced invariants constraints	5.25×10^{-7}	6.7×10^{-5}

Table 3.2: State Variance with and without enforcing invariants.

3.2 Modifying the hyperparameters

Given that the work on invariants yielded little success, we decided to adopt a more classical model optimization approach, namely searching the model’s hyperparameter space to find better combinations of parameters.

We conducted three basic experiments, which we now describe.

3.2.1 Hyperparameters search

We ran a Grid search for the following hyperparameters: N (size of the state latent space), tested for values 50, 100 and 300. β_z , the parameter weighting the role of the loss concerning the VAE priors, tested for 10, 100, 1000, 10000 and β_d , the parameter weighting of the next state prediction loss, tested for 10, 100, 1000, 10000.

We then calculated the χ -square score [69] of each hyperparameter for two target classes: the number of optimal solutions and the number of invariants. For instance, for calculating the χ score of N with regard to the number of solutions, we retrieved from all run, the value N and the number of optimal solutions found. The results are summarized in table 3.3.

	N	β_d	β_z
Sokoban			
χ^2 score / # optimal solutions	7.7e+02	1.5e+05	3.4e+01
χ^2 score / # invariants	6.7e+02	7.8e+04	4.4 e+01
Blocksworld			
χ^2 score / # solutions	1.4e+02	3.04 e+03	1.3e+01
χ^2 score / # invariants	9.3e+02	1.07 e+05	1.67e+01

Table 3.3: Comparison of χ^2 scores for Sokoban and Blocksworld.

β_d is much more correlated to the number of solutions and invariants than N and β_z , also I observed that β_d has a high value when Latplan finds solutions. Therefore, an emphasis on the next state prediction model seems to be important for obtaining more solutions (and more invariants).

3.2.2 Reducing the number of possible actions

By default the output’s layer of the ACTION network is of size 6000, which means that potentially 6000 action’s label can be learned by the model. However after training, all the labels that are never used on the whole dataset are removed (i.e. not translated into PDDL format).

Instead of learning an arbitrary number of actions, we wanted to test the case where the number of possible actions learned is reduced to the number of high-level actions of the domain, *i.e.*, the actions taken by the agent, for instance in Sokoban domain, there are 4 actions: go up, do down, go left and go right.

Unfortunately, it resulted in almost only bad metrics: bad image reconstruction, very low next-state prediction accuracy, and no planning solutions found at all. This result led us to believe that a more consequent change was needed for having a system that learns high-level actions directly (and is actually at the root of our next works, see later sections).

3.2.3 Reducing the size of the learned state space.

Starting from a similar intuition as for reducing the number of actions, we tried to reduce the number N of propositional variables to be learned to an upper bound for the total number of possible states in the domain.

For example, in 8-Mnist puzzle, the total number of states is $9!$, therefore, we can find an upper bound in terms of number of bits necessary for representing $9!$ binary states. Since $\log_2(9!) \approx 18.46$, we have:

$$2^{18} \leq 9! \leq 2^{19}$$

Therefore we can choose 19 bits as a sufficient number to represent all the states. Nevertheless, we chose $N=22$ to provide a higher-capacity latent space that may facilitate learning.

One interesting result from reducing N is that it helps **Latplan** to deal with noisy input as we can see on figure 3.1, which in turns, enables the generation of more visually-correct plans from noisy problems (*i.e.*, where at least one of the state images, initial or goal, is noisy).

In figure 3.2 we summarize all the results described in the chapter. We now introduce **R-latplan** [13] a substantial improvement over **Latplan** that was presented at ICTAI 2024 ¹.

¹the 36th IEEE International Conference on Tools with Artificial Intelligence

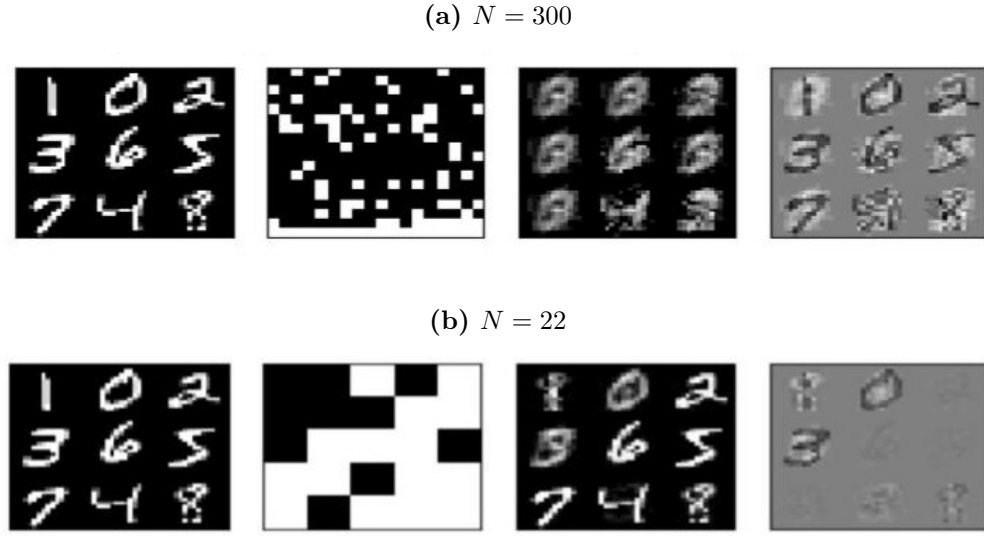


Figure 3.1: Reducing the latent state size, leads to a better state reconstruction from the VAE. From left to right: input image to the VAE, latent state representation of the VAE, outputted image of VAE decoder, difference between input and output.

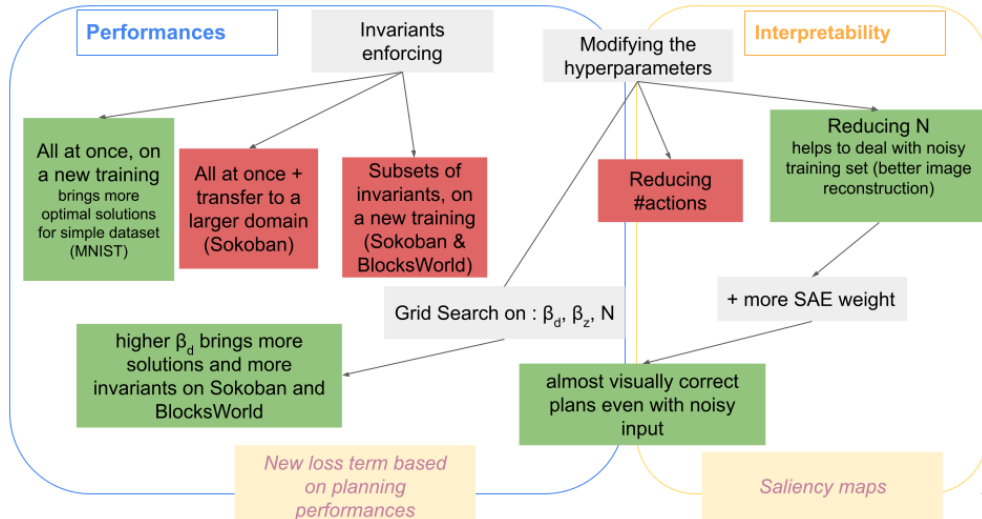


Figure 3.2: Summary of our different improvement on existing Latplan architecture. Green cells indicate positive results, red cells negative results and yellow cells Future Work.

Chapter 4

R-latplan

In this chapter we introduce our new proposal, R-latplan, based on Latplan and with the claim of generating reliable action models.

4.1 Overview of the approach

As see in 2.6, Latplan presents two major flaws regarding the quality of the generated action models, namely, hallucinations in the visual plan generated and no direct link to an agent real actions.

On top of this, for the benchmark considered in Latplan’s paper, Latplan does not succeed on yet simple problems in Planning, such as the Tower Of Hanoi problem with few disks and pegs.

4.2 Methodology

We assume to have access to a set of visual traces, e.g. what can be captured by a camera when observing an autonomous agent operating in an environment, eventually under the control of a teleoperator. A visual trace is a sequence of images, each of which is a visual description of a state of the world. Hence, we assume to have access only to state observations as images taken at discrete time steps, and pairs of images, each of one corresponding to an observed states transition. We also have access to the agent’s actions, *e.g.*, in Hanoi *move(d3, d4)*, in Sokoban *go(right)*, in Blocksworld *pickup(block_A)*. We refer to these actions as **high-level grounded actions**, for the remainder of the thesis, we use the shorthand high-level actions for brevity.

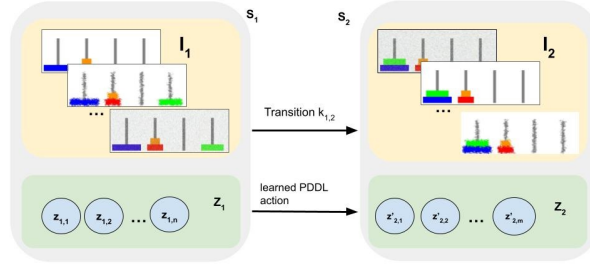


Figure 4.1: An illustration of two states in the Visual DFA connected by a transition.

In the following, we give some definitions that are useful to specify the problem we want to solve.

We define a **Visual DFA** $\mathcal{V}$ as an augmentation of a *planning DFA* where each state s_i of $\mathcal{V}$ is associated with a pair (I_i, Z_i) , where I_i is a set of images, and Z_i is a set of propositional representations of the images in I_i (or encodings in a latent space of propositional states). In particular, given a planning DFA (S, A, π, s_0, G) , let *vimg* be a function that maps each $s_i \in S$ to a set of images I_i that visually represents s_i , i.e., $\text{vimg}(s_i) = I_i$. Let also *img2prop* be a function that maps each image $x_{i,l} \in I_i^1$ to a symbolic propositional representation $z_{i,l} \in Z_i$, i.e., $\text{img2prop}(x_{i,l}) = z_{i,l}$. For each transition $k = (s_i, s_j)$ of $\mathcal{V}$, we have that $\forall x_i \in \text{vimg}(s_i)$, executing k when x_i is the current world observation, results in the world to become a state observed as an image $x_j \in \text{vimg}(s_j)$, see Figure 4.1.

A Visual DFA $\mathcal{V}$ is **noisy** (resp. **crisp**) if for at least one of its states (resp. each of its states) s_i , $|\text{vimg}(s_i)| > 1$ (resp. $|\text{vimg}(s_i)| = 1$). A Visual DFA $\mathcal{V}$ is **complete** if it contains all the possible transitions of the domain (**partial** if not).

We also have a **transition identifier function** that returns a unique identifier for a given pair of states in a given Visual DFA. Figure 4.2 shows an example of an erroneous assignement of ID from the transition identifier function.

We define a **reliable PDDL action** as a PDDL action that can be associated with full certainty to an action performed by an agent. We refer to the transitions in the visual DFA as **low level actions**.

Since we know for each transition performed by the agent, the label returned by the transition identifier and the high level action label of the agent, we can make a direct relation between both. Generally, a high level action corresponds to multiple low level actions (e.g. *pickup(block_A)* corresponds to the set of all the transitions (low level actions) where the preconditions of the action *clear(A)* and *handempty* are true in the source state of the

¹ l indexes an image in the set I_i .

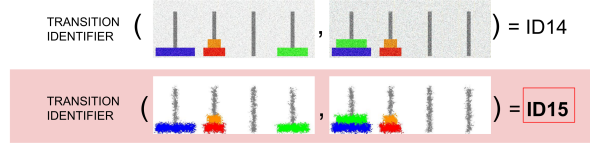


Figure 4.2: An illustration of the case where the transition identifier function labels erroneously two versions of the same transitions with different labels (ID14 and ID15 in the figure), due to visual noise.

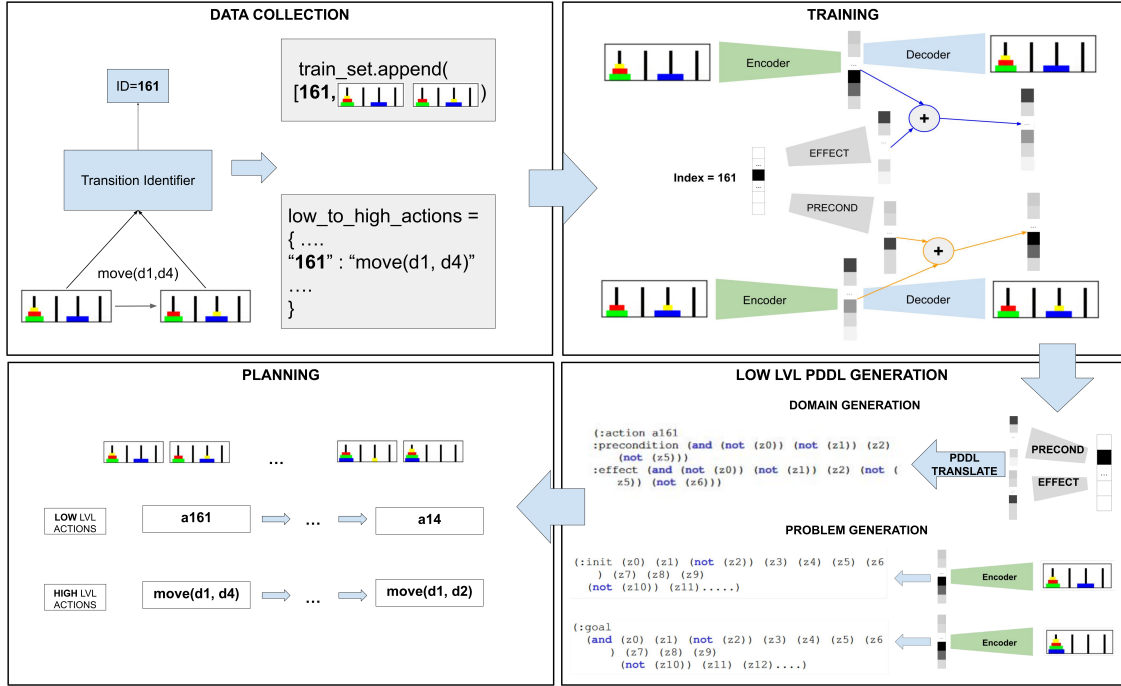


Figure 4.3: The overall workflow of R-Latplan.

transition). Based on these definitions, we formulate our problem as learning each transition of a Visual DFA as a **low level** and **reliable** PDDL action.

For each transition in $\mathcal{V}$, the corresponding action is identified by the following binary vectors (of size N): *neg_pred* (negative preconditions), *pos_pred* (positive preconditions), *add_eff* (add effects), *del_eff* (delete effects). Such binary representations can then be directly transformed into a propositional PDDL domain action. The workflow of R-latplan is summarize in figure 4.3.

We use Latplan’s SAE for learning the *img2prop* function. Moreover, given two states s_i and s_j and a corresponding transition label k identified by the transition identifier, we obtain the effects and preconditions of k (now, interpreted as a PDDL action) with the following:

- $ADD(a) = APPLY(a, 0)$
- $DEL(a) = 1 - APPLY(a, 0)$

- $POS(a) = REGRESS(a, \mathbf{0})$
- $NEG(a) = \mathbf{1} - REGRESS(a, \mathbf{0})$

where $\mathbf{0}, \mathbf{1} \in \{0, 1\}^N$.

We built the transition identifier function so that, given two images of a transition and a dictionary of transitions labels as keys and reference transition images as values, it checks if the two images correspond to one of the existing transitions, if yes, it returns the identified label, if not it creates a new entry in the dictionary and returns a new label.

Since we know the label for each transition of the DFA, we considered the action label as an *observable* (instead of a latent variable to be learned, as explained in 2.6) and we removed the ACTION network and replaced its output with the one-hot encoding of the label (see figure 4.3 top right). Figure 4.4 highlights the difference between Latplan and R-latplan in terms of dependency between latent variables and observables.

The loss functions remain the same to simplify the comparison.²

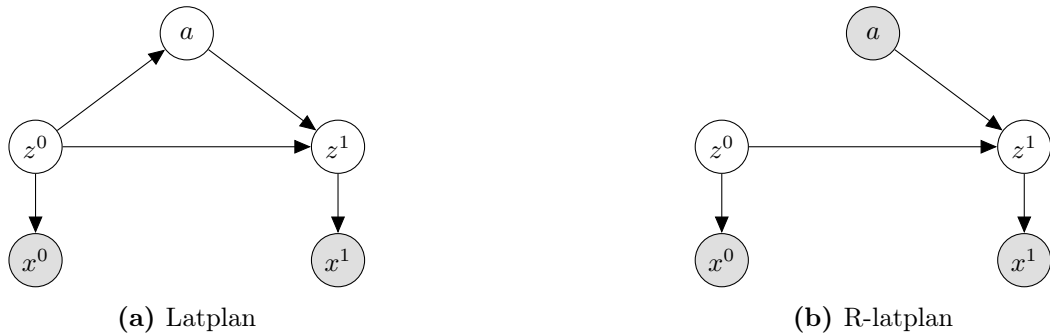


Figure 4.4: Comparison of dependency graphs in Latplan and R-Latplan. A grey node means an observable variable, a white node means a latent variable.

4.3 Research questions

To demonstrate that R-latplan learns reliable symbolic (PDDL) representations of an agent’s actions from noisy and incomplete visual observations, and without explicit human expert knowledge, we experimentally investigate the following research questions:

RQ1: Does R-latplan produces reliable actions schemes even in presence of a *noisy Visual DFA*?

²R-latplan’s source code is available at <https://github.com/aymeric75/R-latplan>.

	Data1,2		Data3,4		Data5	
	#states	#transi.	#states	#transi.	#states	#transi.
Hanoi	256	1459	234	1028	256	1539
Blocks	125	272	125	267	125	286
Sokoban	1270	4152	256	4069	1270	4380

Table 4.1: Number of unique states and transitions.

RQ2: Does R-latplan produces action models that enable Fast Downward (FD) [40] to find optimal plans even when dealing with partial *noisy Visual DFA*?

RQ3: Is R-latplan robust against errors of the *transition identifier function*?

4.4 Benchmarks

In order to answer these questions we prepared the five following pools of data:

Data1: Crisp and Complete Visual DFA contains all the transitions of the Complete Visual DFA, with only one image version for each state s_i ($|I_i| = 1$)

Data2: Noisy and Complete Visual DFA built on **Data1** but for 55% of the transitions, three different image versions ($|I_i| = 3$)

Data3: Crisp and Partial Visual DFA from **Data1**, we removed all but one possible path between two states of the Complete Visual DFA. We created three versions for each domain, with a different node removed for each.

Data4: Noisy and Partial Visual DFA from **Data2**, we followed the same procedure as described for **Data3**.

Data5: Noisy Visual DFA and Erroneous Transition IDs from **Data2**, we duplicated 5% of the transitions, assigning them a different label than the one returned by the transition identifier. This simulates the case of an erroneous *transition identifier function*.

Table 4.1 contains the number of states and uniques transitions in each pool of data.

The experiments we devised are as follows. For each, we trained **R-latplan** on the corresponding data pool.

Experiment 1 (resp. **Exp 2**), consists of training **R-latplan** with all the transitions of Data1 (resp. Data2), then testing the generated PDDL domain from Data1 (resp. Data2) on the three longest problems (i.e. for which the optimal plan is the longest Hamiltonian path³ possible).

Experiment 3 (resp **Exp 4**), consists of training/testing **R-latplan** with all the transitions of Data3 (resp. Data4), one training for each version (i.e. with a different node and set of paths removed). The testing phase was performed with the three PDDL domains generated on the respective removed nodes taken as problems.

Experiment 5, consists of inspecting the generated PDDL from Data5, for the presence of redundant actions, i.e. actions that have the same effects, and for which in any state where one action preconditions' hold the other action's preconditions hold as well.

Experiments **1 to 4** address **RQ1** and **RQ2**, Experiment **5** addresses **RQ3**.

Benchmark Domains and Datasets. Our experimental analysis—similarly to [9]—builds on three well-known planning domains: Hanoi, Blocksworld, and Sokoban. In particular, we used the PDDLgym [76] implementation and the following settings. For the Hanoi domain, we considered four disks and four rods. In the Blocksworld domain, we used coloured blocks and a single gripper. For the Sokoban domain, we considered an 8×8 grid, with 34 free cells and a single box.

It is worth mentioning that in [9] different versions of the same domains have been considered. For instance, the Hanoi domain considered simplified images, where there are no shape differences between disks; for the Blocksworld domain, [9] considered a photorealistic dataset; and for the Sokoban domain, [9] used a grid with 15 free cells and three boxes.

4.5 Experimental setup

For generating the training sets of each domain, we derived first the *crisp DFA*, by using the PDDLgym step function from different starting states, in order to gather all the possible transitions (we kept one version of each transition) of each domain, each transition consists of a pair of images returned by the PDDLgym *render* function. Then, for deriving the different data pools (**Data1-5**), we associated to each image pair representing a transition,

³A plan that does not passes twice by the same node.

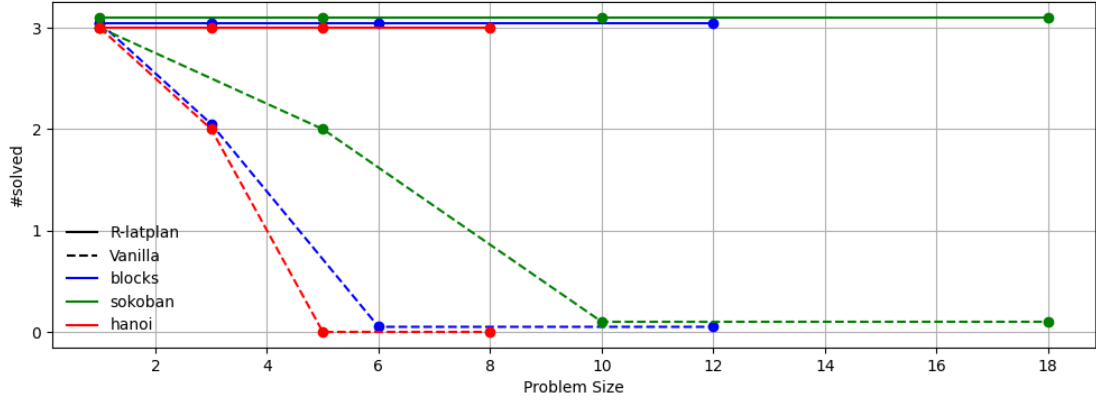


Figure 4.5: Success number of tested executions (**maximum success is 3**) on the longest possible paths from the derived Visual DFAs on **Exp1: Crisp and Complete Visual DFAs** and **Exp2: Noisy and Complete Visual DFAs**

	R-latplan	Asai et. al
Hanoi	70x25	49x18
Blocksworld	54x54	27x27
Sokoban	43x43	22x22

Table 4.2: Image resolutions in pixels

with its transition label, obtaining the latter by using the transition identifier function. Each image was pre-processed for training (see below). For **Data3,4** we leveraged the NetworkX library [36] to obtain a partial Visual DFA for which we identified—using the Dijkstra [25] algorithm—the longest traces.

Image preprocessing. Each image was reduced down to a certain size, determined empirically. In particular for training Latplan [9], we had to reduce the size almost twice compared to R-latplan (see table 4.2), in order to obtain decent performances. We then followed the experimental setup of [9], starting from RGB values expressed in the floating point between 0 and 1, the image goes through histogram equalisation, then re-normalised to have an expected value of 0 and a standard deviation of 1.

Noising procedure. To create *noisy DFAs*, we applied to the final pre-processed image a Gaussian noise centered on 0 with a standard deviation of 0.015 to ensure that the overall noise is—with high probability—within 10% from the original pixel value.

Hyperparameter search. We ran a grid search for the following hyperparameters: N (size of the state latent space), tested for values 50, 100 and 300. β_z , the parameter weighting the role of the loss concerning the VAE priors, testing for 10, 100, 1000, and β_d ,

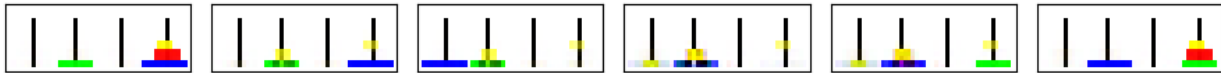


Figure 4.6: Not visually correct plan generated by (vanilla) Latplan

the parameter weighting of the next state prediction loss, tested for 10, 100, 1000.

We selected the combination of hyperparameters leading to the lowest value of the loss term predicting the resulting transition accuracy (see figure 4.8). For Latplan, we had to expand the search space with additional variables and use a genetic search algorithm.⁴

4.6 Results

Regarding **RQ1**, i.e. if the generated PDDL actions are reliable, by design they all comply with the first condition, i.e. they are all linked to a high level action of the agent/teleoperator, since this is part of the data collection process (see figure 4.3). For the second condition, i.e., if for any action and any state on which the action is applicable, the corresponding generated images belong to a set I_i of images of the Visual DFA, we successfully checked this condition by checking if the distance from the canonical image returned by PDDL Gym was under a certain threshold (the same threshold set in the transition identifier function). Furthermore, looking at the images by our human eye we can see that they are visually correct and do not display any odd artifact (e.g. duplication of an object, or wrong colorizations). On the other hand, none of the actions generated by Latplan [9] is reliable, because nothing in the architecture of Latplan links high level actions to the low level PDDL actions generated, furthermore, as one can see in figure 4.6, the images generated are often not visually correct.

Regarding **RQ2**, as shown in figure 4.5, R-latplan produced PDDL actions for which Fast Downward found all optimal plans on both experiments **1-2** with a Complete DFA, i.e. with/without noise, whereas Latplan’s PDDLs lead to only few visually correct plans and only for limited problem size (on Sokoban the maximum problem size solved was six, and only on two instances out of three). Importantly, the noise introduced in the images for experiments **2-4** were exactly the same as in respectively 1 and 3, this means that Latplan and R-latplan are robust against the noise (up to certain amount).

For experiments **3-4**, i.e. with partial DFA, R-latplan generated PDDLs that lead Fast Downward to find the optimal plans for all problem considered (see an example in figure

⁴The GA source code can be found in the latplan’s original code <https://github.com/guicho271828/latplan>

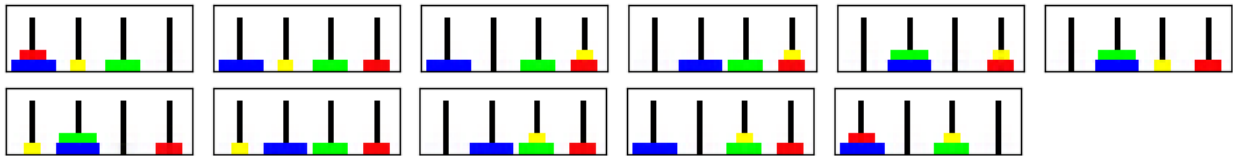


Figure 4.7: Path found by FD from a PDDL generated by R-latplan in Data3 (with the 1-step transition/solution pruned out)

	Lengths of surviving paths	Number of transitions (mean)
Hanoi	10, 10, 10	1028 (vs 1459)
Blocksworld	12, 12, 13	267 (vs 272)
Sokoban	18, 19, 20	4069 (vs 4152)

Table 4.3: lengths of paths found with R-latplan for **Exp3,4**

4.7). On the other hand, Latplan generated PDDLs on which FD found plans of size 1 for all problems. This means that Latplan associated one of the actions it learned (out the 6000) to the 1-step path between the two images, yet this 1-step path was pruned out from the training dataset, if it shows the capacity of Latplan to generalize over unseen transitions it also prove the non reliability in that the produced actions are not conform to the data.

Finally, **RQ3** is linked to experiment **5** namely testing whether R-latplan is robust against errors induced by the transition identifier function, see also Figure 4.2. As discussed in Section 4.4, we check for **redundancy** among the learned actions, which leads us to the results in Table 4.4 showing, e.g. for Hanoi, that 72 groups of redundant actions were found and correspond to the 5% of transitions that were duplicated. These results thus support our claim that R-latplan is robust against errors induced by the transition identifier function.

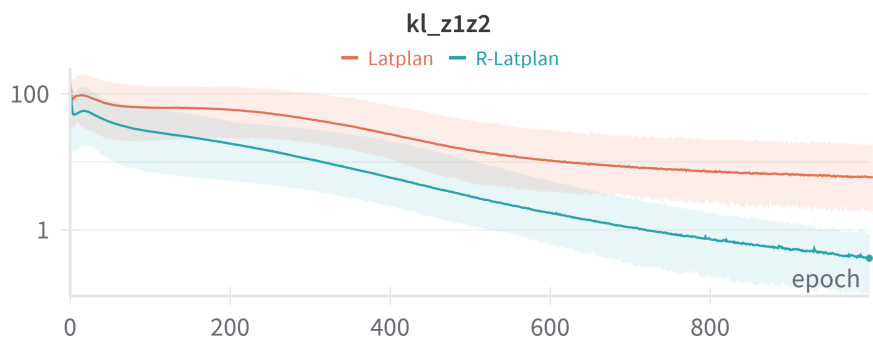


Figure 4.8: Average error (log-scale) and standard deviation in predicting the successor state after applying an action

	Number of groups of redundant actions found	Ground truth
Hanoi	72	72
Blocksworld	14	14
Sokoban	207	207

Table 4.4: Results of **Exp5** showing R-latplan’s robustness against errors produced by the transition identifier function.

4.7 Related Works

In [38], the authors present an approach for automatically generating symbolic action definitions using unlabelled pairs of images representing transitions. They identify objects and their locations by detecting pixel areas changes in a transition. Then, they provide each *object* and *location* with a unique identifier with which they ground two general predicates for all the images: (at ?1 - location ?2 - image object) and (clear ?1 - location). To ground the actions, they use the atoms that change within a transition as *preconditions*, and the value of these changes as *effects*. This approach does not scale well on large state spaces, e.g. the search for new *locations* grows exponentially with the size and complexity of the images. Also, it is not robust against noise since any change in a pixel colour can result in a different *location* area and in the duplication of semantically equal actions. As demonstrated in the following, our proposed system, R-latplan, is domain-independent and partly robust to noisy images.

In [2], the authors implement a system that uses a deep auto-encoder network that takes as input an action and a camera image and outputs the pixel changes happening in a transition. The discrete latent layers hold the information of the image changes associated with an action, and their output is used to train a decision tree that learns probabilistic rules about the action’s effects. Their approach works well for tasks not requiring a global representation of the states, like moving objects with a robot arm but fails on tasks where it is needed, like the MNIST sliding puzzle.

In [91], the authors developed a model that learns a lifted action model from visual traces. In particular, they learn first-order representation of states and actions from sequences of image-action pairs. Their semi-supervised approach uses a manually labelled final state for each trace, therefore, the burden of manually labelling the final state grows exponentially

	[38]	[2]	[91]	[59]	[9]	R-latplan
Subsymbolic representation learning	NO	YES	YES	NO	YES	YES
No need of human-labelled data	NO	YES	NO	YES	YES	YES
Reliability of the learned action models	YES	NO	NO	YES	NO	YES

Table 4.5: Gap analysis with the existing state-of-the-art.

with the complexity of the problem.

In [59] the authors use a top-down approach where they search, using a SAT solver, in a domain space defined from predicates returned by an image parser, in particular the domain to be found must complies with all the partial graphs representing pieces of the state space given as input. As in the other papers presented here, using a parser for retrieving predicates in the images poses a problem since it is prone to identification errors. Furthermore, the solving time the SAT solver uses to find the best domain grows exponentially with the domain complexity.

Table 4.5 summarizes the similarities and differences between the state-of-the-art and our proposed system, R-latplan.

Chapter 5

RC-latplan: compressing the action model

In this chapter we introduce our follow-up work on **R-latplan**, where we describe a method to post-process the PDDL model in order to obtain a compressed action model where each action is a compression of low-level actions obtained in **R-latplan**. Our last and successful approach in these attempts is called **RC-latplan** for **Reliable Clustered Latplan**.

5.1 Intuition

From chapter 4 we know that **R-latplan** returns perfectly functioning PDDL domains that enable a planner to find optimal and visually correct plans. We also know that each (low-level) action learned corresponds to a transition in the visual DFA of the domain at hand, and lastly we know the high-level action associated with each low-level action.

In Chapter 3, among the different approaches we experimented with to improve **Latplan** (without modifying the architecture), we tried to reduce the number of actions to be learned down to the number of high-level grounded actions. This failed and showed the limitation of **Latplan** to learn such a compressed representation of the actions. Yet, having a compressed representation of the actions performed by an agent could lead to various improvements in terms of speed search from the planner (which would use less actions), memory consumption, and also maybe in terms of explainability.

We now approach the problem from another angle, *i.e.*, to use **R-latplan** generated actions as a base, in order to build a more compressed representation.

Let us say that for the high-level grounded action *unstack(Block C)* in the Blocksworld domain, we have the following low-level actions from R-latplan:

```
(:action lla6
  :parameters ()
  :precondition (and (z0) (z2) (z3)
                    (not (z5)) (not (z6))
                    (z14) (not (z20)) (not (z24)))
  :effect (and (not (z0)) (z6) (z7)
              (z8) (not (z14)) (z24)))

(:action lla15
  :parameters ()
  :precondition (and (z1) (not (z3)) (not (z6))
                    (not (z8)) (not (z9))
                    (z19) (not (z22)) (not (z23)))
  :effect (and (not (z1)) (z3)
              (z6) (z9) (not (z17)) (z22)))

(:action lla255
  :parameters ()
  :precondition (and (not (z0)) (z1) (not (z2))
                    (z3) (not (z5)) (z6) (z8))
  :effect (and (not (z1)) (z2) (not (z3))
              (z5) (z14) (z23)))
```

In a first approach (next section), we introduce a top-to-bottom algorithm where we use all the low-level actions of a specific high-level grounded action in order to build a single PDDL action by using Decision Trees to identify patterns among the low-level actions preconditions and effects that we then translate into the preconditions and effects clauses for the new high-level action. Such a high-level grounded action could look like this:

```
(:action hla5
  :parameters ()
  :precondition (and (not (z3)) (not (z4))
                    (z15) (not (z3)) (not (z11))
                    (or
                     (and (z12) (z14) (not (z8)) (not (z10)) )
                     (and (z0) (z5) (not (z9))
                          (not (z11)) (not (z13)) (not (z15)))
                    ))
  :effect (and
    (z3)
    (when (and (z2) (z6) (z12) (z14)
              (not (z8)) (not (z10)) )
      (not (z2))
    )
  )
```

```

      (when (and (not (z1)) (not (z7)) (not (z11))
                (or
                  (and (z9) (z15) (not (z6)))
                  (not (z13)) (not (z15)))))
        (z7)
      )
    (when (and (z5) (not (z15))
              (or
                (and (z7) (z9) (z11) (z13)
                    (z14) (not (z0)) (not (z6)))
                (and (z0) (z6) (not (z1)) (not (z13)))))
      (not (z10))
    )
  )
)

```

In a second approach, called **RC-latplan** (see section 5.3), we employ a bottom-up approach where we use a clustering algorithm that clusters low-level actions of a same high-level grounded action into multiple clusters. We then build, a PDDL action, for each cluster:

```

(:action cla5_6
  :parameters ()
  :precondition (and (z5) (z9) (z14) (not (z2))
                    (not (z4)) (not (z7)) (not (z12)))
  :effect (and (z3) (z15) (not (z0)) (not (z14))
              )
)

(:action cla5_2
  :parameters ()
  :precondition (and (not (z3)) (not (z4))
                    (or (and (z5) (z7) (z13)
                            (not (z11)) (not (z13)) (not (z15)))
                        (and (z0) (z5) (not (z9))
                            (not (z11)) (not (z13)) (not (z15)))))
  :effect (and
    (z3)
    (when (and (not (z1)) (not (z7)) (not (z11))
              (or (and (z9) (not (z6))) (not (z13)))
              (z7)))
    (when (and (z5) (not (z15))
              (or
                (and (z7) (z9) (z11) (z13)
                    (z14) (not (z0)) (not (z6)))
                (and (z0) (z6) (not (z1))
                    (not (z7)) (not (z9)) (not (z11)) (not (z13)))))

```

```

        (not (z10)))
    )
)

```

In both approaches, we use conditional effects [67], that are defined as follow:

Definition 5.1.1 (Conditional effect). A **conditional effect** for a grounded action, is a pair $\langle c, e \rangle$, where:

- c is a formula (the *condition*) in propositional logic, and
- e is an effect (usually a literal or set of literals).

Given a state s , a conditional effect $\langle c, e \rangle$ is applied if and only if $s \models c$, in which case the effect e is executed (i.e., the state is updated according to e). In PDDL-like notation, a conditional effect is written as:

(**when** c e)

This denotes that the effect e only takes place if the condition c holds in the state where the action is applied.

5.2 Method that did not work: Using Decision Trees

5.2.1 Intuition

Since we know that a high-level action is associated with multiple low-level actions, we hypothesize that some correlations exist among the low-level actions of a same high-level action. Specifically, similarities in the activation of certain effects given certain preconditions could exist, in which case using a Decision Tree could help identifying consistent patterns in these mappings, enabling more interpretable and generalizable representations of high-level behaviors and ideally enabling a direct and functional translation from low-level PDDL actions to high-level PDDL actions. In the next section we describe what decision trees are and we illustrate the specific case of the CART [61] algorithm that we used in our experiments.

5.2.2 Learning via Decision trees: a classification task

A Decision Tree (DT) [18] is a supervised learning algorithm that models decisions and their possible consequences as a tree-like structure, where each internal node represents a

decision based on a feature, each branch represents an outcome of that decision, and each leaf represents a final prediction or outcome. It is used for both classification and regression tasks, splitting data recursively based on impurity measures like Gini [93] or entropy to create an interpretable and hierarchical decision-making process.

In the context of our work, we constructed a decision tree (see example in figure 5.1) where a decision about the application of an effect (leaves) is taken depending on the presence of preconditions (root and intermediate nodes).

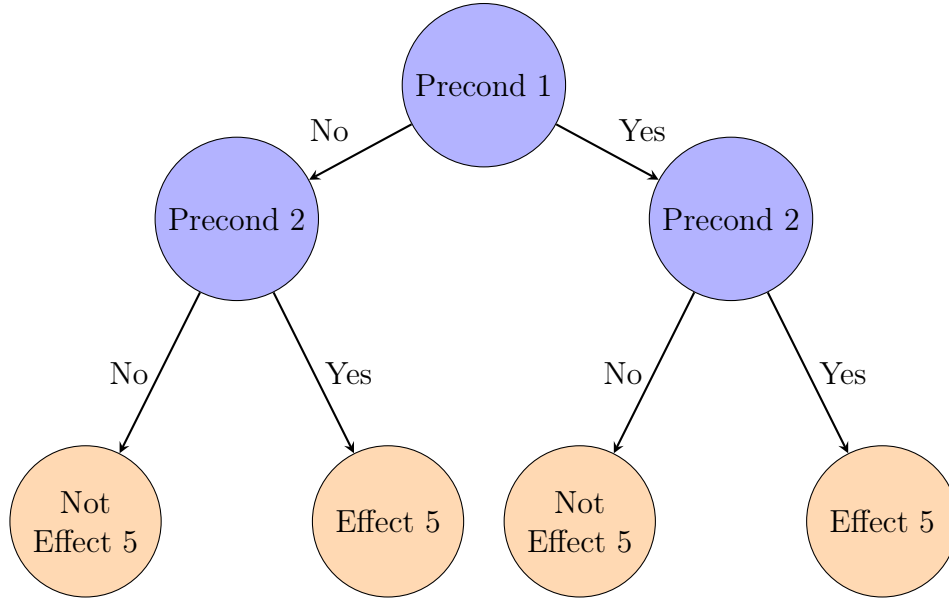


Figure 5.1: A simple decision tree with Precondition 1 as the root, Precondition 2 as intermediate nodes, and the application (or not) of Effect 5 as the leaves.

The CART Algorithm The CART (Classification and Regression Tree) algorithm is a greedy, recursive method for constructing decision trees. At each step, it evaluates all possible splits of the dataset based on a feature and a threshold, selecting the one that maximizes the reduction in uncertainty (or "impurity") in the resulting subsets.

Although CART typically uses the Gini impurity as its default splitting criterion, in our case we opt for the *entropy*, a measure derived from information theory [75]. Entropy quantifies the level of uncertainty or disorder in a dataset and is particularly well-suited for classification tasks where it is desirable to make confident decisions.

Definition 5.2.1. The entropy $H(D)$ of a dataset D is defined as:

$$H(D) = - \sum_{c \in C} p_c \log_2(p_c)$$

where C is the set of possible classes in the dataset and p_c is the proportion of samples in D that belong to class c . A lower entropy indicates a more "pure" set, where the data points are more likely to belong to a single class.

By choosing splits that maximize the reduction in entropy — also known as *information gain* — the CART algorithm constructs a tree that partitions the data in a way that leads to highly confident, interpretable predictions.

Algorithm 2 The CART greedy algorithm uses entropy as the splitting criterion, iteratively maximizing the entropy reduction to grow the decision tree.

Require: Training data $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$, features F

Ensure: Decision tree T

- 1: Initialize the tree T as a single root node containing D
- 2: Define entropy for a dataset D :

$$H(D) = - \sum_{c \in C} p_c \log_2(p_c)$$

where p_c is the proportion of samples in D belonging to class c , and C is the set of unique classes.

- 3: **while** stopping criteria are not met (e.g., max depth, min samples per leaf) **do**
- 4: **for** each node N in T **do**
- 5: **for** each feature $f \in F$ **do**
- 6: **for** each possible split value s of f **do**
- 7: Split D in N into D_{left} and D_{right} using $f \leq s$
- 8: Compute the entropy reduction:

$$\Delta H = H(D) - \frac{|D_{\text{left}}|}{|D|} H(D_{\text{left}}) - \frac{|D_{\text{right}}|}{|D|} H(D_{\text{right}})$$

- 9: **end for**
 - 10: **end for**
 - 11: Choose the split (f^*, s^*) that maximizes ΔH
 - 12: Apply the split (f^*, s^*) to node N
 - 13: **end for**
 - 14: **end while**
 - 15: Assign predictions (e.g., class labels) to the leaf nodes **return** Decision tree T
-

5.2.3 Overview of the Approach

Our approach involves applying Decision Tree learning to induce symbolic high-level actions from the low-level actions. Specifically, we learn two types of predictions using decision trees:

1. **Learning to predict effects in terms of preconditions:** for each high-level action in the domain and for each possible effect, we train a decision tree to predict whether that effect should be applied, based on the presence or absence of certain low-level preconditions.
2. **Learning to predict action applicability in terms of preconditions:** in addition to modeling individual effects, we also train a decision tree to determine whether the high-level action itself is applicable in terms of the low-level preconditions. This helps identify a general precondition clause for the high-level action.

To construct our training data, we use the low-level actions generated by R-latplan. Each low-level action corresponds to a transition in the visual DFA of the domain. Consequently, each effect of a low-level action is treated as a candidate effect for the associated high-level action, and each precondition is treated as a possible precondition of that high-level action.

In order to allow a single high-level (grounded) action to apply different effects under different sets of preconditions, we leverage conditional effects as defined in Section 5.1.1.

Learning to Predict Effects in Terms of Preconditions

Our initial approach was to learn conditional effects, where each effect of a high-level action is predicted using a Decision Tree whose input features are the preconditions. For each high-level action and each of its potential effects, we selected the corresponding low-level actions from R-latplan and divided them into two groups: one where the effect is present, and one where it is not. This allowed us to train a Decision Tree to predict the applicability of the effect in a given state.

After training a Decision Tree for each effect of each high-level action, we only retained the paths that lead to a **True** (positive) leaf — such as the blue leaf shown in Figure 5.2. Importantly, we considered only *right-branching paths*, *i.e.*, branches taken when a condition is true, which corresponds to the presence of a precondition in most samples.

For example, consider the Decision Tree in Figure 5.2, where we use the following notation for a variable z_i (noted z_i): z_i_0 stands for the negative literal $\sim z_i$ and z_i_1 stands

for the positive literal z_i . In the blue node just below the root, condition $z_4_1 \leq 0.5$ implies that when the precondition $\mathbf{z4}$ is present (encoded by the value 1), the decision path follows the right branch. This node is colored blue because the majority of training samples (48 out of 51) have $\mathbf{z4}$ as part of their preconditions. Thus, following the right branch indicates that the precondition is likely satisfied in the state. Conversely, following the left branch does not mean the precondition is violated — only that it is usually absent in the data.

To construct a conditional effect (i.e., the condition part of a **when** clause), we collect the set of literals along a right-branching path, for example:

(and $\neg z2$ $z4$ $\neg z12$)

This yields a conditional effect expressed in PDDL as:

(when
 (and $\neg z2$ $z4$ $\neg z12$)
 $z2$)

Learning to Predict Action Applicability in Terms of Preconditions

Due to the poor planning performance observed with the previous method, we extended our approach by also learning a precondition clause for each high-level action as a whole. We again employed Decision Tree learning — this time to predict whether a high-level action should be applied in a given state.

As before, we extracted right-branching paths leading to **True** leaves, and used the conjunction of literals along each such path as a possible precondition clause. These clauses were then used as the explicit preconditions of the high-level actions in the generated PDDL model.

This complementary step aimed at reducing the overall complexity of the domain by explicitly constraining action applicability, in the hope that doing so would help the planner to prune infeasible branches early during the search process.

Results and Observations

Introducing precondition clauses improved planning performance slightly: the Fast Downward planner was able to generate visually correct plans for problems involving up to three

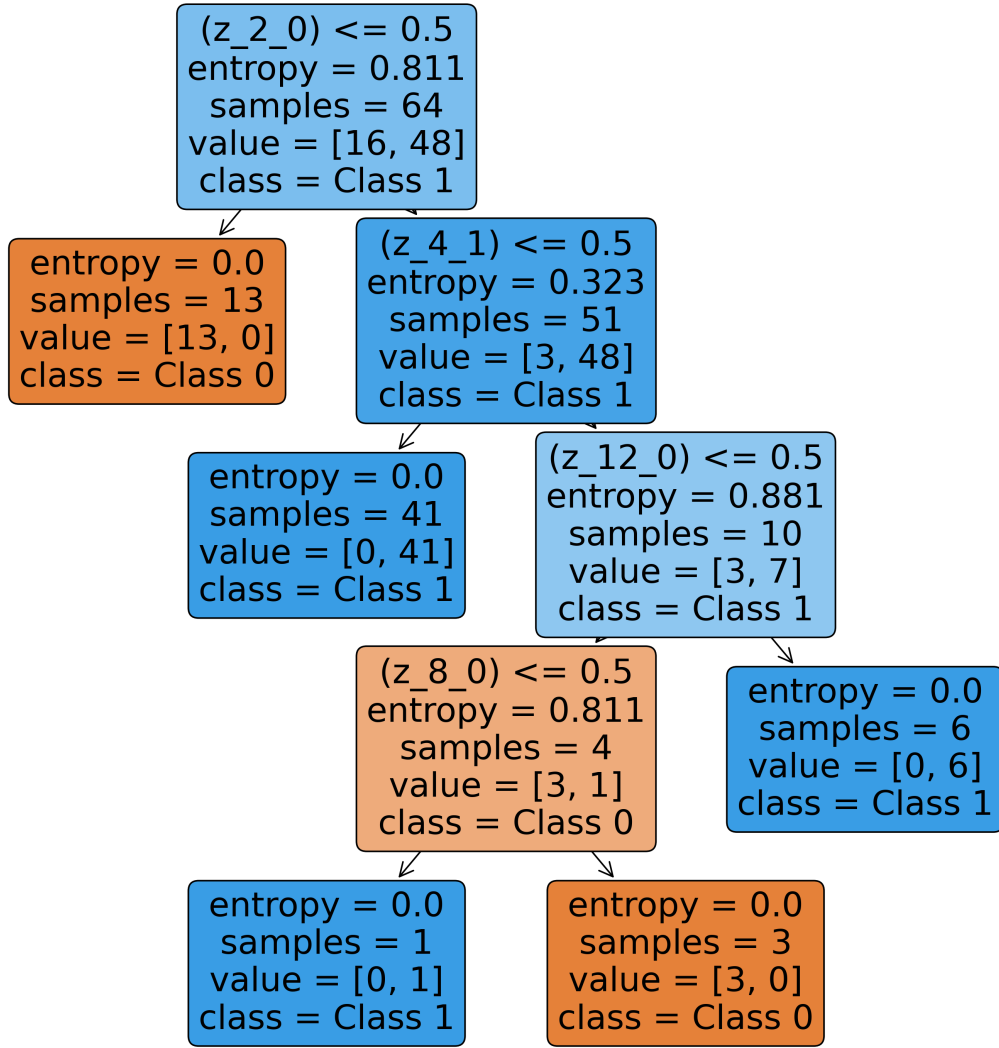


Figure 5.2: Decision Tree predicting the effect "add z2" for high-level action $move(d1, peg2)$ in Hanoi.

steps. However, for longer sequences, the plans generated were no longer consistent with the intended behavior.

Overall, despite extensive testing across various domains and configurations, neither of the decision tree-based methods yielded satisfactory results. We believe that the action model generated by R-latplan is highly sensitive to even the slightest modification. However, Decision Trees perform statistical inference between input data and the output, thereby introducing noise in the way preconditions are linked to the effects, which might break the logical coherence of the action model.

5.3 RC-latplan with Hierarchical Clustering

In examining the R-latplan PDDL domains through manual inspection, we observed that low-level actions associated with the same high-level action often share structural similarities, particularly in their effects. This observation motivates our approach: to cluster low-level actions and investigate whether each cluster can be represented as a distinct PDDL action.

5.3.1 The Hierarchical Clustering Method

Motivation on the type of clustering

We want to cluster low-level actions of the same high-level action. Depending on the domain and on the high-level action, the number of low-level action varies a lot (*e.g.*, from 10 to 100), furthermore we have, a priori, no idea on the number of clusters each high-level action should have. A famous clustering method like K-means [60] [63] requires K, the numbers of clusters, as an input parameter. Instead, we choose a clustering method that finds this number automatically, namely the Hierarchical Clustering.

Choice of a distance

For the distance we choose the Jaccard Distance defined here after.

Definition 5.3.1 (Jaccard Distance for Binary Vectors [81]). Let $\mathbf{u}$ and $\mathbf{v}$ be two binary vectors of equal length. Define:

- M_{11} as the number of positions where both $\mathbf{u}$ and $\mathbf{v}$ are 1,
- M_{10} as the number of positions where $\mathbf{u} = 1$ and $\mathbf{v} = 0$,
- M_{01} as the number of positions where $\mathbf{u} = 0$ and $\mathbf{v} = 1$.

Then, the *Jaccard similarity coefficient* is given by:

$$J(\mathbf{u}, \mathbf{v}) = \frac{M_{11}}{M_{11} + M_{10} + M_{01}}$$

and the *Jaccard distance* is defined as:

$$D_J(\mathbf{u}, \mathbf{v}) = 1 - J(\mathbf{u}, \mathbf{v}) = \frac{M_{10} + M_{01}}{M_{11} + M_{10} + M_{01}}$$

The Jaccard distance is often used when working with binary vectors representing sparse data, where a 1 indicates the presence of a feature and 0 indicates absence. This is because Jaccard distance ignores shared absences, focusing instead on the proportion of shared presences. In contrast, Hamming distance treats all differences equally — even when both vectors lack a feature, which may not carry useful information in many real-world contexts (e.g., recommendation systems, text analysis, etc.).

Example 3. Let the binary vectors be:

$$\mathbf{u} = [1, 0, 0, 1, 0], \quad \mathbf{v} = [1, 1, 0, 0, 0]$$

We compute:

- $M_{11} = 1$ (only position 1 has both 1),
- $M_{10} = 1$ (position 4),
- $M_{01} = 1$ (position 2).

Jaccard distance:

$$D_J(\mathbf{u}, \mathbf{v}) = \frac{M_{10} + M_{01}}{M_{11} + M_{10} + M_{01}} = \frac{2}{3} = 0.\bar{6}$$

Interpretation: The Jaccard distance shows that only 1 out of 3 active features is shared, which reflects the low semantic similarity when presence is what matters.

Hierarchical Clustering

The hierarchical clustering method occurs in five steps:

STEP 1: Choose a Distance Metric and compute a distance matrix between all data points As seen previously we chose the Jaccard distance. We then computed the distance matrix that shows pairwise distances between points.

Penalty on the Jaccard Distance

An analysis on the low-level actions of same high-level action showed us that, many of these low-level actions share contradictory literals in their preconditions. Since we want to build PDDL actions from the clusters, we also want to avoid as much as possible the presence

of such contradictory literals. Indeed, in the process of constructing the action (as shown later), we first create a general precondition clause that consists of the conjunction of single literals and of a disjunction of conjunctions of literals, for example:

:precondition (**and** z1 z2 ... (**or** (**and** z1 z3 ...) (**and** z4 z5 ...)

the disjunction of conjunctions is the disjunctions of all the preconditions of the low-level actions of the cluster **pruned** of the intersection literals (that are present in the left part of the expression). Having too many contradictory literals between different low-level preconditions could prevent factorizing this right part further, this is why we introduced the penalty on the contradictory preconditions as a hyper parameter for our tests. Note that the low-level preconditions of a low-level action are defined with a binary vector where the first half (left part) accounts for positive literals and the second half (right part) for negative literals, such that for a vector of size 32, the first element indicates the truth value of z_0 and the 17th element the truth value of $\sim z_0$.

Definition 5.3.2 (Jaccard Distance with Penalty term).

$$D_{PJ}(\mathbf{u}, \mathbf{v}) = \frac{M_{10} + M_{01}}{M_{11} + M_{10} + M_{01}} + \lambda \cdot \frac{C}{L}$$

M_{11} = Number of positions where $u_i = 1$ and $v_i = 1$

M_{10} = Number of positions where $u_i = 1$ and $v_i = 0$

M_{01} = Number of positions where $u_i = 0$ and $v_i = 1$

Left_x = Left half of vector x

Right_x = Right half of vector x

$C = |\text{Left}_u \wedge \text{Right}_v| + |\text{Left}_v \wedge \text{Right}_u|$ (cross-half overlap)

L = Length of the binary vectors

$\lambda > 0$ (penalty weight)(*settolinourexperiments*)

STEP 2: Choose a Linkage Method In this step, we decide how distances between clusters are calculated as you we will merge them.

The various choices are:

- **single**: nearest point between clusters.
- **complete**: farthest point between clusters.
- **average**: average pairwise distance.
- **ward**: minimizes total variance (not recommended for binary data).

STEP 3: Agglomerative Clustering: In this step, we iteratively merge data points into clusters. More specifically:

1. Start with each point as its own cluster.
2. Iteratively:
 - (a) Merge the two closest clusters (based on linkage and distance metric).
 - (b) Update the distance matrix (in particular, compute the new distances between the new cluster and the other ones).
3. Repeat until all points are in one cluster.

STEP 4: Build the Dendrogram The Dendrogram is a tree diagram showing which clusters were merged at what distance. The y-axis typically represents the distance (or dissimilarity) at which clusters were joined.

For instance, in figure 5.3, which is an example of a Dendrogram used in our work, we can see that at distance 0, the data points 2 and 9 were merged into one cluster.

STEP 5: Cut the Tree In this final step we chose a threshold (a distance) for cutting the tree, which means that we consider all the clusters formed at this threshold. Going back to the dendrogram of figure 5.3; just for the sake of an example, if we decide to cut the tree at threshold 0.6, we obtain 3 clusters:

- cluster 1 with elements: 0, 6, 4, 1, 8
- cluster 2 with element 7
- cluster 3 with elements: 2, 3, 5 and 9

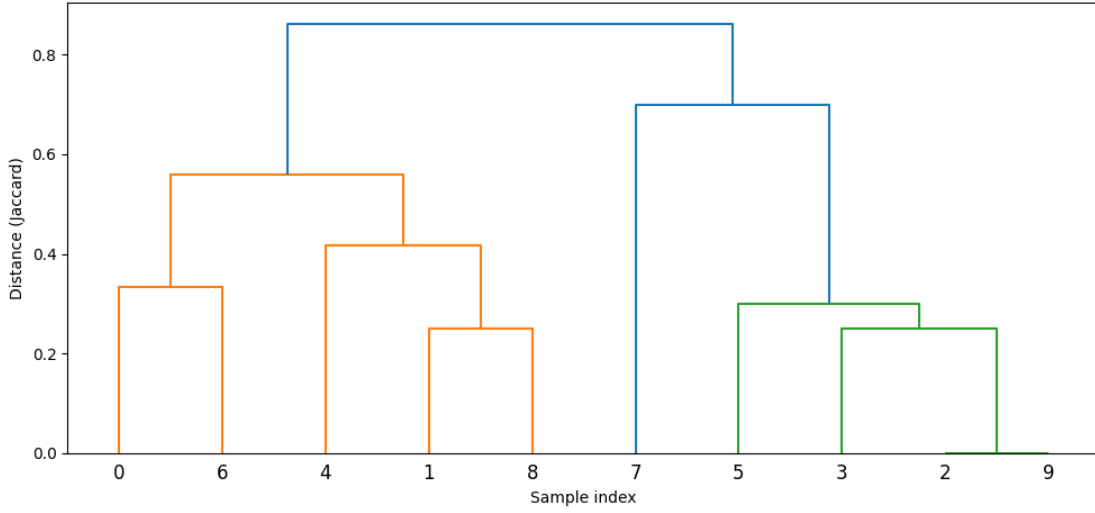


Figure 5.3: Dendrogram of clustering low level actions

Importantly, this "cutting" can also be done automatically by computing a score for each clustering and keeping the clustering with the best score.

In our work we chose the *Silhouette score* for deciding, automatically, when to cut the tree.

Definition 5.3.3 (Silhouette score). Let $a(i)$ be the average distance between point i and all other points in the **same cluster**, and let $b(i)$ be the minimum average distance from point i to all points in any **other cluster** (i.e., the nearest neighboring cluster). Then the **Silhouette score** for a single point i is defined as:

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}$$

The value of $s(i)$ lies in the interval:

$$-1 \leq s(i) \leq 1$$

The **overall Silhouette score** for a cluster of data points (with n points) is the average of the individual scores:

$$S = \frac{1}{n} \sum_{i=1}^n s(i)$$

	only_preconds	only_effects	both
True	#cluster: 91	#cluster: 55	#cluster: 67
	silhouette: 0.24	silhouette: 0.9	silhouette: 0.41
	#con_pre: 0.46	#con_pre: 3.29	#con_pre: 1.45
	#con_eff: 0.0	#con_eff: 0.0	#con_eff: 0.0
False	#cluster: 84	#cluster: 55	#cluster: 48
	silhouette: 0.23	silhouette: 0.9	silhouette: 0.43
	#con_pre: 0.76	#con_pre: 3.29	#con_pre: 4.0
	#con_eff: 0.0	#con_eff: 0.0	#con_eff: 0.0

Table 5.1: Comparison of clustering low-level actions of the high-level action $move(d1, d2)$, under different conditions.

#cluster: number of clusters,

#con_pre: number of contradictions among the preconditions,

#con_eff: number of contradictions among the effects.

5.3.2 Testing different hyperparameters for the clusterings

In order to cluster low-level actions using the clustering algorithm just described, we tested different configurations for the following two hyperparameters:

- data source for the clustering, *i.e.*, we used a binary representation for representing the low-level actions, and we tested either the whole action itself (a set of preconditions and a set of effects) or either only the preconditions or only the effects.
- penalty on the Jaccard distance, *i.e.*, we tested with or without the penalty term on the Jaccard distance, as described in 5.3.2.

Results of the different clustering scenarios

In table 5.1, we show different metrics about the clusters in different scenarios, for a specific high-level action ($move(d1, d2)$ in Hanoi).

Knowing that high-level action $move(d1, d2)$ corresponds to 192 low-level actions, we notice that the number of clusters made out of these low-level actions is from around 2.10 to 3.5 lower than the number of llas.

Also, the highest silhouette score is when we use only the effects as base data for constructing the clusters. In this scenario, the presence or not of the penalty does not change any metric (indeed, since the penalty is about the preconditions).

We obtain similar results for the other two domains: Sokoban and Blocksworld.

For the rest of our experiments, we then choose to learn clusters only based on effects.

5.3.3 Translation of the clusters into PDDL actions

First, let us introduce some notation:

- hla_i : high-level action i
- we use the letter c to specify the index of a cluster associated to a high level action
- lla_{jc} : low-level action j of cluster c.
- llp_{jc} : low-level preconditions set for lla_{jc}
- lle_{jc} : low-level effects set for cluster c.
- llp_{kjc} : low-level precondition k for low-level precondition set llp_{jc} .
- lle_{kjc} : low-level effect k for the low-level effect set lle_{jc} .

The pseudo-code for deriving a PDDL action for each cluster is given in 5.3.3.

For each cluster of each high-level action, it does the following:

1. (l. 3-24) construct the Precondition clause as a logical formula of the form:

: precondition (**and** intersect_preconds (union_of_pruned_lpps))

2. (l. 25-47) construct the Effect clause as a logical formula of the form:

: effect (**and** intersect_effects when_clauses)

Here is an example of the construction of the two clauses (Precondition and Effect) for one cluster of a high-level action from Hanoi:

First, we retrieve for each low-level action, the corresponding low-level preconditions:

$$llp1 = \{z_1 z_2 z_3 z_4 z_5 z_6 z_7 z_9 z_{14} z_{15} \neg z_0 \neg z_8 \neg z_{10} \neg z_{11} \neg z_{12} \neg z_{13}\}$$

$$llp2 = \{z_2 z_4 z_6 z_7 z_9 z_{15} \neg z_0 \neg z_1 \neg z_3 \neg z_{10} \neg z_{12} \neg z_{14}\}$$

$$llp3 = \{z_1 z_2 z_3 z_4 z_6 z_7 z_9 z_{15} \neg z_0 \neg z_{10} \neg z_{11} \neg z_{12} \neg z_{13} \neg z_{14}\}$$

$$llp4 = \{z_1 z_2 z_4 z_6 z_9 z_{15} \neg z_0 \neg z_3 \neg z_7 \neg z_{10} \neg z_{12} \neg z_{13} \neg z_{14}\}$$

$$llp5 = \{z_1 z_2 z_3 z_4 z_6 z_7 z_9 z_{13} \neg z_0 \neg z_{10} \neg z_{11} \neg z_{14} \neg z_{15}\}$$

$$llp6 = \{z_1 z_2 z_4 z_6 z_7 z_9 z_{15} \neg z_0 \neg z_3 \neg z_{10} \neg z_{12} \neg z_{13} \neg z_{14}\}$$

$$llp7 = \{z_2 z_4 z_6 z_8 z_9 z_{13} z_{14} \neg z_0 \neg z_1 \neg z_3 \neg z_7 \neg z_{10} \neg z_{11} \neg z_{15}\}$$

$$llp8 = \{z_1 z_2 z_4 z_5 z_6 z_7 z_8 z_9 z_{13} z_{14} \neg z_0 \neg z_3 \neg z_{10} \neg z_{11} \neg z_{15}\}$$

And the corresponding lles:

$$\begin{aligned}
 lle1 &= \{z_0 z_{10} \neg z_4 \neg z_6\} \\
 lle2 &= \{z_0 z_5 z_{10} \neg z_4 \neg z_6\} \\
 lle3 &= \{z_0 z_{10} \neg z_4 \neg z_6\} \\
 lle4 &= \{z_0 z_{10} \neg z_4 \neg z_6\} \\
 lle5 &= \{z_0 z_5 z_{10} \neg z_4 \neg z_6\} \\
 lle6 &= \{z_0 z_{10} \neg z_4 \neg z_6\} \\
 lle7 &= \{z_0 z_{10} \neg z_4 \neg z_6\} \\
 lle8 &= \{z_0 z_{10} \neg z_4 \neg z_6\}
 \end{aligned}$$

In lines 11-16 (see pseudo-code), we start by constructing *precondition_clause* with the intersection of the literals among the preconditions of the low-level actions for the cluster (*inter_llp_c*):

$$precondition_clause = (and\ z_2\ z_4\ z_6\ z_9\ \neg z_0\ \neg z_{10})$$

Then (l.17-33), we continue constructing *precondition_clause* using the *union_pruned_llps_c* which is the union of llps pruned of the literals of *inter_llp_c*:

$$\begin{aligned}
 precondition_clause &= \\
 &(and\ z_2\ z_4\ z_6\ z_9\ \neg z_0\ \neg z_{10}) \\
 &\text{or} \\
 &(and\ z_1\ z_3\ z_5\ z_7\ z_{14}\ z_{15}\ \neg z_8\ \neg z_{11}\ \neg z_{12}\ \neg z_{13}) \\
 &(and\ z_7\ z_{15}\ \neg z_1\ \neg z_3\ \neg z_{12}\ \neg z_{14}) \\
 &(and\ z_1\ z_3\ z_7\ z_{15}\ \neg z_{11}\ \neg z_{12}\ \neg z_{13}\ \neg z_{14}) \\
 &(and\ z_1\ z_{15}\ \neg z_3\ \neg z_7\ \neg z_{12}\ \neg z_{13}\ \neg z_{14}) \\
 &(and\ z_1\ z_3\ z_7\ z_{13}\ \neg z_{11}\ \neg z_{14}\ \neg z_{15}) \\
 &(and\ z_1\ z_7\ z_{15}\ \neg z_3\ \neg z_{12}\ \neg z_{13}\ \neg z_{14}) \\
 &(and\ z_8\ z_{13}\ z_{14}\ \neg z_1\ \neg z_3\ \neg z_7\ \neg z_{11}\ \neg z_{15}) \\
 &(and\ z_1\ z_5\ z_7\ z_8\ z_{13}\ z_{14}\ \neg z_3\ \neg z_{11}\ \neg z_{15}))
 \end{aligned}$$

Now, we start to construct the *effect_clause* using the intersection of all the literal effects in the cluster (*inter_lle_c*):

$$\begin{aligned} effect_clause = \\ (and\ z_0\ z_{10}\ \neg z_4\ \neg z_6 \end{aligned}$$

Finally, for each effect literal that is among the *lles* but not in their intersection, we construct a “when” clause, in our case only the literal z_5 is not in the intersection. First, we consider only the *llas* that have z_5 in their effects, namely *lla2* and *lla5*. Then, we consider their pruned preconditions, namely:

$$\begin{aligned} llp2_pruned &= \{z_7\ z_{15}\ \neg z_1\ \neg z_3\ \neg z_{12}\ \neg z_{14}\} \\ llp5_pruned &= \{z_1\ z_3\ z_7\ z_{13}\ \neg z_{11}\ \neg z_{14}\ \neg z_{15}\} \end{aligned}$$

and take their intersection:

$$intersect_precond_when = z_7\ \neg z_{14}$$

We prune out from the given pruned *llps* any literal of this intersection:

$$\begin{aligned} llp2_pruned_bis &= \{z_{15}\ \neg z_1\ \neg z_3\ \neg z_{12}\} \\ llp5_pruned_bis &= \{z_1\ z_3\ z_{13}\ \neg z_{11}\ \neg z_{15}\} \end{aligned}$$

and we construct the following “when” clause (in PDDL format):

$$\begin{aligned} &(\text{when } (\text{ and } z_7\ \neg z_{14} \\ &\quad (\text{or } (\text{ and } z_{15}\ \neg z_1\ \neg z_3\ \neg z_{12}) (\text{ and } z_1\ z_3\ z_{13}\ \neg z_{11}\ \neg z_{15})) \\ &\quad z_5 \\ &)) \end{aligned}$$

Finally, we can construct the full action for the cluster:

```

(:action a0_0

:precondition (and
    z2 z4 z6 z9 ¬z0 ¬z10
    (or
        (and z1 z3 z5 z7 z14 z15 ¬z8 ¬z11 ¬z12 ¬z13)
        (and z7 z15 ¬z1 ¬z3 ¬z12 ¬z14)
        (and z1 z3 z7 z15 ¬z11 ¬z12 ¬z13 ¬z14)
        (and z1 z15 ¬z3 ¬z7 ¬z12 ¬z13 ¬z14)
        (and z1 z3 z7 z13 ¬z11 ¬z14 ¬z15)
        (and z1 z7 z15 ¬z3 ¬z12 ¬z13 ¬z14)
        (and z8 z13 z14 ¬z1 ¬z3 ¬z7 ¬z11 ¬z15)
        (and z1 z5 z7 z8 z13 z14 ¬z3 ¬z11 ¬z15)
    )
)

:effect (and
    ¬z0 z10 ¬z4 ¬z6
    (when (and z7 ¬z14 (or (and z15 ¬z1 ¬z3 ¬z12)
        (or z1 z3 z13 ¬z11 ¬z15))))
    z5
)
)

```

Algorithm 3: Construction of a PDDL action for each cluster

- 1: **for** each high-level action hla_i **do**
- 2: **for** each cluster c in hla_i **do**
 - // Building the precondition clause for the cluster c*
 - 3: $inter_llp_c \leftarrow \bigcap_j llp_{jc}$ $\triangleright$ Intersection of all the precondition literals in c
 - 4: $pruned_llps_c \leftarrow []$
 - 5: **for** each index j of low-level actions lla_{jc} in cluster c **do**

```

6:       $reduced\_llp_{jc} \leftarrow llp_{jc} \setminus inter\_llp_c$   $\triangleright$  we remove intersection literals from each
      llp
7:      Append  $reduced\_llp_{jc}$  to  $pruned\_llps_c$ 
8:  end for
9:   $union\_pruned\_llps_c \leftarrow \bigcup_j pruned\_llps_c[j]$   $\triangleright$  union of the pruned llps
10:   $precondition\_clause \leftarrow$  empty string
11:  if  $inter\_llp_c \neq \emptyset$  then
12:      Append ‘(AND’ to  $precondition\_clause$ 
13:      for each literal  $p$  in  $inter\_llp_c$  do
14:          Append  $p$  to  $precondition\_clause$  with a space
15:      end for
16:  end if
17:  if  $union\_pruned\_llps_c \neq \emptyset$  then
18:      Append ‘(OR’ to  $precondition\_clause$ 
19:      for each set  $llp\_pruned$  in  $union\_pruned\_llps_c$  do
20:           $and\_clause\_tmp \leftarrow$  empty string
21:          if  $llp\_pruned \neq \emptyset$  then
22:               $and\_clause\_tmp \leftarrow$  (AND
23:              for each literal  $l$  in  $llp\_pruned$  do
24:                  Append  $l$  to  $and\_clause\_tmp$  with a space
25:              end for
26:              append “)” to  $and\_clause\_tmp$ 
27:          end if
28:          if  $and\_clause\_tmp \neq \emptyset$  then
29:              Append  $and\_clause\_tmp$  to  $precondition\_clause$ 
30:          end if
31:      end for
32:      Append ‘)’ to  $precondition\_clause$ 
33:  end if
34:  if  $inter\_llp_c \neq \emptyset$  then
35:      Append ‘)’ to  $precondition\_clause$ 
36:  end if
37:

```

```

// Building the effects clause for the cluster c
38:    $inter\_lle_c \leftarrow \bigcap_j lle_{jc}$ 
39:    $pruned\_lles_c \leftarrow []$ 
40:   for each index  $j$  of low-level actions  $lla_{jc}$  in cluster  $c$  do
41:      $reduced\_lle_{jc} \leftarrow lle_{jc} \setminus inter\_lle_c$ 
42:     Append  $reduced\_lle_{jc}$  to  $pruned\_lles_c$ 
43:   end for
44:    $effect\_clause \leftarrow$  empty string
45:   if  $inter\_lle_c \neq \emptyset$  then
46:     Append ‘(AND’ to  $effect\_clause$ 
47:     for each literal  $e$  in  $inter\_lle_c$  do
48:       Append  $e$  to  $effect\_clause$  with space
49:     end for
50:   end if
51:    $union\_pruned\_effects_c \leftarrow \bigcup_j pruned\_lles_c[j]$ 
52:   for each literal  $e$  in  $union\_pruned\_effects_c$  do ▷ Building when clauses
53:      $J_e \leftarrow \{j \mid e \in pruned\_lles_c[j]\}$  ▷ indices of llas with effect e
54:      $grouped\_llps \leftarrow \{pruned\_llps_c[j] \mid j \in J_e\}$ 
55:      $intersect\_precond\_effect_e \leftarrow \bigcap_{j \in J_e} pruned\_llps\_c[j]$ 
56:      $union\_precond\_effect_e \leftarrow \bigcup_{j \in J_e} pruned\_llps\_c[j]$ 
57:      $CONDITION \leftarrow$  empty string
58:     if  $intersect\_precond\_effect_e \neq \emptyset$  then
59:       Append ‘(AND’ to  $CONDITION$ 
60:       for each literal  $e$  in  $intersect\_precond\_effect_e$  do
61:         Append  $e$  to  $CONDITION$  with space
62:       end for
63:     end if
64:     if  $union\_precond\_effect_e \neq \emptyset$  then
65:       append "(OR " to  $CONDITION$ 
66:       for each set  $s$  in  $union\_precond\_effect_e$  do
67:          $and\_clause\_tmp \leftarrow (AND$ 
68:         for each literal  $l$  in  $s$  do
69:           append  $l$  to  $and\_clause\_tmp$ 

```

```

70:          end for
71:          append " )" to and_clause_tmp
72:          append and_clause_tmp to CONDITION
73:        end for
74:        append ")" to CONDITION
75:      end if
76:       $EFFECS \leftarrow e$ 
77:      Append string 'when CONDITION then EFFECS' to effect_clause
78:    end for
79:  end for
80: end for

```

5.3.4 Results

Resulting PDDL domains

Table 5.2 shows the number of actions generated by R-latplan against the number of actions in the clustered representation. The clustered representation reduces by a factor of ~ 5.41 the number of actions compared to R-latplan.

Planning performances

We conducted the same experiments as experiments 1 and 2, described in 4.4, namely, we tested Latplan, R-latplan and RC-latplan using crisp and Complete Visual DFA (Data1) and the Noisy and Complete Visual DFA (Data2). As shown in 5.4 we obtain exactly the same results, specifically RC-latplan was able to generate the same plans as R-latplan. We also checked for the semantic of the actions used (since we can associate each clustered action to a high-level action) and noticed that all actions used are correct, *i.e.*, correspond to the high level action that the agent would use.

	Sokoban	Hanoi	Blocksworld
R-latplan	4631	1468	577
RC-latplan	516	377	171

Table 5.2: Number of actions in R-latplan vs RC-latplan

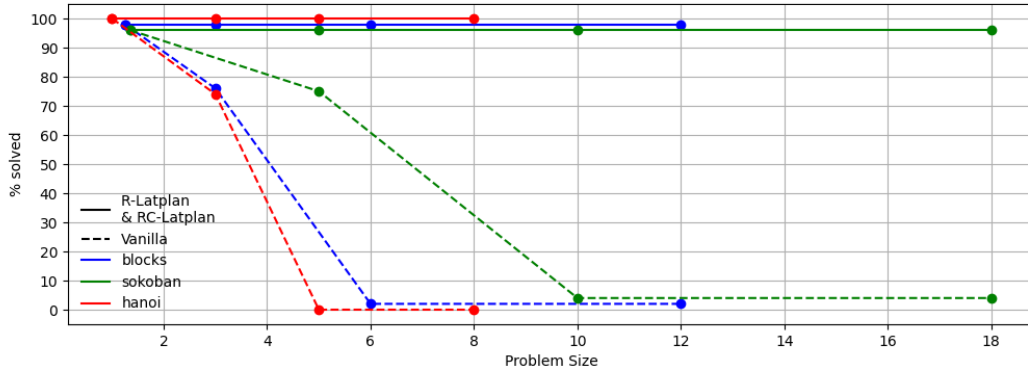


Figure 5.4: Success number of tested executions (**maximum success is 3**) on the longest possible paths from the derived Visual DFAs on **Exp1: Crisp and Complete Visual DFAs** and **Exp2: Noisy and Complete Visual DFAs**

	Speed	# Generated States	Mem. Peak
R-latplan	0.012s	7385	13652 KB
RC-latplan	0.010s	5730	12788 KB

Table 5.3: Planning metrics: R-latplan vs RC-latplan

Since R-latplan and RC-latplan have the same results in terms of the generated plans, we also wanted to compare the speed, the memory and the number of nodes tested during the search (blind) made by Fast Downward. For this test, we used only the Sokoban domain since it has the most numerous number of actions, nodes and the longest possible plans (up to 20), we ran Fast Downward on both R-latplan and RC-latplan, on three 20-steps problems and computed the mean of the three metrics considered. The results are summarized in table 5.3.

Comprehensive comparison of R-latplan and RC-latplan reachability

In order to have a more accurate idea of the difference between R-latplan and RC-latplan representations, we proceeded with the following test (see pseudo-code in Algorithm 4):

1. Find all the pairs of nodes in the Full DFA, such that there exists a path of at least one step between them.
2. For each pair, generate the corresponding PDDL problem file, and, using the domain file (of R-latplan or RC-latplan), test if Fast Downward can solve it.

	#pairs with a path in full dfa	coverage R-latplan	coverage RC-latplan
Hanoi	256	256	238
Blocks	15000	15000	14554
Sokoban	11130	11130	10850

Table 5.4: Comparing DFA coverage of R-latplan and RC-latplan

3. If both R-latplan and RC-latplan can achieve the same number of solutions, it supports the claim of logical equivalence between them.

Algorithm 4 Reachability test between for comparing R-latplan and RC-latplan

```

1: full_dfa_problems  $\leftarrow$  all pairs of distinct reachable nodes in the full DFA
2: pairs_solved_with_R-latplan  $\leftarrow$  [ ]
3: pairs_solved_with_RC-latplan  $\leftarrow$  [ ]
4: for each pair p in full_dfa_problems do
5:   Generate problem.pddl for pair p
6:   Run Fast Downward with domain_R-Latplan.pddl and generated problem.pddl
7:   if a plan is returned and the plan has at least 1 step then
8:     Append p to pairs_solved_with_R-latplan
9:   end if
10:  Run Fast Downward with domain_RC-Latplan.pddl and generated problem.pddl
11:  if a plan is returned and the plan has at least 1 step then
12:    Append p to pairs_solved_with_RC-latplan
13:  end if
14: end for
15:
    // Reporting results
16: Print the size of full_dfa_problems
17: Print the size of pairs_solved_with_R-latplan
18: Print the size of pairs_solved_with_RC-latplan

```

Table 5.4 shows that R-latplan has a complete coverage of the problems in the DFA, whereas the coverage of RC-latplan is slightly lower. This can be explained by the way to clustered actions are built. To illustrate it better, here is an example of a problem that R-latplan would cover but RC-latplan would not:

Example 4. Let's consider two low-level actions of R-latplan, *lla1* and *lla2* with respectively a set of preconditions (*llp1* and *llp2*) and a set of effects (*lle1* and *lle2*):

- *llp1* : { *z1* *z2* }

- lle1 : { z6 }
- llp2 : { z1 z2 z3 }
- lle2 : { z6 z7 }

And a planning problem:

- Initial state: { $\neg z7$ z1 z2 z3 }
- Goal state: { $\neg z7$ z6 }

Let's also say that lla1 and lla2 are clustered into the same cluster in RC-latplan, and let's call the corresponding (clustered) action *cla*:

```
(:action cla
:parameters ()
:precondition (and z1 z2 (or z3))

:effect (
      z6
      (when (z3)
        z7
      )
    )
)
```

If we use the R-latplan domain to solve the problem, FD will choose action lla1 to solve it. But if we run FD with RC-latplan domain, it will not find a solution since in the effect part of *cla* the conditional effect *z7* will also be applied.

This problem can be summarized as follow: *a clustered action with conditional effects applies all the effects that are present among the low-level actions that are applicable in a state.*

5.4 RC-latplan with Clustering on the effects

In order to solve the problem described in example 4, one option is to cluster only low-level actions that have exactly the same effects and to leave the low-level actions that have a

unique set of effects as they are, *i.e.*, R-latplan low-level actions. We will now prove that RC-latplan with this type of clustering is logically equivalent to R-latplan. For the remainder of this proof, we will refer to RC-latplan with clustering on the effects simply as RC-latplan. We then confirm the logical equivalence empirically, by showing the DFA coverage.

5.4.1 Proving logical equivalence

Definition 5.4.1. A planning domain model A is equivalent to a planning domain model B if and only if for any plan using A from an initial state s to a state satisfying the problem goal, we can derive a plan from s using B that reaches the same goal, and vice versa.

Definition 5.4.2 (Entailment). In logic, for two logical propositions A and B, we say that A entails B (written $A \models B$) if B is true in every possible interpretation (or model) where A is true.

Property 5.4.1. If ϕ_i is one of the formulas in the disjunction $\phi_1 \vee \phi_2 \vee \dots \vee \phi_n$, and ϕ_i is true, then:

$$\phi_i \models \bigvee_{j=1}^n \phi_j$$

Property 5.4.2 (Factorization of a disjunction of conjunctions). Let $\phi_i = \bigwedge_{l \in L_i} l$ be a conjunction of literals of the set of literals L_i . For n sets L_i such that $i = 1, \dots, n$; let $C = \bigcap_{i=1}^n L_i$ be the set of literals common to all ϕ_i .

Then, by using the distributivity of $\wedge$ over $\vee$, the following equivalence holds:

$$\bigvee_{i=1}^n \left(\bigwedge_{l \in L_i} l \right) \equiv \left(\bigwedge_{l \in C} l \right) \wedge \left(\bigvee_{i=1}^n \left(\bigwedge_{l \in L_i \setminus C} l \right) \right)$$

i.e., a disjunction of conjunctions can be factorized as a conjunction of the intersection literals and of a disjunction of the conjunctions pruned of the intersection.

Let us introduce useful notations:

- c is a cluster of low-level actions in RC-latplan.
- c_a is the PDDL action made from the actions of cluster c .
- lla_{ic} is the low-level action i of cluster c .
- llp_c refers to the preconditions of low-level actions of cluster c .

- llp_{ic} is the set of preconditions of low-level action i of cluster c .
- lle_c refers to the effects of low-level actions of cluster c .
- lle_{ic} is the set of effects of low-level action i of cluster c .
- p_c is the precondition clause of c_a .
- e_c is the effect clause of c_a .

Property 5.4.3. The precondition clause of an RC-latplan action is logically equivalent to the disjunction of the low-level preconditions of the low-level actions of cluster c .

Proof. Let us consider a low-level precondition set llp_{ic} of the low-level action lla_{ic} belonging to a cluster c :

$$llp_{ic} = \bigwedge_{l \in llp_{ic}} l$$

And the disjunction of llp_c (*i.e.*, of the n low-level preconditions associated with the n low-level actions of c) :

$$\bigvee_{i=1}^n \left(\bigwedge_{l \in llp_{ic}} l \right)$$

Let llp_c^{core} be the set of literals common to all llp_{ic} :

$$llp_c^{\text{core}} = \bigcap_{i=1}^n llp_{ic}$$

Then, by the property 5.4.3, the disjunction can be rewritten as:

$$\left(\bigwedge_{l \in llp_c^{\text{core}}} l \right) \wedge \left(\bigvee_{i=1}^n \left(\bigwedge_{l \in llp_{ic} \setminus llp_c^{\text{core}}} l \right) \right)$$

which is how the precondition clause of RC-latplan is built, as shown in section 5.3.3.

□

Property 5.4.4. For any state where an action from R-latplan is applicable, there exists an action from RC-latplan that is also applicable and whose applied effects are the same.

Proof. Let us consider a state s where an action lla_i from R-latplan is applicable. Let us note llp_i its preconditions and lle_i its effects.

In RC-latplan with a Clustering on the effects, the action lla_i is either used as it is (because it does not share the same effects of any other low-level action), in which case the property holds, or it is integrated into a cluster.

If lla_i belongs to a cluster c —noted lla_{ic} —and if it is applicable therefore, by properties 5.4.1 and 5.4.3, c_a is also applicable. And since the effects are the same between lla_i and c_a , the effects applied by c_a are also lle_i .

□

Property 5.4.5. For any state where an action from RC-latplan is applicable, there exists an action from R-latplan that is also applicable and which applied effects are the same.

Proof. Let us consider a state s where an action a_c from RC-latplan is applicable. If a_c is applicable then by properties 5.4.3 and 5.4.1 it means that at least one low-level action lla_{ic} of R-latplan is also applicable. If lla_{ic} is applicable, by definition, the effects applied by lla_{ic} are exactly the same as the effects applied by c .

□

Property 5.4.6. Any problem solved with R-latplan can be solved with RC-latplan.

Proof. Let us consider a plan solvable by R-latplan action model. Then for any state of the plan—apart from the goal—, there exists at least one applicable low-level action of R-latplan and by property 5.4.4 there exists also one applicable action of RC-latplan which applies the same effects.

□

Property 5.4.7. Any problem solved with RC-latplan can be solved with R-latplan.

Proof. Let us consider a plan solvable by RC-latplan action model. Then for any state of the plan—apart from the goal—, there exists at least one applicable clustered action of RC-latplan and by property 5.4.5 there exists also one applicable action of R-latplan which applies the same effects.

□

Property 5.4.8. RC-latplan action model is logically equivalent to R-latplan action model.

Proof. Since any problem solved with R-latplan can be solved with RC-latplan and vice versa, the two action models are logically equivalent.

□

	Sokoban	Hanoi	Blocksworld
R-latplan	4631	1468	577
RC-latplan with clustering on the effects	517	421	334

Table 5.5: Number of actions in R-latplan vs RC-latplan with clustering of the effects

	Speed	# States	Generated	Mem. Peak
R-latplan	0.012s	7385		13652 KB
RC-latplan with clustering on the effects	0.011s	5741		12829 KB

Table 5.6: Planning metrics: R-latplan vs RC-latplan

5.4.2 Results

Resulting PDDL domain

Table 5.5 shows the number of PDDL actions with the new clustering. We obtained a decrease of the number of actions compared to R-latplan by a factor of ~ 4.72 , which is slightly less than with the hierarchical clustering method (~ 5.41).

Planning performances

As for RC-latplan with hierarchical clustering we obtained the same results on the benchmark of problems, namely, the same plans as in R-latplan were generated. The semantic of the actions was also correct (*i.e.*, each clustered action used in the plans corresponded to the high-level action taken by the agent).

Regarding the speed, memory consumption and number of nodes tested during the search by Fast Downward, we obtained also similar results — with a marginal difference on all metrics —, summarized in table 5.6.

Reachability test

We tested the reachability of RC-latplan with the new clustering, the results shown in table 5.7 confirms the equivalence with the R-latplan representation.

	#pairs with a path in full dfa	coverage R-latplan	coverage RC-latplan
Hanoi	256	256	256
Blocks	15000	15000	15000
Sokoban	11130	11130	11130

Table 5.7: Comparing DFA coverage of R-latplan and RC-latplan with same effects clustering

Chapter 6

Visual Reward Machines

Visual Reward Machines [84] is a work that I co-authored during my PhD, in the context of the course *Non-Markov Decision Processes and Reinforcement Learning*. This work is closely related to my thesis, as it tackles the problem of learning a decision model—specifically, an automaton and a reinforcement learning policy—from sequences of images. While the representation differs (automaton vs. planning domain), the underlying challenge of inferring structured behavior models from raw visual data is central to both. The first part of this thesis focused on learning symbolic planning models—specifically, PDDL action schemas—from raw visual observations, enabling classical planning agents to operate in fully observable but perception-based environments. That work emphasized the challenge of grounding high-level, interpretable action models from unstructured inputs without supervision. In this chapter, we shift our attention to a complementary learning paradigm: Reinforcement Learning (RL). While classical planning requires a known symbolic model of the environment, RL agents learn to act through interaction, typically without an explicit model. However, as with planning, the use of raw perceptual data introduces significant complexity, especially in non-Markovian settings where decisions depend on the history of states. This motivates a neurosymbolic approach in RL as well—one that integrates structured symbolic reasoning (like automata or temporal logic) with deep learning techniques. In this context, we introduce Visual Reward Machines (VRMs), a novel framework that extends prior work on Reward Machines by removing the need for a predefined symbol grounding function. This approach shares the same core motivation as our planning work: learning meaningful, interpretable structure from high-dimensional visual data to support effective sequential decision-making.

6.1 Introduction

Non-Markovian Reinforcement Learning (RL) tasks [10] are extremely difficult to solve because intelligent agents must consider the entire history of state-action pairs to act rationally in the environment. However, the current line of research in this area involves bypassing non-Markovianity by augmenting the state space with a set of *features* that encode the environment history and solving the augmented-state MDP with known RL algorithms.

Therefore, the main challenge is how to construct these features. In the case of non-symbolic-state MDPs, the most popular approach is to combine RL algorithms with the use of Recurrent Neural Networks [39][35][52], which automatically extract features from data sequences.

For problems with a symbolic finite state space, most works use LTL [71] or LTLf [24] to specify the temporally-extended task. This specification is then compiled into an automaton, and the automaton state is combined with the environment state to make the decision process Markovian. An example of this approach is the so-called Reward Machines [19]. Reward machines (RM) can also be applied to non-symbolic-state environments for which a symbol grounding function is known [89]. The latter maps the environment’s raw state into a boolean interpretation over the symbols used by the specifications in LTL and makes the symbols observable. Therefore, summarizing RMs assumes prior knowledge of: (i) the symbol grounding function, (ii) the LTL or the Deterministic Finite Automaton (DFA) specification. Despite prior work that discards the assumption of knowing the temporal specification [30][92][73][29], no work assumes not knowing the symbol grounding function. We emphasize that learning this function for realistic environments with raw and/or high-dimensional states can be challenging and requires a large amount of labeled data.

In previous work by my colleagues [76], it was shown that LTLf specifications can be exploited in image sequence classification to accelerate the learning process, even when the symbols in the specification alphabet are not grounded in the images. In another related study [75], they introduced a recurrent neural network architecture inspired by Probabilistic Finite Automata (PFA) to perform DFA induction via gradient descent. In this work, we combine [86] and [85] to implement a neurosymbolic version of reward machines that we call *Visual Reward Machines* (VRM). The contribution of this paper is a novel formulation of reward machines that considers the symbol grounding function as a part of the specification. We show that we can relax the assumption of knowing the symbol grounding function and

that the LTLf specifications can be exploited to increase the performance of RL algorithms in visual non-Markovian tasks, even if the specification is not grounded in the environment states.

The remainder of this chapter is organized as follows: In Section 6.3, we review related works; in Section 6.2, we provide some preliminary knowledge on Reward Machines and Probabilistic Finite Automata; we define Visual Reward Machines and describe their implementation using neural networks in Section 6.4; we report preliminary experiments validating our method in Section 6.5; finally, we discuss final considerations and directions for future work in Section 6.6.

6.2 Background

Reward Machines Reward machines (RM) [19] are an automata-based representation of non-Markovian reward functions. RMs provide a normal-form representation for reward specification in a diversity of formal languages. Given a finite set of propositions P representing abstract properties or events observable in the environment, RMs specify temporally extended rewards over these propositions while exposing the compositional reward structure to the learning agent. Formally, a simple Reward Machine is a tuple $RM = (2^P, Q, q_0, \delta_t, \delta_r)$, where 2^P is the automaton alphabet, Q is the set of states, q_0 is the initial state, $\delta_t : Q \times 2^P \rightarrow Q$ is the transition function and $\delta_r : Q \times 2^P \rightarrow \mathbb{R}$ is the reward function.

Probabilistic Finite Automaton A Probabilistic Finite Automaton (PFA) A_p is a tuple $(P, Q, i_p,$

$\delta_{tp}, f_p)$, where P is the alphabet, Q is the finite set of states, $i_p : Q \rightarrow [0, 1]$ is the probability for a state to be an initial state, $\delta_{tp} : Q \times P \times Q \rightarrow [0, 1]$ is the transition probability function, and $f_p : Q \rightarrow [0, 1]$ is the probability of a state to be final. We have therefore $\sum_{q' \in Q} \delta_{tp}(q, p, q') = 1$, and $\sum_{q \in Q} i(q) = 1 \forall q \in Q, \forall a \in P$. We can also represent the PFA in matrix form as a *transition matrix* M_t , an *input vector* v_i and an *output vector* v_o . Matrix $M_t \in \mathbb{R}^{|P| \times |Q| \times |Q|}$ contains at index (p, q, q') the value of $\delta_{tp}(q, p, q')$. We denote as $M_t[p] \in \mathbb{R}^{|Q| \times |Q|}$ the 2D transition matrix for symbol p . The input vector $v_i \in \mathbb{R}^{1 \times |Q|}$ contains at index k the probability of state q_k to be an initial state. The output vector $v_o \in \mathbb{R}^{|Q| \times 1}$ has in position k the probability of state q_k to be accepting. Given a string

$x = x[0]x[1]...x[l - 1]$, we denote as $q_{p,0}, q_{p,1}...q_{p,l}$ the sequence of probabilities to visit a certain state, where $q_{p,t} \in \mathbb{R}^{1 \times |Q|}$ is a row vector containing at position k the probability to stay in state k at time t .

$$\begin{aligned} q_{p,0} &= v_i \\ q_{p,t} &= q_{p,t-1} \times M_t[x[t]] \quad \forall t > 0 \end{aligned} \tag{6.1}$$

The probability of being in a final state at time t is the inner product $q_{p,t} \times v_o$. Therefore the probability of a string to be accepted is the probability to be in a final state in the last computed state $q_{p,l}$, and it is calculated as follows

$$v_i \times M_t[x[0]] \times M_t[x[1]] \times ... \times M_t[x[l - 1]] \times v_o \tag{6.2}$$

6.3 Related work

LTL formulas are widely used in Reinforcement Learning (RL) to specify non-markovian tasks [15]. Some work assumes prior knowledge of the LTL task specification [7][16], which is compiled into an automaton and monitored during the task execution to produce non-markovian rewards for the task. Some other works focus on learning the task machine from traces of symbolic observations and rewards received by the environment, by using known methods for automata induction [9][10][11][12]. All these works, however, use symbolic data and do not consider the problem of discovering latent symbols in the data. For this reason, they are applicable only in symbolic-state environments or continuous problems for which a mapping between the continuous state and a symbolic interpretation is known, also called labeled MDP [8]. Many works assume to know an imperfect symbol grounding function for the task [17][18][19]. Namely, a function that sometimes makes mistakes in predicting symbols from states or predicts a set of probabilistic beliefs over the symbol set instead of a boolean interpretation. These works can be considered a preliminary step towards integration with non-symbolic domains. However, they do not assess the problem of learning the symbol grounding function, but only how to use a pre-trained imperfect symbol grounder. One work [20] uses the LTLf specification of the task without assuming any knowledge of the symbol grounding function. This work employs a neural network architecture that is shaped as the LTL formula to learn a representation of states, that can be easily transferred to different LTL tasks in the same environment. The capability to

transfer the representation to new tasks suggests that the representation can capture the semantics of symbols in some ways. However, the representation does not exactly ground the symbols in the environment observations.

6.4 Method

6.4.1 Visual Reward Machine definition

We define a Visual Reward Machine as a tuple $VRM = (S, P, Q, R, q_0, \delta_{tp}, \delta_{rp}, sg)$, where S is the set of input data, possibly infinite and continuous, the machine can process; P is a finite set of symbols; Q is a finite set of states, R is a finite set of rewards; q_0 is the initial state; $\delta_{tp} : Q \times P \times Q \rightarrow [0, 1]$ is the transition probability function; $\delta_{rp} : Q \times R \rightarrow [0, 1]$ is the reward probability function and $sg : S \times P \rightarrow [0, 1]$ is the *symbol grounding* probability function.

Similarly to a RM, a VRM produces a reward sequence by processing a data sequence. Unlike RM, however, the input data does not need to be symbolic but can be of any type. The symbol grounding function maintains the link with a symbolic representation, assigning to a data instance $s \in S$ a probability value for each symbol in the alphabet P . In our setting, the Reward Machine is a probabilistic Moore Machine taking as input *probabilistic* symbols. Let us assume that all the information in the definition is known. We represent the Moore machine in matrix representation, as described in Section 6.2, as an input vector v_i encoding the initial state, a transition matrix M_t encoding the transition function, and an output matrix M_o representing the reward function. Given an input sequence of data $s_1, s_2, \dots, s_l$, the VRM outputs a sequence of l probability vectors over the machine states $h_1, h_2, \dots, h_l$ and a sequence of l probability vectors over the machine outputs (which correspond to reward values) $r_1, r_2, \dots, r_l$ as follows.

$$\begin{aligned}
 h_0 &= v_i \\
 x_t &= sg(s_i) \\
 h_t &= \sum_{p=0}^{p=|P|} x_t[p](h_{t-1} \times M_T[p]) \\
 r_t &= h_t \times M_O
 \end{aligned} \tag{6.3}$$

Where we denote with $V[i]$ (or $v[i]$), the component i of matrix (vector) V (v).

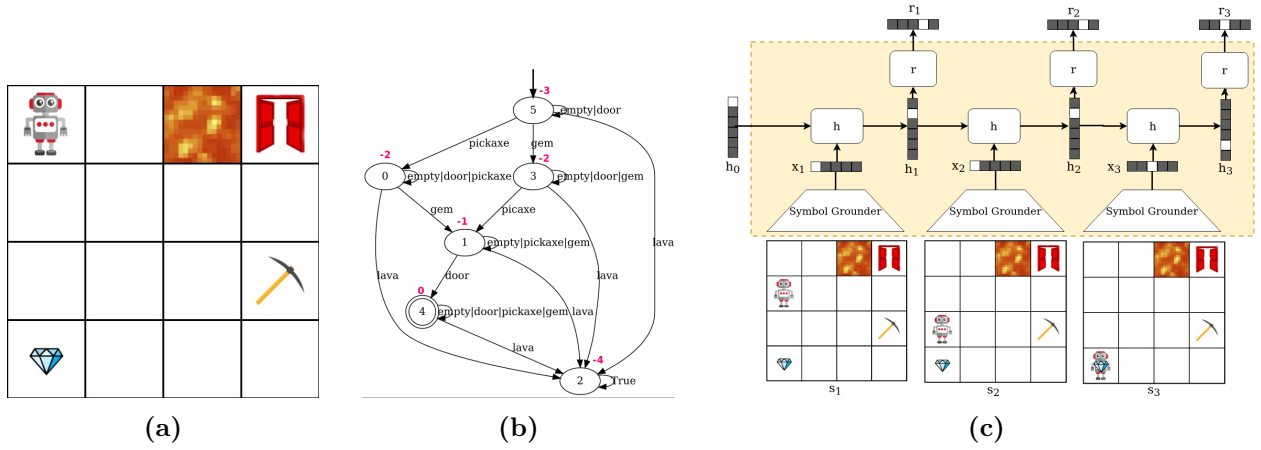


Figure 6.1: a) Visual Minecraft environment. b) Non-Markovian task specification for the Minecraft agent: the agent has to collect the pickaxe and the gem (in any order) and then go to the door, while always avoiding the lava. c) Implementation of a Visual Reward Machine with convolutional and recurrent neural modules.

6.4.2 VRM implementation with neural networks

This section describes a neural network architecture implementing Visual Reward Machines.

We define the VRM implementation as a neural network composed of two parts: a convolutional module and a recurrent module. The convolutional module implements the *symbol grounding* function. This module handles the perception of symbols from non-symbolic states. It is implemented as a softmax-activated Convolutional Neural Network (CNN), having the output layer size equal to the number of symbols. The recurrent module corresponds to a *parametric probabilistic Moore Machine*. This module implements the reward machine with a Recurrent Neural Network (RNN) of a specific structure that allows extracting from the network parameters an equivalent Moore Machine. This network is based on an extension of [85].

In the VRM definition, we use a probabilistic relaxation of both the symbol grounding function and the Reward Machine. These probabilistic relaxations are fundamental for implementing the whole system with neural networks (NN). NNs, in fact, are intrinsically continuous since they learn through a gradient-based optimization process, and they can struggle in learning a ‘crispy’ symbolic logical model. In our case, the logical model we want to induce from data is the Moore Machine (MM), representing the non-markovian reward. The intuition behind our work is that Probabilistic Moore Machines (PMM) are closely related to Recurrent Neural Networks, since they calculate the next state and output using multiplications between continuous vectors and matrices, in the same way as RNNs do. However, (boolean) machines can also be represented in matrix form, considering their

matrix representation comprises only one-hot row vectors. Following this idea, we define the recurrent module as a parametric PMM in which we can drive the representation to be close to one-hot during training, allowing logical induction through backpropagation. We obtain this effect using an activation function that smoothly approximates discrete output, as in prior work [48] [88] [54]. In particular, we use a modified version of the classical softmax activation functions: $\text{softmax}_\tau(x, \tau) = \text{softmax}(x/\tau)$; where $0 < \tau \leq 1$ is a temperature value controlling the activation steepness: when τ is close to 0, the τ -softmax returns outputs close to one-hot vectors; when τ is 1, the function is the softmax.

Following the VRM implementation with parametric models.

$$\begin{aligned}
h_0 &= v_i \\
x_t &= \text{CNN}(s_t; \theta_{sg}) \\
h_t &= \sum_{p=0}^{p=|P|} x_t[p](h_{t-1} \times \text{softmax}_\tau(\theta_{M_T}, \tau)[p]) \\
r_t &= h_t \times \text{softmax}_\tau(\theta_{M_O}, \tau)
\end{aligned} \tag{6.4}$$

where θ_{sg} , θ_{M_T} and θ_{M_O} are the model trainable parameters. θ_{sg} are the symbol grounder parameters, while θ_{M_T} and θ_{M_O} are the parameters of the recurrent module. In particular, θ_{M_T} and θ_{M_O} are two matrices having the same dimensions of matrices M_T and M_O (namely $\theta_{M_T} \in \mathbb{R}^{|P| \times |Q| \times |Q|}$ and $\theta_{M_O} \in \mathbb{R}^{|Q| \times |R|}$) and the softmax_τ activation is applied to the last matrix dimension. A scheme of the proposed implementation is shown in Figure 6.1(c).

6.5 Preliminary experiments

In this section, we report some preliminary experiments that validate the proposed framework. The implementation code can be found at <https://github.com/whitemech>. In particular, we explore several learning procedures. In the first experiment, we test *offline symbol grounding*, which refers to the capability to learn the *symbol grounding function* by leveraging the Moore Machine structure and sequences of states-rewards of a hand-crafted dataset. For this test, we initialize the Moore Machine parameters θ_{M_T} and θ_{M_O} with the given task specification and train only the CNN weights by minimizing the cross-entropy between the network predictions and the reward labels.

In the second experiment, we test a combination of *RL and online symbol grounding*. In this test, we ground the symbols of the task specifics *on the fly*, meaning we use sequences

of image states and rewards collected by the agent while learning to perform the task. We use the learned symbol grounding to proceed on the automaton at each step and estimate the current automaton state. The latter is combined with the environment visual state to form a *Markovian* state representation, which is used by the RL algorithm to estimate the optimal policy.

While this paper mainly focuses on learning the grounding function, the framework is very versatile, and many other learning-reasoning combinations are possible. For instance, we test learning the DFA structure from traces of *imperfectly grounded symbols*, which are symbols for which we have only a probabilistic prediction instead of a strictly boolean interpretation. Although many techniques for DFA induction exist in the literature [5][45][56], and few extensions can handle noise in the output label [62] [83], to the best of our knowledge, there are no extensions to handle noise in the *input symbols*. In our experiments, we simulate the use of an imperfect pretrained classifier for symbol grounding by corrupting the input symbols with Gaussian noise, and we train the recurrent part of the VRM on this noisy dataset. We conduct our preliminary experiments in this direction on Tomita Languages, which are a popular benchmark for DFA induction. We leave the integration of DFA induction with RL in non-Markovian environments for future research.

6.5.1 Visual Minecraft Environment

In this section, we introduce the Visual Minecraft environment that we used to conduct our preliminary experiments, as shown in Figure 6.1(a). This environment is an example of a non-Markovian task with non-symbolic states, similar to the one considered by [20] and [58], but with image states.

A robotic agent can navigate through a grid world environment using four movement actions: *left*, *right*, *up*, *down*. Each cell of the grid can either be empty or contain one of the following items: *pickaxe*, *gem*, *lava*, or *door*. The task requires the agent to collect a pickaxe and a gem (in any order), and then proceed to the door, while always avoiding the lava. The task specification is represented by the deterministic finite automaton (DFA) shown in Figure 6.1(b), which is expressed in terms of five symbols, $P = \text{pickaxe}, \text{gem}, \text{lava}, \text{door}, \text{empty}$. Each symbol indicates whether the corresponding item is present in the cell occupied by the agent.

To express the non-markovian reward function, we transformed the DFA into a Moore

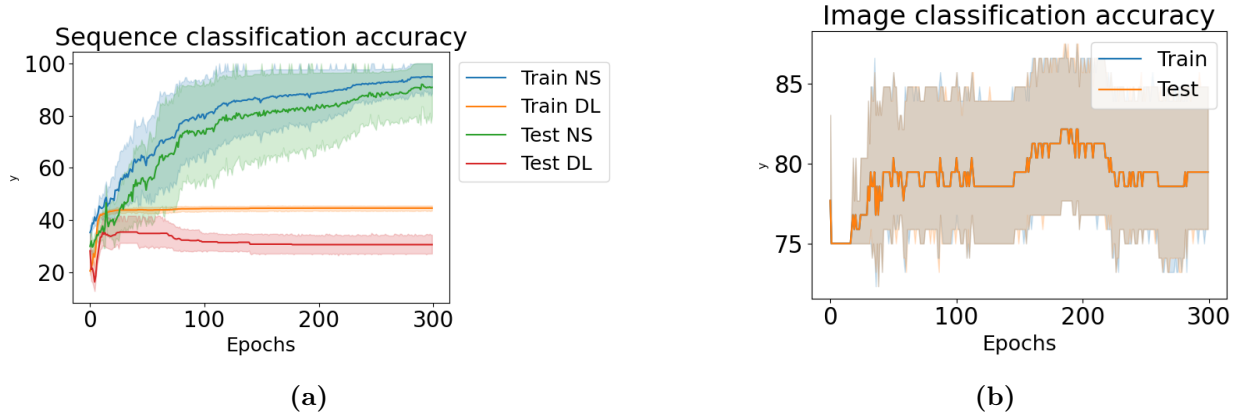


Figure 6.2: Results obtained by exploiting the reward machine structure to learn the symbol grounding function offline for the Visual Minecraft environment. a) Visual Reward Machine accuracy over sequences. b) Symbol grounding accuracy over single images.

machine by assigning a reward function to its states. The reward function is maximum for the final states and decreases as the distance from the final state increases. The values of the reward function are indicated in red on the automaton states in Figure 6.1(b).

The environment state is an image similar to the one depicted in Figure 6.1(a), and it does not provide any information about the items collected by the agent up to that point in time, such as the gem or pickaxe. This makes the environment non-Markovian because it is impossible to obtain a comprehensive inventory of all the elements collected by the agent from the current state alone. Previous actions and observations must be considered. Furthermore, an additional challenge is introduced by the fact that the specifics of the task are not directly usable since the symbols are not observable. We do not know how to recognize them in the image.

6.5.2 Offline symbol grounding

Dataset To apply offline symbol grounding to the Minecraft environment, we created a dataset simulating 40 episodes in the environment and collecting the images rendered by the environment. Each episode lasts for 30 steps, producing a sequence of 30 images that correspond to the environment states and a sequence of 30 reward values that are used to label the image sequence *at each step*. Episodes are balanced between positive and negative examples, with 20 episodes where the agent correctly collects all the items and goes to the door while avoiding the lava, and 20 episodes where it fails. We split the dataset into 80% for training and 20% for testing.

Results Figure 6.2 shows the train and test sequence classification accuracy in Figure (a) and the symbol grounding accuracy on single images in Figure (b). Let us note that the task specification has two ungroundable symbols: *gem* and *pickaxe*. Since the agent can collect these two items in any order, the framework does not receive enough supervision to distinguish between them, resulting in the image classification not achieving 100% accuracy. However, sequence classification can still achieve top accuracy even if the symbol grounder confuses the gem for the pickaxe and vice versa. We compared our approach with a purely deep-learning-based approach that learns to classify sequences end-to-end using a CNN and an LSTM, which performed poorly in the task, obtaining only 40% sequence accuracy. Investigating the reason for these poor performances, we found that it almost never predicts rewards of -1 and -2, corresponding to the scarcest reward labels in the dataset. In fact, even if we balanced the dataset between positive (reward = 0 in the last step) and negative (reward < 0 in the last step) episodes, the reward labels are not balanced within the episodes. Learning classification tasks from highly biased data with neural networks can be very challenging, as shown in this experiment. However, our approach is unaffected by the label imbalance. Additionally, we note that the environment highly biases the distribution of symbols and reward labels in sequences. For example, the ‘empty’ symbol is much more frequent than the others (because the agent cannot jump from one item to the other, but it has to walk and observe many empty cells in between). Additionally, most possible symbolic traces are unfeasible in the environment and, therefore, never observed. This can complicate the image classification task in case it would be approached with supervised learning.

6.5.3 Reinforcement Learning and online grounding

In this second experiment, we perform online symbol grounding and use it to estimate the machine state, which we exploit to speed up policy learning. Unlike in the previous experiments, the dataset is not required to be balanced, as it is collected by the agent while exploring the environment. We construct a Markov state by concatenating features extracted by a CNN from the image with the estimated automaton state. We use Advantage Actor-Critic (A2C) [66] to learn a policy on this state representation. We compare this approach, which exploits the ungrounded DFA specification, with A2C using the state of an LSTM trained end-to-end as the state representation.

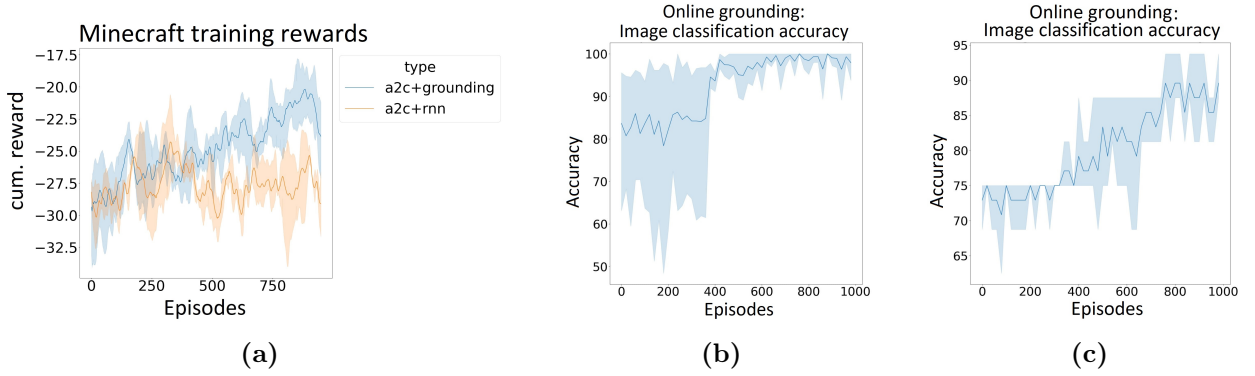


Figure 6.3: Results obtained by exploiting the reward machine structure to learn the symbol grounding function for the Visual Minecraft environment. a) Training rewards. b) Reward prediction accuracy during training. c) Symbol grounding accuracy during training.

Training settings We ran three experiments with three random seeds for each approach and report the mean and standard deviation. We allowed each approach to train for 1000 episodes before stopping the training. We used a learning rate of 0.0007 and an entropy coefficient of 0.0001 as hyperparameters for A2C.

The symbol grounder was trained in the same way described in the previous section, with the newly acquired data, every 40 episodes. The baseline method used a one-layer LSTM of hidden size 256. We tested different hidden sizes for the LSTM (50 and 256) and two layers instead of one, and found that the one-layer LSTM with a hidden size of 256 was the best configuration.

Results Figure 6.3 shows the results obtained in the experiment. Figure 6.3(a) shows the training rewards for our method (A2C+grounding) and the baseline (A2C+RNN). The figure shows that our method outperforms the baseline. Figure 6.3(b) and (c) show the sequence classification and the symbol grounding classification accuracy of the Visual Reward Machine obtained during the training of the RL agent. The sequence classification accuracy tends to be quite high from the beginning of training, while the symbol grounding classification accuracy achieves top results only after 800 episodes. This is reasonable since the VRM needs to observe some positive episodes to correctly ground all the symbols (e.g., the symbol *door* can be grounded only from a trajectory completing the task), and positive episodes are observable only when the RL module has learned a good policy. Training rewards increase coherently with the improvement in the VRM’s performance. Despite these being only preliminary experiments, the results are encouraging. In particular, they confirm our intuition that symbolic temporal specifications can also be exploited in visual tasks for which

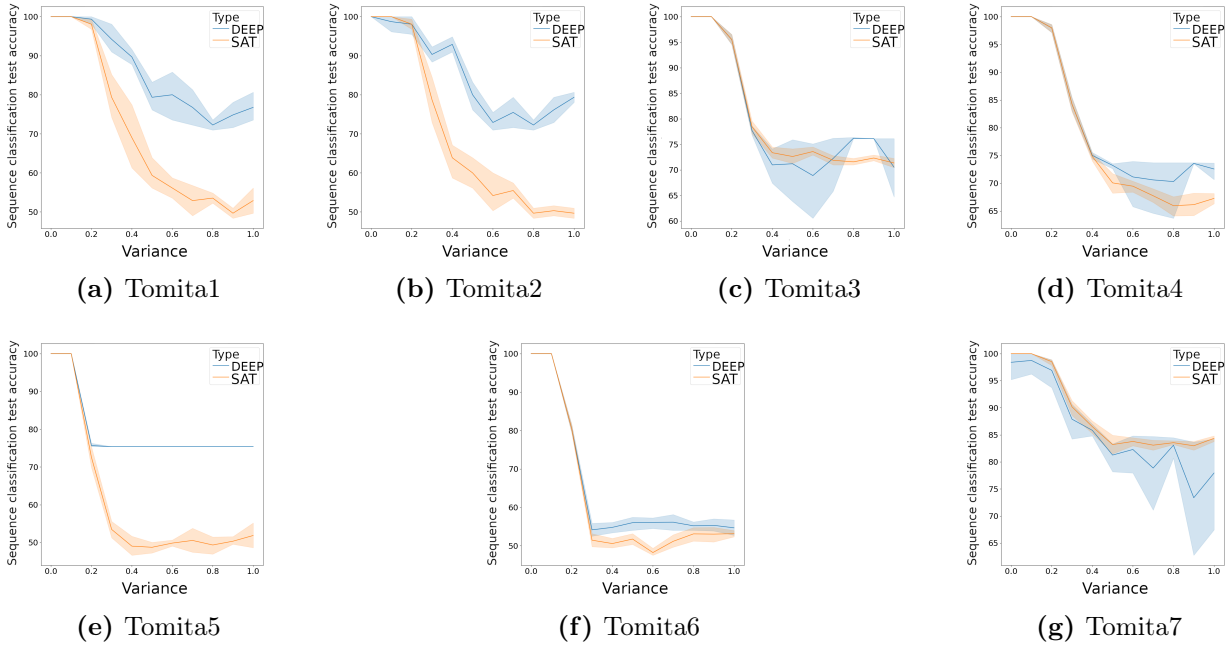


Figure 6.4: Learning DFA from traces composed of imperfectly grounded symbols: results on the 7 Tomita languages.

we do not know the symbol grounding function.

6.5.4 Learning the machine from imperfectly grounded symbols

In this experiment, we test the capability of our framework to learn DFA specifications from sequences of imperfectly grounded symbols. Specifically, we only trained the recurrent module with traces of symbols and accepted-rejected labels generated by the Tomita languages [82]. We corrupted the one-hot representation of symbols in the training traces by adding Gaussian noise with a mean of zero and variable variance. Figure 6.4 compares our approach (Deep) with the SAT-based DFA-inductor (SAT) [95]. Since boolean logic induction methods like DFA-inductor cannot handle probabilistic truth values, we discretize the corrupted inputs to the nearest one-hot vector before passing them to DFA-inductor. The figures show how the two methods' test accuracy (on the y-axis) is affected by different values of noise variance (on the x-axis). In particular, we observe that our method's test accuracy degrades less with increasing noise variance, making it more robust to noise in the symbol grounding compared to DFA-inductor.

6.6 Conclusion

In this chapter, we introduced Visual Reward Machines (VRMs), a neurosymbolic framework designed to address non-Markovian reinforcement learning tasks using raw perceptual inputs. Unlike traditional approaches that require a predefined symbol grounding function and a known temporal specification, VRMs jointly learn both components from data, enabling effective decision-making in visually complex environments where history must be considered.

This work complements the earlier part of the thesis, where we addressed the challenge of learning PDDL action models for classical planning directly from images. Both efforts share a common goal: to bridge the gap between low-level perception and high-level symbolic reasoning, allowing agents to autonomously acquire interpretable, structured models from unstructured sensory input. While the planning work focused on enabling deliberative reasoning by extracting deterministic transition models, the VRM framework targets sequential decision-making in settings where behavior depends on temporal patterns in the agent’s experience.

An exciting direction for future work lies in merging these two paradigms. One promising avenue is to extend the VRM framework toward the induction of planning-compatible action models—where learned automata and reward structures are transformed into symbolic operators with explicit preconditions and effects. Conversely, techniques developed in the planning setting, such as invariant learning and action clustering, could enhance the structure and interpretability of the automata learned by VRMs.

Chapter 7

Conclusions and Future Works

The PDDL language, along with its various extensions, remains a cornerstone for enabling autonomous agents to plan in a structured, interpretable, and explainable manner. However, the manual construction of such representations poses a significant bottleneck in the development of intelligent agents. While numerous approaches have sought to automate the learning of action models in unsupervised or semi-supervised settings, they often remain domain-specific and still require the manual specification of predicates and variables — limiting their scalability and generality.

Latplan marked a notable advancement as the first end-to-end machine learning model capable of inducing PDDL representations directly from raw data by leveraging the latent space of neural networks. Nevertheless, it suffers from two key limitations: (1) the generation of hallucinated plans (section 2.5.2) that are not executable in the real environment, and (2) the learned actions lack any grounding in human-understandable semantics (section 2.5.3). In our work we firstly aimed at improving the state-of-the-art in the field of learning PDDL representation, automatically, from images, and we started by trying to improve Latplan.

Our first approach (3.1) was to induce invariants in Latplan training. Invariants are logical constraints that exists among the variables of the domain and can be automatically retrieved with different tools. Our idea was to firstly train Latplan in order to obtain the invariants and retrain it again enforcing the invariants in the loss function using fuzzy logic. This approach, while interesting showed positive results only on simple domains (ie 8-MNIST puzzle), where slightly more visually correct and optimal plans were produced. But the issue of uninterpretability of the actions remained.

The second approach (3.2) consisted of improving Latplan through the modification of

its hyper parameters, either automatically (grid search) or manually. For the grid search we considered three hyperparameters: the size of the latent space, the weight on the next state reconstruction loss term and the weight on the prior (Bernoulli’s law) of the VAE latent state. We showed that a higher weight on the next state reconstruction was positively correlated to the obtention of more solutions and more invariants. For the manual modification of the hyper-parameters we considered, in two different experiments, fixing the size of the latent space and the number of actions to be learned, based on domain-specific consideration, *e.g.*, the size of the latent space was fixed in order to limit the number of possible states to be learn to the number of possible states in the domain considered. Limiting the latent space size (*i.e.*, reducing it, compared to the original setting **Latplan**), led to a better reconstruction of the images by the decoder of the VAE, under noisy input. Limiting the number of actions (*i.e.*, reducing it drastically, from around 6000 to 22 in Hanoi) actually worsened the performances of the system.

In a third approach (4), considering that these architecture-agnostic methods led to shallow results, we decided to modify **Latplan** and proposed **R-latplan**. In **R-latplan** we replaced the ACTION network of **Latplan**, that learns the action’s label through back-propagation, with a deterministic function, called the *transition identifier*, that labels each unique transition of the domain’s Visual DFA, by using pixel difference between the two images of the transition. This modification led **R-latplan** to substantially outperform **Latplan** since we could obtain optimal and visually correct plans for the more challenging problems, in each domain. Furthermore we showed that **R-latplan** was resilient to noisy data, more specifically to image noise and to noise in the action’s labels, *i.e.*, when the transition identifier would output wrong labels. More specifically, in these “noisy” scenarios, **R-latplan** can still generate correct plans and learn the correct action model for a transition even when the label outputted by the transition identifier is wrong. One limitation of our work is the number of low-level PDDL actions which grows with the size of the state space, in turn increasing the processing time of FD.

This is why, in a fourth work (5), we tested different methods to compress the action model of **R-latplan** in order to solve the scalability issue due to action proliferation. In this work we first made an attempt at learning one PDDL action per grounded high-level action, using Decision Tree Learning (5.2), but this turned out inefficient since the plans generated were not visually correct for long problems. We then experimented a bottom-up approach, called **RC-latplan**, where we used the **R-latplan** actions and clustered them using two different

methods: a hierarchical clustering algorithm based on the distance between actions binary representations (5.3) and a clustering on the effects (5.4). For each cluster we produced a PDDL action where the precondition clause corresponds to the disjunction of corresponding low-level preconditions and the effect clause to a conjunction of common effects (among the low-level actions) and — in the case of the hierarchical clustering — of conditional effects. The hierarchical clustering method led to improved performances in terms of search speed, memory consumption, and number of generated nodes during the search. We also found that all the plans identified in the benchmarks of **R-latplan** were recovered using the clustered representation. However, a more detailed analysis revealed that the overall coverage across all problem instances within each domain was lower in **RC-latplan** compared to **R-latplan**. We attributed this outcome to the manner in which clustered effects are defined, demonstrating that for any given clustered action, all effects present in the corresponding applicable low-level actions are simultaneously triggered — thereby restricting the space of possible effects. In order to bypass this issue, we decided, in a last work (5.4), to only cluster low-level actions that have exactly the same set of effects. We proved, theoretically that this clustering led to a representation that is logically equivalent to the **R-latplan** representation and we confirmed this equivalence experimentally by showing that the coverage over the problems in all domain was equal between the two representations.

Our findings suggest that a lot of research remains to be done with regards to **Latplan** and the improved versions we proposed. In particular, other architectural modifications could lead to obtain, without post-processing, a PDDL action for each grounded high-level action. Regarding the work on **RC-latplan**, since redundancies of the effects and preconditions exist within each cluster, a research toward lifting the model could be interesting (some recent attempts in this line of work already exist [77][16]), for example, there might be a relation between the variables derived from the Fast Downward preprocessor and the sets of effects in the clusters, such that a high-level action could, potentially, accept such variables as a parameters. In general, learning a subsymbolic PDDL representation from images, without supervision, should be applied to more complex types of planning (hierarchical planning, temporal planning etc.).

In a subsidiary work (6) that I co-authored in the context of my PhD studies, I also explored RL-based planning, with the goal of improving policy learning in visually rich, sparse-reward environments. We introduced Visual Reward Machines (VRMs), a method that leverages symbolic representations of reward structures learned from visual traces of

task executions. Specifically, we learn a visual representation of a Deterministic Finite Automaton (DFA) that encodes the structure of valid task progressions, and we integrate this representation into the agent’s state input by concatenating the DFA state vector to the visual observation. This enriched input space helps guide the RL agent through long-horizon tasks by providing structured, high-level signals. While this contribution lies outside the core scope of action model learning in PDDL, it reflects a shared objective: improving autonomous decision-making by extracting and exploiting symbolic structure from unstructured sensory data. Future work regarding the VRM framework could consist of using planning-based knowledge in the definition of the RL agent input state; indeed, recent works [34] [55] learn to extract, automatically and without supervision, a PDDL-based representation of states and actions, from traces.

Ultimately, unifying model learning for both planning and reinforcement learning could lead to agents that reason, plan, and act seamlessly in rich, partially understood visual environments.

Bibliography

- [1] C. Aeronautiques, A. Howe, C. Knoblock, I. D. McDermott, A. Ram, M. Veloso, D. Weld, D. W. Sri, A. Barrett, D. Christianson, et al. Pddl: the planning domain definition language. *Technical Report, Tech. Rep.*, 1998.
- [2] A. Ahmetoglu, M. Y. Seker, J. Piater, E. Oztop, and E. Ugur. Deepsym: Deep symbol generation and rule learning for planning from unsupervised robot interaction. *Journal of Artificial Intelligence Research*, 75, 2022.
- [3] V. Alcázar and A. Torralba. A reminder about the importance of computing and exploiting invariants in planning. In *Proceedings of the International Conference on Automated Planning and Scheduling*, volume 25, pages 2–6, 2015.
- [4] M. Anderson and T. Feil. Turning lights out with linear algebra. *Mathematics Magazine*, 71(4):300–303, 1998.
- [5] D. Angluin. Learning regular sets from queries and counterexamples. *Information and Computation*, 75(2):87–106, 1987.
- [6] M. Asai. Photo-realistic blocksworld dataset. *arXiv preprint arXiv:1812.01818*, 2018.
- [7] M. Asai and A. Fukunaga. Classical planning in deep latent space: Bridging the subsymbolic-symbolic boundary, 2017.
- [8] M. Asai and A. Fukunaga. Classical planning in deep latent space: Bridging the subsymbolic-symbolic boundary. In *Proceedings of the aaai conference on artificial intelligence*, volume 32, 2018.
- [9] M. Asai, H. Kajino, A. Fukunaga, and C. Muise. Classical planning in deep latent space. *Journal of Artificial Intelligence Research*, 74, 2022.

- [10] F. Bacchus, C. Boutilier, and A. Grove. Rewarding behaviors. pages 1160–1167, Portland, OR, 1996.
- [11] S. Badreddine, A. d’Avila Garcez, L. Serafini, and M. Spranger. Logic tensor networks. *Artificial Intelligence*, 303:103649, 2022.
- [12] A. Barbin, F. Cerutti, and A. E. Gerevini. Addressing the symbol grounding problem with constraints in neuro-symbolic planning. 2022.
- [13] A. Barbin, F. Cerutti, and A. E. Gerevini. Learning reliable pddl models for classical planning from visual data. In *36th IEEE International Conference on Tools with Artificial Intelligence (ICTAI)*, 2024.
- [14] A. Biere, M. Heule, H. van Maaren, and T. Walsh, editors. *Handbook of Satisfiability*, volume 185 of *Frontiers in Artificial Intelligence and Applications*. IOS Press, 2009.
- [15] A. Bieszczad and S. Kuchar. Neurosolver learning to solve towers of hanoi puzzles. In *2015 7th International Joint Conference on Computational Intelligence (IJCCI)*, volume 3, pages 28–38. IEEE, 2015.
- [16] B. Bonet and H. Geffner. Learning first-order symbolic representations for planning from the structure of the state space, 2020.
- [17] C. Boutilier, T. Dean, and S. Hanks. Decision-theoretic planning: Structural assumptions and computational leverage. *Journal of Artificial Intelligence Research*, 11:1–94, 1999.
- [18] L. Breiman, J. H. Friedman, R. A. Olshen, and C. J. Stone. *Classification and Regression Trees*. Wadsworth International Group, 1984.
- [19] A. Camacho, R. Toro Icarte, T. Q. Klassen, R. Valenzano, and S. A. McIlraith. Ltl and beyond: Formal languages for reward function specification in reinforcement learning. In *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence, IJCAI-19*, pages 6065–6073. International Joint Conferences on Artificial Intelligence Organization, 7 2019.
- [20] J. Corazza, I. Gavran, and D. Neider. Reinforcement learning with stochastic reward machines. *Proceedings of the AAAI Conference on Artificial Intelligence*, 36(6):6429–6436, Jun. 2022.

- [21] J. Culberson. Sokoban is pspace-complete. 1997.
- [22] J. Cullen and A. Bryman. The knowledge acquisition bottleneck: Time for reassessment? *Expert Systems*, 5:216–225, 1988.
- [23] P. Dagum and M. Luby. Approximating probabilistic inference in bayesian belief networks is np-hard. *Artificial intelligence*, 60, 1993.
- [24] G. De Giacomo and M. Y. Vardi. Linear temporal logic and linear dynamic logic on finite traces. In *Proceedings of the Twenty-Third International Joint Conference on Artificial Intelligence*, IJCAI '13, page 854–860. AAAI Press, 2013.
- [25] E. W. Dijkstra. A note on two problems in connexion with graphs. *Numerische mathematik*, 1, 1959.
- [26] K. Erol, J. Hendler, and D. S. Nau. Htn planning: Complexity and expressivity. In *Proceedings of the Twelfth National Conference on Artificial Intelligence (AAAI)*, pages 1123–1128, 1994.
- [27] R. E. Fikes and N. J. Nilsson. Strips: A new approach to the application of theorem proving to problem solving. *Artificial Intelligence*, 2(3):189 – 208, 1971.
- [28] M. P. Fourman. Propositional planning. In *Proceedings of AIPS-00 Workshop on Model-Theoretic Approaches to Planning*, pages 10–17, 2000.
- [29] D. Furelos-Blanco, M. Law, A. Jonsson, K. Broda, and A. Russo. Induction and exploitation of subgoal automata for reinforcement learning. *J. Artif. Int. Res.*, 70:1031–1116, may 2021.
- [30] M. Gaon and R. Brafman. Reinforcement learning with non-markovian rewards. *Proceedings of the AAAI Conference on Artificial Intelligence*, 34(04):3980–3987, Apr. 2020.
- [31] H. Geffner and B. Bonet. *Classical Planning: Full Information and Deterministic Actions*, chapter 2, pages 15–36. Springer International Publishing, Cham, 2013.
- [32] A. Gerevini and L. K. Schubert. Discovering state constraints in discoplan: Some new results. In *AAAI/IAAI*, pages 761–767, 2000.
- [33] M. Ghallab, D. Nau, and P. Traverso. *Automated Planning: Theory and Practice*. Morgan Kaufmann, 2004.

- [34] M. Grand, D. Pellier, and H. Fiorino. An accurate pddl domain learning algorithm from partial and noisy observations. In *2022 IEEE 34th International Conference on Tools with Artificial Intelligence (ICTAI)*, pages 734–738, 2022.
- [35] D. Ha and J. Schmidhuber. Recurrent world models facilitate policy evolution. In S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 31. Curran Associates, Inc., 2018.
- [36] A. Hagberg, P. Swart, and D. Chult. Exploring network structure, dynamics, and function using networkx. In *Proceedings of the 7th Python in Science Conference*, 2008.
- [37] P. Hájek. *Metamathematics of fuzzy logic*, volume 4. Springer Science & Business Media, 2013.
- [38] H. Harman and P. Simoens. Generating symbolic action definitions from pairs of images: Applied to solving towers of hanoi. AAAI Press, 2020.
- [39] N. Heess, J. J. Hunt, T. P. Lillicrap, and D. Silver. Memory-based control with recurrent neural networks. *CoRR*, abs/1512.04455, 2015.
- [40] M. Helmert. The fast downward planning system. *Journal of Artificial Intelligence Research*, 26:191–246, 2006.
- [41] M. Helmert. Concise finite-domain representations for pddl planning tasks. *Artificial Intelligence*, 173(5-6):503–535, 2009.
- [42] M. Helmert and C. Domshlak. Landmarks, critical paths and abstractions: what’s the difference anyway? In *Proceedings of the International Conference on Automated Planning and Scheduling*, volume 19, pages 162–169, 2009.
- [43] M. Helmert, P. Haslum, J. Hoffmann, and R. Nissim. Merge-and-shrink abstraction: A method for generating lower bounds in factored state spaces. *Journal of the ACM (JACM)*, 61(3):1–63, 2014.
- [44] J. A. Hendler, A. Tate, and M. Drummond. Ai planning: Systems and techniques. *AI magazine*, 11(2):61–61, 1990.

-
- [45] M. J. H. Heule and S. Verwer. Exact dfa identification using sat solvers. In *Proceedings of the 10th International Colloquium Conference on Grammatical Inference: Theoretical Results and Applications*, ICGI'10, page 66–79, Berlin, Heidelberg, 2010. Springer-Verlag.
- [46] R. T. Icarte, T. Q. Klassen, R. Valenzano, and S. A. McIlraith. Reward machines: Exploiting reward function structure in reinforcement learning. *Journal of Artificial Intelligence Research*, 73:173–208, 2022.
- [47] E. Jang, S. Gu, and B. Poole. Categorical reparameterization with gumbel-softmax. *arXiv preprint arXiv:1611.01144*, 2016.
- [48] E. Jang, S. Gu, and B. Poole. Categorical reparameterization with gumbel-softmax. In *5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24-26, 2017, Conference Track Proceedings*. OpenReview.net, 2017.
- [49] N. L. Johnson, S. Kotz, and N. Balakrishnan. *Continuous univariate distributions, volume 2*, volume 2. John wiley & sons, 1995.
- [50] J. M. Joyce. *Kullback-Leibler Divergence*. 2011.
- [51] A. Junghanns and J. Schaeffer. Sokoban: Enhancing general single-agent search methods using domain knowledge. *Artificial Intelligence*, 129(1-2):219–251, 2001.
- [52] S. Kapturowski, G. Ostrovski, W. Dabney, J. Quan, and R. Munos. Recurrent experience replay in distributed reinforcement learning. In *International Conference on Learning Representations*, 2019.
- [53] D. P. Kingma, M. Welling, et al. An introduction to variational autoencoders. *Foundations and Trends® in Machine Learning*, 2019.
- [54] R. Kusters, Y. Kim, M. Collery, C. de Sainte Marie, and S. Gupta. Differentiable rule induction with learned relational features. In A. d’Avila Garcez and E. Jiménez-Ruiz, editors, *Proceedings of the 16th International Workshop on Neural-Symbolic Learning and Reasoning as part of the 2nd International Joint Conference on Learning & Reasoning (IJCLR 2022)*, Cumberland Lodge, Windsor Great Park, UK, September 28-30, 2022, volume 3212 of *CEUR Workshop Proceedings*, pages 30–44. CEUR-WS.org, 2022.

- [55] L. Lamanna and L. Serafini. Action model learning from noisy traces: a probabilistic approach. In *34th International Conference on Automated Planning and Scheduling*, 2024.
- [56] K. J. Lang, B. A. Pearlmutter, and R. Price. Results of the abbadingo one dfa learning competition and a new evidence-driven state merging algorithm. In *ICGI 1998: Grammatical Inference*, number 1433 in Lecture Notes in Computer Science book series (LNCS), pages 1–12. Springer, 1998. Cite as: Lang K.J., Pearlmutter B.A., Price R.A. (1998) Results of the Abbadingo one DFA learning competition and a new evidence-driven state merging algorithm. In: Honavar V., Slutzki G. (eds) Grammatical Inference. ICGI 1998. Lecture Notes in Computer Science, vol 1433. Springer, Berlin, Heidelberg.
- [57] Y. LeCun, B. Boser, J. S. Denker, D. Henderson, R. E. Howard, W. Hubbard, and L. D. Jackel. Backpropagation applied to handwritten zip code recognition. *Neural computation*, 1(4):541–551, 1989.
- [58] A. C. Li, Z. Chen, P. Vaezipoor, T. Q. Klassen, R. T. Icarte, and S. A. McIlraith. Noisy symbolic abstractions for deep RL: A case study with reward machines. *CoRR*, abs/2211.10902, 2022.
- [59] A. O. Liberman, B. Bonet, and H. Geffner. Learning first-order symbolic planning representations that are grounded. *arXiv preprint arXiv:2204.11902*, 2022.
- [60] S. Lloyd. Least squares quantization in pcm. *IEEE transactions on information theory*, 28(2):129–137, 1982.
- [61] W.-Y. Loh. Classification and regression trees. *Wiley interdisciplinary reviews: data mining and knowledge discovery*, 1(1):14–23, 2011.
- [62] S. Lucas and T. Reynolds. Learning deterministic finite automata with a smart state labeling evolutionary algorithm. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 27(7):1063–1074, 2005.
- [63] J. MacQueen. Some methods for classification and analysis of multivariate observations. In *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability, Volume 1: Statistics*, volume 5, pages 281–298. University of California press, 1967.

- [64] C. J. Maddison, A. Mnih, and Y. W. Teh. The concrete distribution: A continuous relaxation of discrete random variables, 2017.
- [65] R. Manhaeve, S. Dumancic, A. Kimmig, T. Demeester, and L. De Raedt. Deepproblog: Neural probabilistic logic programming. In S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 31. Curran Associates, Inc., 2018.
- [66] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. Lillicrap, T. Harley, D. Silver, and K. Kavukcuoglu. Asynchronous methods for deep reinforcement learning. In *International conference on machine learning*, pages 1928–1937. PMLR, 2016.
- [67] A. Mordoch, E. Scala, R. Stern, and B. Juba. Safe learning of pddl domains with conditional effects. In *Proceedings of the International Conference on Automated Planning and Scheduling*, volume 34, pages 387–395, 2024.
- [68] J. Oncina and P. García. Inferring regular languages in polynomial update time. *World Scientific*, 01 1992.
- [69] K. Pearson. Contributions to the mathematical theory of evolution. *Philosophical Transactions of the Royal Society of London. A*, 185:71–110, 1894.
- [70] M. A. Peot and D. E. Smith. Conditional nonlinear planning. In *Proceedings of the First International Conference on AI Planning Systems (AIPS)*, pages 189–197, 1992.
- [71] A. Pnueli. The temporal logic of programs. In *18th Annual Symposium on Foundations of Computer Science, Providence, Rhode Island, USA, 31 October - 1 November 1977*, pages 46–57. IEEE Computer Society, 1977.
- [72] S. Richter and M. Westphal. The lama planner: Guiding cost-based anytime planning with landmarks. *Journal of Artificial Intelligence Research*, 39:127–177, 2010.
- [73] A. Ronca, G. P. Licks, and G. D. Giacomo. Markov abstractions for PAC reinforcement learning in non-markov decision processes. In *Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence, IJCAI 2022, Vienna, Austria, 23-29 July 2022*, pages 3408–3415, 2022.
- [74] S. J. Russell and P. Norvig. *Artificial Intelligence: a modern approach*. Pearson, 3 edition, 2009.

- [75] C. E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27(3):379–423, 1948.
- [76] T. Silver and R. Chitnis. Pddl-gym: Gym environments from pddl problems. In *International Conference on Automated Planning and Scheduling (ICAPS) PRL Workshop*, 2020.
- [77] S. Sreedharan and M. Katz. Optimistic exploration in reinforcement learning using symbolic model estimates. *Advances in Neural Information Processing Systems*, 36:34519–34535, 2023.
- [78] L. Steels et al. The symbol grounding problem has been solved. so what’s next. *Symbols and embodiment: Debates on meaning and cognition*, pages 223–244, 2008.
- [79] R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction*. The MIT Press, second edition, 2018.
- [80] M. Taddeo and L. Floridi. Solving the symbol grounding problem: a critical review of fifteen years of research. *Journal of Experimental & Theoretical Artificial Intelligence*, 17(4):419–445, 2005.
- [81] P.-N. Tan, M. Steinbach, M. A. Oshin, and V. Kumar. *Introduction to Data Mining*. Pearson, 05 2005.
- [82] M. Tomita. Dynamic construction of finite automata from examples using hill-climbing. In *Proceedings of the Fourth Annual Conference of the Cognitive Science Society*, pages 105–108, Ann Arbor, Michigan, 1982.
- [83] V. I. Ulyantsev, I. Zakirzyanov, and A. A. Shalyto. Bfs-based symmetry breaking predicates for dfa identification. In *Language and Automata Theory and Applications*, 2015.
- [84] E. Umili, F. Argenziano, A. Barbin, R. Capobianco, et al. Visual reward machines. In *Neural-Symbolic Learning and Reasoning 2022*, volume 3212, pages 255–267. 2023.
- [85] E. Umili and R. Capobianco. Deepdfa: a transparent neural network design for dfa induction, 04 2023.

- [86] E. Umili, R. Capobianco, and G. D. Giacomo. Grounding ltl specifications in images. In *Proceedings of the 16th International Workshop on Neural-Symbolic Learning and Reasoning as part of the 2nd International Joint Conference on Learning & Reasoning (IJCLR 2022), Cumberland Lodge, Windsor Great Park, UK, September 28-30, 2022*, pages 45–63, 2022.
- [87] T. Van Erven and P. Harremos. Rényi divergence and kullback-leibler divergence. *IEEE Transactions on Information Theory*, 60(7):3797–3820, 2014.
- [88] H. Walke, D. Ritter, C. Trimbach, and M. Littman. Learning finite linear temporal logic specifications with a specialized neural operator, 2021.
- [89] C. Wang, Y. Li, S. L. Smith, and J. Liu. Continuous motion planning with temporal logic specifications using deep neural networks, 2020.
- [90] A. G. Weber. The usc-sipi image database: Version 5. <http://sipi.usc.edu/database/>, 2006.
- [91] K. Xi, S. Gould, and S. Thiébaux. Neuro-symbolic learning of lifted action models from visual traces. In *34th International Conference on Automated Planning and Scheduling*, 2024.
- [92] Z. Xu, B. Wu, A. Ojha, D. Neider, and U. Topcu. Active finite reward automaton inference and reinforcement learning using queries and counterexamples. In *Machine Learning and Knowledge Extraction - 5th IFIP TC 5, TC 12, WG 8.4, WG 8.9, WG 12.9 International Cross-Domain Conference, CD-MAKE 2021, Virtual Event, August 17-20, 2021, Proceedings*, pages 115–135, 2021.
- [93] Y. Yuan, L. Wu, and X. Zhang. Gini-impurity index analysis. *IEEE Transactions on Information Forensics and Security*, 16:3154–3169, 2021.
- [94] L. A. Zadeh. Fuzzy logic. *Computer*, 21(4):83–93, 1988.
- [95] I. Zakirzyanov, A. Morgado, A. Ignatiev, V. I. Ulyantsev, and J. Marques-Silva. Efficient symmetry breaking for sat-based minimum dfa inference. In *LATA*, 2019.
- [96] H. H. Zhuo and Q. Yang. Action-model acquisition for planning via transfer learning. *Artificial intelligence*, 212:80–103, 2014.